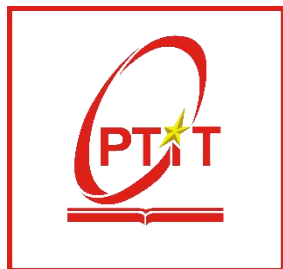


BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO ĐỒ ÁN
TỐT NGHIỆP ĐẠI HỌC

Đề tài: “XÂY DỰNG WEBSITE ĐẶT VÉ
XEM SỰ KIỆN THỂ THAO ”

Giáo viên hướng dẫn: ThS. Châu Văn Vân
Lớp: D21CQCNHT01-N
Thành viên nhóm: Triệu Quốc Đạt - N21DCCN016
Vũ Quốc Hoàng Anh - N21DCCN101
Khóa: 2021 – 2026
Ngành: Công nghệ thông tin
Hệ: Đại học Chính quy

TRIỆU QUỐC ĐẠT
VŨ QUỐC HOÀNG ANH

MSSV: N21DCCN016
MSSV: N21DCCN101

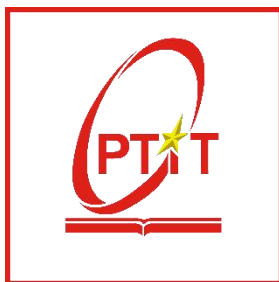
CHUYÊN NGÀNH: HỆ THỐNG THÔNG TIN 2021-2026
CHUYÊN NGÀNH: HỆ THỐNG THÔNG TIN 2021-2026

Lớp: D21CQCNHT01-N
Lớp: D21CQCNHT01-N

TP.
HCM
2025

TP.HCM, tháng 12, 2025

**BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

***Đề tài: “XÂY DỰNG WEBSITE ĐẶT VÉ
XEM SỰ KIỆN THỂ THAO ”***

Giáo viên hướng dẫn:	ThS. Châu Văn Vân
Lớp:	D21CQCNHT01-N
Thành viên nhóm:	Triệu Quốc Đạt - N21DCCN016 Vũ Quốc Hoàng Anh - N21DCCN101
Khóa:	2021 – 2026
Ngành:	Công nghệ thông tin
Hệ:	Đại học Chính quy

TP.HCM, tháng 12, 2025

LỜI CẢM ƠN

Trước tiên, nhóm em xin gửi lời cảm ơn chân thành và sâu sắc đến Thầy Châu Văn Vân – giảng viên Khoa Công nghệ Thông tin, người đã trực tiếp hướng dẫn, định hướng và hỗ trợ nhóm em trong suốt quá trình thực hiện đồ án tốt nghiệp. Thầy không chỉ tận tình góp ý về mặt kỹ thuật, mà còn truyền cảm hứng nghiên cứu và tinh thần nghiêm túc trong học tập, giúp nhóm em hoàn thiện đồ án một cách hiệu quả.

Nhóm em cũng xin cảm ơn Quý Thầy Cô trong Khoa Công nghệ Thông tin đã trang bị cho nhóm em những kiến thức nền tảng quý báu trong suốt quá trình học tập tại trường. Những kiến thức đó chính là hành trang quan trọng để nhóm em có thể vận dụng vào việc nghiên cứu và xây dựng đề tài này.

Cuối cùng, nhóm em xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, hỗ trợ và đồng hành cùng nhóm em trong suốt chặng đường học tập và hoàn thành đồ án tốt nghiệp.

Nhóm em xin chân thành cảm ơn!

PHÂN CÔNG NHIỆM VỤ

1. Triệu Quốc Đạt - N21DCCN016

Phát triển phân hệ :

- Admin & Employee : Quản lý sự kiện (giải đấu, sân, đội tuyển, trận đấu), quản lý vé, quản lý khuyến mãi, quản lý hoàn vé, phản hồi qua Email, AI dự báo doanh thu và khuyến nghị giá, AI khuyến nghị mức khuyến mãi, chức năng tích điểm dành cho khách hàng.
- Customer: Thực hiện quy trình đặt hàng, tích hợp cổng thanh toán MoMo, tính năng Redis giữ vé tạm thời và Websocket phát tín hiệu.

2. Vũ Quốc Hoàng Anh - N21DCCN101

Phát triển phân hệ:

- Admin & Employee : đăng ký/đăng nhập, quản lý nhân viên, quản lý khách hàng, quản lý đơn hàng, quản lý giao dịch ,quản lý báo cáo, giao diện DashBoard.
- Customer: Phát triển Chatbot AI (RAG + LLaMA) hỗ trợ khách hàng.

Mục lục

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT	vi
DANH SÁCH HÌNH ẢNH	viii
MỞ ĐẦU	1
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	5
1.1 Tổng quan về hệ thống bán vé	5
1.1.1 Mua vé trực tiếp	5
1.1.2 Mua vé trực tuyến	7
1.1.3 Về rủi ro và biến cố	8
1.2 Công nghệ phát triển ứng dụng Web	11
1.2.1 Frontend (HTML/CSS/JavaScript)	12
1.2.2 Hệ quản trị cơ sở dữ liệu MySQL	12
1.2.3 Tích hợp thanh toán trực tuyến MoMo	12
1.3 Công nghệ xử lý dữ liệu và thời gian thực	13
1.3.1 Redis và Cơ chế khóa tài nguyên (Resource Locking)	13
1.3.2 WebSocket và Cơ chế phát tín hiệu (Signaling)	14
1.4 Trí tuệ nhân tạo và học máy	15
1.4.1 Kiến trúc RAG và AI Agent (Ứng dụng cho Chatbot)	15
1.4.2 Các thuật toán dự báo và lý do chọn Random Forest	16
Chương 2: Thiết kế hệ thống	19
2.1 Phân tích yêu cầu người dùng và Đối tượng sử dụng	19
2.1.1 Yêu cầu chung	19
2.1.2 Yêu cầu cho Quản lý	19
2.1.3 Yêu cầu cho Nhân viên	21
2.1.4 Yêu cầu cho Khách hàng	21
2.1.5 Yêu cầu xử lý rủi ro và tích hợp	22
2.1.6 Yêu cầu báo cáo và phân quyền	23
2.2 Mô hình Use Case	23
2.2.1 Quản lý nhân viên	23
2.2.2 Chức năng đặt đơn cho khách hàng	25
2.2.3 Quản lý thông tin cá nhân của khách hàng	26
2.2.4 Khách hàng theo dõi đơn hàng	27

2.2.5 Khách hàng đăng nhập/đăng ký tài khoản	29
2.2.6 Quản lý trận đấu	31
2.2.7 Quản lý vé	32
2.2.8 Theo dõi đơn hàng	33
2.2.9 Đặt vé trực tuyến	35
2.3 Thiết kế cơ sở dữ liệu	36
2.3.1 Diagram	36
2.3.2 Các bảng thực thể	36
2.4 Thiết kế thuật toán AI - AGENT và học máy	44
2.4.1 Thiết kế Hệ thống Chatbot (Hybrid RAG & AI Agent)	44
2.4.2 Thiết kế Mô hình Dự báo Giá (Price Prediction Model)	45
Chương 3: Triển khai hệ thống	47
3.1 Môi trường cài đặt và công cụ phát triển	47
3.1.1 Phần cứng và Hệ điều hành	47
3.1.2 Môi trường phát triển ứng dụng (Application Stack)	47
3.1.3 Môi trường tích hợp thanh toán (Payment Gateway)	47
3.1.4 Môi trường phát triển Trí tuệ nhân tạo (AI Ecosystem)	48
3.1.5 Công cụ quản lý và triển khai (DevOps Tools)	49
3.2 Các chức năng chính	49
3.2.1 Chức năng dành cho Quản lý	49
3.2.2 Chức năng dành cho Nhân viên	49
3.2.3 Chức năng dành cho Khách hàng	50
3.2.4 Chức năng kỹ thuật, vận hành và tích hợp	50
3.2.5 Báo cáo và thống kê	51
3.3 Trí tuệ nhân tạo	51
3.3.1 Học máy - dự báo doanh thu và khuyến nghị giá	51
3.3.2 CHATBOT RAG + AI AGENT	57
CHƯƠNG 4: KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ	62
4.1.1 Màn hình Dashboard	62
4.1.2 Màn hình quản lý vé	62
4.1.3 Màn hình quản lý nhân viên	63
4.1.4 Màn hình quản lý thông tin khách hàng	64

4.1.5 Màn hình quản lý đơn hàng	64
4.1.6 Màn hình quản lý giải đấu	64
4.1.7 Màn hình quản lý các đội	65
4.1.8 Màn hình quản lý sân vận động	65
4.1.9 Màn hình quản lý trận đấu	66
4.1.10 Màn hình quản lý khuyến mãi	67
4.1.11 Màn hình quản lý hoàn vé	67
4.1.12 Màn hình báo cáo	68
4.1.13 Màn hình quản lý các giao dịch	68
4.1.14 Màn hình quản lý khách hàng	69
4.1.15 Màn hình quản lý đơn hàng	69
4.1.16 Giao diện người dùng và Chatbot AI	70
4.5.1 Mục tiêu kiểm thử	74
4.5.2 Quy trình nghiệp vụ (Logic hệ thống)	74
4.5.3 Đánh giá kết quả	76
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	77
TÀI LIỆU THAM KHẢO	79

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký hiệu / Viết tắt	Tên đầy đủ	Giải thích
AI	Artificial Intelligence	Trí tuệ nhân tạo, được dùng trong Chatbot hỗ trợ khách hàng.
API	Application Programming Interface	Giao diện lập trình ứng dụng, dùng để giao tiếp giữa Client và Server.
DB	Database	Cơ sở dữ liệu của hệ thống.
RDBMS	Relational Database Management System	Hệ quản trị cơ sở dữ liệu quan hệ (MySQL).
BCTN	Báo cáo thực tập tốt nghiệp	Tài liệu báo cáo của sinh viên.
CRUD	Create – Read – Update – Delete	Các thao tác chính trên dữ liệu.
JWT	JSON Web Token	Chuẩn bảo mật dùng cho xác thực người dùng.
UI	User Interface	Giao diện người dùng.
UX	User Experience	Trải nghiệm người dùng.
HTML	HyperText MarkupLanguage	Ngôn ngữ đánh dấu tạo trang web.
CSS	Cascading Style Sheets	Ngôn ngữ dùng để định dạng giao diện web.
JS	JavaScript	Ngôn ngữ lập trình chạy trên trình duyệt.
MVC	Model – View – Controller	Mô hình kiến trúc ứng dụng web.
ML	Machine Learning	Học máy, dùng để dự báo giá vé.
RAG	Retrieval Augmented Generation	Kỹ thuật AI dùng trong Chatbot.
LLaMA	Large Language Model Meta AI	Mô hình ngôn ngữ lớn dùng cho Chatbot.
OTP	One-Time Password	Mã xác thực một lần, dùng khi đăng ký/đăng nhập.

Ký hiệu / Viết tắt	Tên đầy đủ	Giải thích
QR	Quick Response Code	Mã QR in trên vé xem thể thao.
IPN	Instant Payment Notification	Thông báo thanh toán từ MoMo trả về hệ thống.
MoMo	Mobile Money	Ví điện tử dùng để thanh toán online.
HTTP	HyperText Transfer Protocol	Giao thức truyền tải dữ liệu web.
REST	Representational State Transfer	Phong cách thiết kế API phổ biến.
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi giữa client – server.
SSD	Solid State Drive	Bộ nhớ lưu trữ tốc độ cao.
CPU	Central Processing Unit	Bộ xử lý trung tâm.
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên.
Redis	Remote Dictionary Server	Bộ nhớ đệm in-memory, dùng để giữ ghế tạm thời & real-time ticketing.
TTL	Time To Live	Số giây dữ liệu tồn tại trong Redis.
WS	WebSocket	Công nghệ truyền dữ liệu thời gian thực.
IDE	Integrated Development Environment	Môi trường lập trình (VS Code, IntelliJ,...).
PDF	Portable Document Format	Định dạng tài liệu dùng để xuất báo cáo.
UML	Unified Modeling Language	Ngôn ngữ mô tả mô hình hệ thống.
ERD	Entity Relationship Diagram	Sơ đồ quan hệ thực thể.
AES	Advanced Encryption Standard	Thuật toán mã hoá mật khẩu, token,...

DANH SÁCH HÌNH ẢNH

Hình 1: Mô hình Linear Regression	16
Hình 2: Mô hình Prophet	17
Hình 3: Usecase Quản lý nhân viên	23
Hình 4: Usecase Đặt vé offline cho khách hàng	25
Hình 5: Usecase Quản lý thông tin cá nhân của khách hàng	26
Hình 6: Usecase Khách hàng theo dõi đơn hàng của mình	27
Hình 7: Usecase Khách hàng đăng nhập, đăng ký tài khoản	29
Hình 8: Usecase Quản lý trận đấu	31
Hình 9: Usecase Quản lý vé	32
Hình 10: Usecase Quản lý đơn hàng	33
Hình 11: Usecase Đặt vé trực tuyến	35
Hình 12: Sơ đồ quan hệ cơ sở dữ liệu	36
Bảng 1: Kết quả so sánh giữa 3 mô hình	53
Hình 13: Giao diện AI khuyến nghị giá	56
Hình 14: Màn hình Các chỉ số thống kê	62
Hình 15: Màn hình quản lý vé	62
Hình 16: Màn hình tạo vé	63
Hình 17: Thao tác tạo vé cho trận đấu	63
Hình 18: Màn hình Quản lý nhân viên	63
Hình 19: Màn hình Quản lý thông tin khách hàng	64
Hình 20: Màn hình Quản lý đơn hàng	64
Hình 21: Màn hình Quản lý danh sách các giải đấu	64
Hình 22: Màn hình Danh sách các đội	65
Hình 23: Màn hình Danh sách các sân vận động	65
Hình 24: Danh sách khu vực của từng sân	65
Hình 25: Màn hình Danh sách các trận đấu	66
Hình 26: Màn hình thêm lịch thi đấu	66
Hình 27: Màn hình Danh sách các đợt khuyến mãi	67
Hình 28: Màn hình quản lý hoàn vé	67
Hình 29: Màn hình Báo cáo theo doanh thu	68
Hình 30: Màn hình Quản lý các giao dịch	68
Hình 31: Màn hình Quản lý thông tin khách hàng	69
Hình 32: Màn hình Quản lý đơn hàng	69
Hình 33: Giao diện người dùng và Chatbot hỗ trợ khách hàng	70
Hình 34: Màn hình email khách hàng nhận được sau khi được hoàn vé thành công ..	71
Hình 35: Giữ ghế tạm thời cho khách hàng	72
Hình 36: Màn hình thanh toán bằng MoMo	73
Hình 37: Màn hình sau khi thanh toán MoMo thành công	74

Hình 38: Khách hàng được sử dụng điểm tích được khi đặt hàng	75
Hình 39: Màn hình thông tin điểm thưởng của khách hàng	75

MỞ ĐẦU

1. Lý do chọn đề tài

Ngày nay, các môn thể thao như bóng đá, bóng chuyền, bóng rổ,.. không chỉ đơn thuần là những môn thể thao mà còn là niềm đam mê của hàng triệu người hâm mộ trên thế giới. Việc xem trực tiếp không chỉ mang lại nhiều trải nghiệm thú vị mà còn mang cảm giác cho người hâm mộ đến gần hơn với môn thể thao và cầu thủ yêu thích của mình.

Tuy nhiên, quá trình đặt vé để xem các trận đấu trực tiếp thường gặp nhiều khó khăn như phải xếp hàng dài để mua vé, quá tải hệ thống tại sân vận động hoặc thiếu thông tin đầy đủ và kịp thời về các trận đấu. Điều này tạo ra không ít khó khăn và cản trở cho người hâm mộ trong việc tiếp cận các trận đấu mà họ mong muốn.

Trong bối cảnh công nghệ phát triển mạnh mẽ, việc xây dựng một nền tảng trực tuyến chuyên nghiệp để tối ưu hóa quy trình đặt vé trở nên cần thiết hơn bao giờ hết. Dự án “Website đặt vé xem sự kiện thể thao” ra đời nhằm cung cấp một nền tảng trực tuyến tiện lợi, giúp người hâm mộ có thể dễ dàng tìm kiếm thông tin về các trận đấu, đặt chỗ và thanh toán vé xem thể thao mọi lúc mọi nơi.

Đề tài nghiên cứu này tập trung vào việc xây dựng và quản lý hệ thống bán vé cho các sự kiện thể thao, đặc biệt là các trận đấu bóng đá, bóng chuyền,... trên toàn quốc. Hệ thống bán vé sẽ giúp đơn giản hóa quy trình quản lý bán vé, từ việc phân phối vé, thanh toán, giao vé cho đến xử lý các yêu cầu hoàn hoặc đổi vé sau khi bán. Với việc ứng dụng công nghệ, hệ thống không chỉ phục vụ công tác bán vé trực tiếp mà còn cho phép khách hàng mua vé trực tuyến một cách thuận tiện và hiệu quả.

Đề tài "Xây dựng website đặt vé xem các sự kiện thể thao" được lựa chọn nghiên cứu và thực hiện nhằm giải quyết các vấn đề trên thông qua việc áp dụng các công nghệ tiên tiến:

- Django và MySQL: Đảm bảo tính ổn định, bảo mật và khả năng mở rộng của hệ thống backend và quản trị dữ liệu.
- Redis và WebSocket: Giải quyết bài toán hiệu năng và trải nghiệm thời gian thực (Real-time), cho phép thông báo đến khách hàng trạng thái ghế ngồi ngay lập tức, tránh tình trạng xung đột đơn hàng.
- AI Chatbot & NLP: Tự động hóa quy trình chăm sóc khách hàng, giúp giải đáp thắc mắc nhanh chóng và giảm tải cho nhân sự vận hành.

Machine Learning (Random Forest, Linear Regression, Prophet): Cung cấp công cụ phân tích, dự báo xu hướng đặt vé, hỗ trợ ra quyết định kinh doanh dựa trên dữ liệu (Data-driven decision making). Đề tài không chỉ dừng lại ở việc xây dựng một website thương mại điện tử thông thường mà còn hướng đến một giải pháp công nghệ toàn diện, đáp ứng nhu cầu khắt khe của thị trường vé sự kiện hiện đại.

2. Mục tiêu và phạm vi nghiên cứu

- Mục tiêu của đề tài

Đề tài hướng tới các mục tiêu cụ thể sau:

Về mặt hệ thống:

- Xây dựng thành công hệ thống website đặt vé sự kiện thể thao hoàn chỉnh với đầy đủ các chức năng: tìm kiếm, đặt vé, thanh toán và quản lý khách hàng, đơn hàng, doanh số,...

Về mặt công nghệ:

- Làm chủ và ứng dụng framework Django kết hợp với cơ sở dữ liệu quan hệ MySQL để xây dựng cấu trúc dữ liệu chặt chẽ.
- Ứng dụng Redis để caching (lưu trữ tạm thời) nhằm tăng tốc độ truy xuất dữ liệu và giảm tải cho Database chính.
- Xây dựng cơ chế "thông báo tức thì" (Broadcasting) sử dụng WebSocket, đóng vai trò như một hệ thống loa phát thanh kỹ thuật số.

Về mặt Trí tuệ nhân tạo (AI) và Khoa học dữ liệu:

- Trợ lý ảo thông minh (AI Agent & RAG): Thay thế các chatbot kịch bản truyền thống bằng mô hình AI Agent kết hợp kỹ thuật RAG (Retrieval-Augmented Generation). Chatbot có khả năng truy xuất dữ liệu động từ hệ thống (lịch thi đấu, giá vé, quy định hoàn vé) để trả lời khách hàng chính xác theo ngữ cảnh thực tế.
- Tối ưu hóa doanh thu (AI Prediction): Ứng dụng thuật toán Random Forest để phân tích mối quan hệ phi tuyến tính giữa các yếu tố (thời gian, độ hot trận đấu, vị trí ghế) và doanh thu. Từ đó, hệ thống đưa ra khuyến nghị mức giá bán hợp lý nhất cho Admin để tối đa hóa lợi nhuận.

Về mặt nghiệp vụ thương mại:

- Tích hợp cổng thanh toán điện tử MoMo, mang lại trải nghiệm thanh toán không tiền mặt nhanh chóng và tiện lợi.
- Xây dựng quy trình nghiệp vụ hoàn vé tự động chặt chẽ: Hệ thống tự động cập nhật lại trạng thái ghế vào kho vé, và gửi Email thông báo kết quả cho khách hàng.

- Phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Quy trình đặt sự kiện thể thao, công nghệ WebSocket, Redis, kiến trúc RAG trong AI, và thuật toán Random Forest.

Phạm vi chức năng:

- Phân hệ Khách hàng: Tìm kiếm sự kiện, Đặt vé (Real-time), Thanh toán qua MoMo (môi trường Sandbox), Yêu cầu hoàn vé, Nhận thông báo Email, Chat với AI Agent.
- Phân hệ Quản trị (Admin): Quản lý sự kiện, quản lý nhân viên và khách hàng, cấu hình hoàn vé, xem Dashboard doanh thu, nhận dự báo và khuyến nghị giá từ AI, quản lý vé và khuyến mãi.

Giới hạn đề tài:

- Các giao dịch thanh toán MoMo thực hiện trên môi trường thử nghiệm (Testnet).
- Dữ liệu huấn luyện cho mô hình Random Forest được tổng hợp từ dữ liệu giả lập dựa trên các quy luật kinh tế thực tế của thị trường vé thể thao.
- Chưa thể tích hợp bán vé cho nhiều môn thể thao mà chỉ một vài môn thể thao nhất định.

3. Ý nghĩa khoa học và thực tiễn của đề tài**Ý nghĩa khoa học**

- Ứng dụng kỹ thuật RAG và AI Agent: Đề tài nghiên cứu cách khắc phục nhược điểm "ảo giác" (hallucination) của các mô hình ngôn ngữ lớn (LLM) bằng cách kết hợp với cơ sở dữ liệu nội bộ thông qua RAG, giúp AI trả lời chính xác các thông tin đặc thù của doanh nghiệp.

- Nâng cao độ chính xác trong dự báo: Việc lựa chọn Random Forest thay vì các mô hình tuyến tính (Linear Regression) thể hiện tư duy khoa học trong việc xử lý dữ liệu phức tạp. Random Forest có khả năng học các quy tắc định giá phân tầng (Tiered Pricing) và các yếu tố phi tuyến tính, mang lại kết quả dự báo sát thực tế hơn.

- Sử dụng cơ chế TTL (Time-To-Live) của Redis để thực hiện kỹ thuật khóa tạm thời (Temporary Locking), giải quyết bài toán giữ ghế trong khoảng thời gian ngắn (2-3 phút) và tự động thu hồi tài nguyên, ngăn chặn triệt để xung đột dữ liệu (Race Condition).

- Sử dụng WebSocket đóng vai trò lớp phát tín hiệu (Signaling Layer), đảm bảo tính nhất quán hiển thị (View Consistency) giữa tất cả người dùng trong thời gian thực.

Ý nghĩa thực tiễn

- Tăng cường trải nghiệm khách hàng (CX):

- Khách hàng không còn gặp tình trạng "đặt hụt" hay lỗi hệ thống do xung đột dữ liệu nhờ cơ chế Real-time.
- Quy trình hoàn vé minh bạch và tự động giúp tăng niềm tin của khách hàng vào dịch vụ.

- Công cụ hỗ trợ ra quyết định (Decision Support System): Hệ thống không chỉ là nơi bán vé mà còn là "trợ lý" cho nhà quản lý. Thay vì định giá theo cảm tính, Admin được AI gợi ý mức giá tối ưu dựa trên dữ liệu, giúp cân bằng giữa tỷ lệ lấp đầy sân (Fill rate) và doanh thu.
- Tiết kiệm chi phí vận hành: Chatbot AI Agent hoạt động 24/7 giúp giảm tải đáng kể cho bộ phận chăm sóc khách hàng, đặc biệt là trong các đợt mở bán cao điểm.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1 Tổng quan về hệ thống bán vé

1.1.1 Mua vé trực tiếp

- Tiếp đón và tư vấn

Khi người hâm mộ đến quầy bán vé, nhân viên sẽ chào đón và hỏi về nhu cầu của khách hàng, chẳng hạn như trận đấu họ muốn xem hoặc các yêu cầu đặc biệt về vị trí ghế ngồi. Để phục vụ khách hàng tốt nhất, nhân viên sẽ yêu cầu khách cung cấp một số thông tin cơ bản như loại giải đấu, tên đội tuyển hay ngày giờ tổ chức trận đấu mà họ muốn tham gia. Sau đó, nhân viên sẽ cung cấp cho khách hàng thông tin chi tiết về các trận đấu hiện có, giúp họ lựa chọn trận đấu phù hợp.

- Chọn vé và khu vực ghế ngồi

Một trong những yếu tố quan trọng trong quá trình mua vé là lựa chọn vị trí ghế ngồi. Nhân viên sẽ cung cấp cho khách hàng sơ đồ ghế ngồi để họ có thể dễ dàng hình dung về vị trí của các khu vực trong sân vận động. Người mua có thể chọn ghế ngồi theo sở thích, ví dụ như chọn gần sân đấu để có trải nghiệm gần gũi hơn với các vận động viên, hoặc chọn các khu vực có tầm nhìn đẹp và thoải mái hơn. Các khu vực khác nhau sẽ có các mức giá khác nhau. Để thuận tiện hơn, nhân viên có thể hỗ trợ khách hàng tìm kiếm các khu vực ghế còn trống và giải đáp các thắc mắc về sự sắp xếp các khu vực ghế.

- Nhập thông tin và xử lý thanh toán

Sau khi người mua vé đã quyết định lựa chọn trận đấu và khu vực ghế ngồi, nhân viên sẽ tiến hành nhập các thông tin cần thiết vào hệ thống máy tính để xử lý giao dịch. Các thông tin này bao gồm số lượng vé mà khách hàng muốn mua, loại vé (vé khu VIP, A, B,...), và vị trí ghế ngồi mà họ đã chọn. Hệ thống sẽ tính toán tổng số tiền phải thanh toán, sau đó nhân viên sẽ thông báo cho khách hàng về giá trị của vé.

Khách hàng có thể thanh toán theo các phương thức khác nhau, tùy thuộc vào quy định của sân vận động. Khách hàng có thể thanh toán bằng tiền mặt tại quầy bán vé. Trong trường hợp khách hàng muốn thanh toán nhanh chóng và tiện lợi hơn, họ cũng có thể sử dụng thẻ tín dụng hoặc thẻ ghi nợ. Nhân viên sẽ hỗ trợ khách hàng trong việc sử dụng các thiết bị thanh toán, bao gồm việc quét thẻ hoặc nhập thông tin thẻ một cách bảo mật.

- In vé và cung cấp thông tin chi tiết

Khi giao dịch thanh toán hoàn tất, nhân viên sẽ lập tức in vé ngay tại quầy bán vé. Vé được in sẽ có các thông tin quan trọng như mã vé, vị trí ghế ngồi đã được chọn, đội tuyển tham gia và thông tin chi tiết về ngày giờ diễn ra trận đấu. Đặc biệt, mỗi vé sẽ được in một mã QR code, mã này sẽ được quét khi khách hàng vào sân vận động, giúp quá trình kiểm tra vé diễn ra nhanh chóng và chính xác.

- Hướng dẫn và kết thúc giao dịch

Sau khi giao dịch hoàn tất và vé đã được cung cấp, nhân viên sẽ hướng dẫn khách hàng về quy trình vào sân, cách sử dụng mã QR code khi kiểm tra vé tại cổng vào. Đồng thời, nếu khách hàng có bất kỳ thắc mắc nào về việc tham gia sự kiện, nhân viên sẽ trả lời và cung cấp các thông tin bổ sung về sân vận động, các tiện ích như nhà vệ sinh, khu vực ăn uống, hoặc bất kỳ dịch vụ hỗ trợ nào khác.

- Chính sách hoàn trả:

a. Hoàn trả vé:

Trận đấu bị hủy: Nếu trận đấu không thể diễn ra vì lý do bất khả kháng (chẳng hạn như thời tiết xấu, sự cố về sân vận động, hoặc các yếu tố khác ngoài ý muốn), khách hàng có thể yêu cầu hoàn trả vé đã mua.

Thay đổi lịch trình thi đấu: Nếu trận đấu bị dời lịch sang một thời gian khác, khách hàng có quyền yêu cầu hoàn trả vé nếu không thể tham gia vào ngày mới.

Vé bị lỗi hoặc không hợp lệ: Nếu vé có sự cố như mã vé sai, thông tin vé không chính xác hoặc không hợp lệ, khách hàng có thể yêu cầu hoàn trả hoặc đổi vé mới.

b. Đổi trả vé:

Trận đấu bị hủy hoặc thay đổi lịch: Khách hàng có thể yêu cầu đổi vé sang trận đấu khác hoặc hoàn trả vé theo yêu cầu. Điều này sẽ được xử lý tùy vào chính sách cụ thể của từng sự kiện.

Vé bị lỗi hoặc không hợp lệ: Trong trường hợp này, khách hàng có thể yêu cầu đổi vé mới với thông tin chính xác, hoặc hoàn trả vé nếu không thể sử dụng vé bị lỗi.

c. Thủ tục hoàn, đổi trả vé:

Khách hàng phải cung cấp thông tin vé đã mua, bao gồm mã vé và lý do đổi trả khi yêu cầu hoàn trả hoặc đổi vé.

Đối với vé mua trực tuyến, vé sẽ được hoàn trả qua hình thức thanh toán ban đầu (thẻ tín dụng, thẻ ghi nợ, hoặc chuyển khoản ngân hàng). Vé mua trực tiếp có thể được hoàn trả tại quầy bán vé.

Trong một số trường hợp, sẽ có yêu cầu khách hàng gửi lại vé qua email hoặc đến quầy bán vé để xử lý hoàn trả hoặc đổi vé.

Lưu ý:

Chính sách hoàn, đổi trả vé có thể thay đổi tùy vào sự kiện hoặc các điều kiện đặc biệt, vì vậy người hâm mộ cần kiểm tra các quy định cụ thể trước khi mua vé.

Các vé đã qua sử dụng hoặc không hợp lệ (vì lý do khác) sẽ không được hoàn trả hoặc đổi vé.

1.1.2 Mua vé trực tuyến

- Đăng ký tài khoản và cung cấp thông tin cá nhân

Bước đầu tiên để mua vé trực tuyến là người dùng cần đăng ký tài khoản trên hệ thống bán vé trực tuyến của sân vận động. Quá trình đăng ký yêu cầu người dùng cung cấp các thông tin cá nhân cơ bản như họ tên, số điện thoại, email, tên đăng nhập, và mật khẩu. Các thông tin này sẽ được sử dụng để xác thực tài khoản, đảm bảo tính bảo mật và giúp hệ thống có thể liên lạc hay thông báo với khách hàng trong trường hợp cần thiết.

Khi đã hoàn tất việc đăng ký và tạo tài khoản, người dùng sẽ nhận được email xác nhận đăng ký và kích hoạt tài khoản. Việc này giúp đảm bảo rằng tài khoản của người dùng là hợp lệ và có thể sử dụng để thực hiện các giao dịch mua vé sau này. Sau khi tài khoản được kích hoạt, người dùng sẽ có thể sử dụng tên đăng nhập và mật khẩu để đăng nhập vào hệ thống bán vé trực tuyến.

- Đăng nhập và lựa chọn trận đấu

Sau khi đã đăng nhập thành công vào hệ thống bán vé trực tuyến, người dùng sẽ có thể dễ dàng xem các trận đấu sắp tới. Các trận đấu này thường được hiển thị theo lịch trình rõ ràng, với thông tin chi tiết về ngày giờ tổ chức, giải đấu, vòng loại và các đội tham gia. Tại giao diện mua vé trực tuyến, người dùng có thể lọc các trận đấu theo các tiêu chí như đội bóng yêu thích, thời gian diễn ra trận đấu hoặc loại giải đấu mà họ quan tâm.

- Lựa chọn vé và khu vực ghế ngồi

Sau khi đã chọn được trận đấu mà mình muốn tham gia, người dùng tiếp tục chọn vé theo khu vực ghế ngồi. Hệ thống sẽ hiển thị sơ đồ chi tiết, giúp người mua dễ dàng hình dung về vị trí của các khu vực trong sân vận động. Sơ đồ này sẽ thể hiện các khu vực ghế còn trống, các khu vực VIP, A, B,.. các dãy ghế có giá cao hoặc thấp tùy vào sự sắp xếp của sân vận động.

Người dùng có thể chọn chỗ ngồi theo sở thích của mình, ví dụ như chọn khu gần sân đấu để có trải nghiệm gần gũi với các vận động viên hoặc chọn khu vực có tầm nhìn đẹp hơn dù có thể phải chi trả một mức giá cao hơn. Sau khi lựa chọn vị trí, người dùng sẽ thấy tổng số vé cần mua và tổng số tiền phải thanh toán cho giao dịch.

- Kiểm tra thông tin vé và giá vé

Trước khi tiến hành thanh toán, hệ thống sẽ yêu cầu người dùng kiểm tra lại tất cả thông tin về vé, số lượng vé và mức giá vé đã chọn. Đây là bước quan trọng để đảm bảo rằng người dùng không mắc phải sai sót trong quá trình lựa chọn vé. Thông tin này sẽ bao gồm các chi tiết như tên trận đấu, đội tham gia, ngày giờ tổ chức, số lượng vé, loại vé, và các thông tin về khu vực ghế ngồi. Sau khi kiểm tra và xác nhận thông tin, người dùng có thể tiếp tục chuyển sang bước thanh toán.

- Chọn phương thức thanh toán và thực hiện giao dịch

Hệ thống hỗ trợ nhiều phương thức thanh toán đa dạng, giúp người dùng lựa chọn phương thức thuận tiện nhất cho mình. Các phương thức thanh toán phổ biến bao gồm thẻ tín dụng, thẻ ghi nợ, chuyển khoản ngân hàng trực tuyến, hoặc các cổng thanh toán điện tử khác như ví điện tử (PayPal, Visa, v.v.).

Khi người dùng đã chọn phương thức thanh toán, hệ thống sẽ yêu cầu nhập thông tin thẻ tín dụng hoặc thẻ ghi nợ nếu thanh toán qua thẻ, hoặc thông tin tài khoản ngân hàng nếu thực hiện thanh toán qua chuyển khoản. Hệ thống sẽ xử lý giao dịch và xác nhận thanh toán một cách nhanh chóng và bảo mật. Trong trường hợp thanh toán qua cổng trực tuyến, người dùng có thể sử dụng mã OTP hoặc các phương thức xác thực hai yếu tố để hoàn tất giao dịch.

- Nhận vé và mã QR code qua email

Sau khi thanh toán thành công, vé sẽ được gửi tự động qua email của người dùng. Email này sẽ chứa các thông tin chi tiết về vé, bao gồm mã vé, vị trí ghế ngồi, thông tin trận đấu, và đặc biệt là mã QR code. Mã QR code này là yếu tố quan trọng trong việc vào sân vận động, khi người hâm mộ cần xuất trình mã QR tại các cổng kiểm tra vé để vào sân. Mỗi mã QR code chỉ có hiệu lực cho một vé cụ thể, giúp hệ thống kiểm soát chính xác và nhanh chóng người tham gia sự kiện.

- Theo dõi lịch sử mua vé và thay đổi thông tin cá nhân

Người dùng có thể dễ dàng theo dõi lịch sử mua vé của mình thông qua tài khoản cá nhân. Trong trường hợp người dùng muốn thay đổi thông tin cá nhân (ví dụ: sửa đổi địa chỉ email, số điện thoại, hoặc mật khẩu), họ có thể dễ dàng thực hiện các thay đổi này ngay trên trang cá nhân của mình. Hệ thống cũng cho phép người dùng kiểm tra lịch sử các vé đã mua và các giao dịch trước đó để dễ dàng quản lý.

1.1.3 Về rủi ro và biến cố

- Lỗi hệ thống và sự cố kỹ thuật

Mô tả: Các sự cố như hệ thống không thể truy cập, lỗi trong quá trình thanh toán, hoặc vé không hiển thị chính xác trên hệ thống có thể xảy ra.

Cách xử lý: Nhân viên kỹ thuật cần phải theo dõi hệ thống liên tục và khắc phục sự cố khi có lỗi phát sinh. Nếu gặp sự cố, thông báo ngay đến khách hàng qua email hoặc tin nhắn để họ nắm được tình hình và có phương án dự phòng (như thay đổi phương thức thanh toán, đặt vé lại, v.v.). Hệ thống cần có kế hoạch khôi phục và sao lưu dữ liệu để tránh mất mát thông tin.

- Hết vé hoặc bán quá số lượng quy định

Mô tả: Có thể xảy ra trường hợp bán vé quá số lượng quy định do sự cố hệ thống hoặc nhân viên không kịp cập nhật số lượng vé còn lại.

Cách xử lý: Nhân viên phải cập nhật số lượng vé còn lại sau mỗi giao dịch hoặc theo dõi chặt chẽ tình trạng vé trong thời gian thực. Nếu khách hàng đặt vé khi số lượng vé đã hết, cần thông báo ngay lập tức và đưa ra các phương án thay thế như mời khách hàng chọn trận đấu khác hoặc nhận hoàn tiền. Nếu khách hàng bị ảnh hưởng bởi tình trạng này, thực hiện hoàn tiền nhanh chóng và đầy đủ.

- Khách hàng đến muộn và không thể vào trận đấu

Mô tả: Một số khách hàng có thể đến muộn và không thể vào sân để tham gia trận đấu.

Cách xử lý: Cung cấp thông tin rõ ràng về thời gian mở cửa, thời gian trận đấu bắt đầu trên website và các thông báo trước trận đấu. Nếu khách hàng đến muộn, có thể nhân viên bán vé cần kiểm tra tình hình và thông báo nếu có khả năng vào sân (ví dụ: nếu trận đấu chỉ mới bắt đầu khoảng 15 phút). Nếu không thể vào, nhân viên cần giải thích và đưa ra các hướng xử lý, như hoàn trả vé nếu có chính sách.

- Thay đổi lịch trình trận đấu

Mô tả: Trận đấu có thể bị hoãn hoặc thay đổi thời gian tổ chức do lý do khách quan (thời tiết xấu, vấn đề an ninh,...) hoặc tổ chức (chính sách thay đổi, sự cố từ đội bóng).

Cách xử lý: Cập nhật thông tin về sự thay đổi lịch trình ngay lập tức trên website và gửi thông báo qua email hoặc tin nhắn đến khách hàng đã mua vé. Cung cấp chính sách đổi vé dễ dàng cho khách hàng nếu trận đấu bị thay đổi hoặc hủy bỏ. Đảm bảo rằng khách hàng có thể chọn ngày giờ mới cho trận đấu hoặc đổi sang trận đấu khác. Đảm bảo khách hàng có thể yêu cầu hoàn tiền đầy đủ nếu họ không thể tham gia trận đấu vào thời gian mới.

- Sự cố về giá vé và khuyến mãi

Mô tả: Có thể xảy ra trường hợp giá vé sai lệch hoặc không áp dụng đúng chương trình khuyến mãi.

Cách xử lý: Nhân viên bán vé và quản lý cần xác minh kỹ lưỡng các thông tin trước khi đăng tải trên hệ thống. Đảm bảo rằng tất cả các khuyến mãi, giảm giá đều rõ ràng và chính xác. Nếu phát hiện sai sót về giá vé hoặc khuyến mãi, cần thông báo kịp thời cho khách hàng và cung cấp giải pháp (hoàn lại phần chênh lệch, áp dụng khuyến mãi đúng). Nếu xảy ra sai sót liên quan đến giá vé, cần thực hiện các bước để đảm bảo quyền lợi khách hàng (ví dụ: trả lại số tiền thừa cho khách, hoàn lại vé nếu giá không đúng).

- Khách hàng yêu cầu đổi, trả vé không hợp lệ

Mô tả: Một số khách hàng có thể yêu cầu đổi hoặc trả vé không hợp lệ (vì lý do cá nhân, không phải vì lý do chính đáng như hủy trận đấu).

Cách xử lý: Công ty cần có một chính sách rõ ràng về việc đổi trả vé, bao gồm những trường hợp hợp lệ và không hợp lệ. Nhân viên cần kiểm tra kỹ lý do yêu cầu đổi trả,

nếu lý do không hợp lệ thì cần thông báo cho khách hàng biết và giải thích rõ ràng. Nếu trường hợp hợp lệ, cung cấp hỗ trợ hoàn tiền hoặc đổi vé nhanh chóng. Nếu không hợp lệ, cần thông báo rõ ràng để tránh hiểu lầm.

- Gian lận trong mua bán vé

Mô tả: Có thể xảy ra tình trạng mua vé giả mạo hoặc vé được bán lại với giá cao hơn (spamming vé) gây ảnh hưởng đến khách hàng và uy tín hệ thống.

Cách xử lý: Hệ thống cần có tính năng kiểm tra vé điện tử để xác nhận tính hợp lệ của vé khi khách hàng đến quầy hoặc khi vào sân. Xây dựng các biện pháp bảo mật như mã QR độc đáo, hệ thống vé điện tử không thể sao chép hoặc làm giả. Nếu phát hiện vé giả, yêu cầu khách hàng cung cấp thông tin chi tiết để xử lý (hoàn lại tiền nếu là vé hợp pháp và xử lý các bên liên quan nếu có hành vi gian lận).

- Sự cố an ninh

Mô tả: Các sự cố về an ninh, ví dụ như sự cố tại sân vận động (với đám đông, các hành vi gây rối hoặc có mối nguy hiểm).

Cách xử lý: Cập nhật thông tin an ninh về quy trình ra vào sân, kiểm tra vé và túi xách trước khi vào. Trong trường hợp có sự cố, thông báo kịp thời cho người hâm mộ và phối hợp với các cơ quan chức năng để giải quyết nhanh chóng. Sẵn sàng có kế hoạch sơ tán nếu xảy ra tình huống nguy hiểm, và đảm bảo có phương án hỗ trợ khách hàng kịp thời.

1.2 Công nghệ phát triển ứng dụng Web

Để đáp ứng yêu cầu về tính tương tác cao (Real-time) và khả năng mở rộng linh hoạt, đề tài lựa chọn Kiến trúc Client-Server (Mô hình tách biệt Backend - Frontend). Trong đó, Backend đóng vai trò cung cấp dịch vụ dữ liệu thông qua chuẩn RESTful API, còn Frontend chịu trách nhiệm hiển thị và tương tác người dùng.

Kiến trúc tổng thể của hệ thống

Hệ thống được chia làm hai phân hệ giao tiếp với nhau qua giao thức HTTP/HTTPS và WebSocket:

- Backend (Server-side): Sử dụng Framework Django kết hợp với MySQL để quản lý logic nghiệp vụ, xử lý thanh toán và lưu trữ dữ liệu. Backend không trực tiếp trả về giao diện HTML mà trả về dữ liệu định dạng JSON.
- Frontend (Client-side): Sử dụng HTML/CSS và JavaScript để xây dựng giao diện. Frontend gửi yêu cầu (Request) đến Backend để lấy dữ liệu và hiển thị động lên màn hình.

Backend với Framework Django

a. Giới thiệu và vai trò:

- Django là một framework phát triển web mã nguồn mở mạnh mẽ viết bằng Python. Trong đề tài này, Django không được sử dụng theo mô hình MVT truyền thống (render HTML) mà đóng vai trò là một API Service.

b. Cơ chế hoạt động trong mô hình API:

- Thay vì sử dụng tầng Template để kết xuất HTML, Django xử lý luồng dữ liệu như sau:

1. URL Routing: Điều hướng yêu cầu API từ Client (ví dụ: GET /api/events/).
2. View (Logic): Thực hiện nghiệp vụ (Query Database, tính toán giá, kiểm tra Redis).
3. Serializer (Thay thế cho Template): Chuyển đổi dữ liệu phức tạp từ Model (Python Objects) sang định dạng JSON để Client có thể đọc được.
4. Response: Trả về kết quả JSON kèm theo mã trạng thái HTTP (200 OK, 400 Bad Request...).

c. Lý do lựa chọn

- Hỗ trợ tuyệt vời cho RESTful: Django giúp xây dựng các API endpoint nhanh chóng, chuẩn mực và dễ bảo trì.
- Bảo mật: Tích hợp sẵn các cơ chế xác thực (Authentication) và phân quyền (Authorization) để bảo vệ các API quan trọng.

1.2.1 Frontend (HTML/CSS/JavaScript)

Frontend được xây dựng độc lập với các công nghệ web tiêu chuẩn:

- HTML5/CSS3: Xây dựng cấu trúc và định dạng giao diện người dùng, đảm bảo tính thẩm mỹ và Responsive (thích ứng) trên nhiều thiết bị.
- JavaScript (ES6+): Đóng vai trò cốt lõi trong việc gọi API (sử dụng fetch) và xử lý logic hiển thị tại trình duyệt. Đặc biệt, JavaScript chịu trách nhiệm kết nối WebSocket để lắng nghe các sự kiện thời gian thực (như trạng thái ghế thay đổi) và cập nhật giao diện ngay lập tức mà không cần tải lại trang.

1.2.2 Hệ quản trị cơ sở dữ liệu MySQL

a. Giới thiệu chung MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở phổ biến nhất thế giới. MySQL sử dụng Ngôn ngữ truy vấn mang tính cấu trúc (SQL) để quản lý và thao tác dữ liệu.

b. Đặc điểm kỹ thuật và vai trò trong hệ thống bán vé sự kiện, tính toàn vẹn của dữ liệu là ưu tiên hàng đầu (ví dụ: một đơn hàng đã thanh toán thì không được phép mất dữ liệu, số tiền trừ đi phải khớp với số vé). MySQL đáp ứng tốt yêu cầu này nhờ tuân thủ chuẩn ACID:

Atomicity (Tính nguyên tử): Đảm bảo một giao dịch (transaction) đặt vé hoặc thành công hoàn toàn, hoặc thất bại hoàn toàn. Nếu lỗi xảy ra giữa chừng (ví dụ: mất mạng khi đang thanh toán), toàn bộ giao dịch sẽ được rollback (hoàn tác) để tránh sai lệch dữ liệu.

Consistency (Tính nhất quán): Dữ liệu luôn ở trạng thái hợp lệ trước và sau giao dịch.

Isolation (Tính cô lập): Các giao dịch diễn ra đồng thời (nhiều người mua vé cùng lúc) được xử lý độc lập, hạn chế xung đột.

Durability (Tính bền vững): Dữ liệu sau khi lưu trữ sẽ tồn tại vĩnh viễn trong ổ cứng, đảm bảo an toàn ngay cả khi hệ thống gặp sự cố mất điện.

c. Sự kết hợp Django - MySQL - Redis

- MySQL: Lưu trữ dữ liệu gốc ("nguồn sự thật" - Single Source of Truth).

- Redis: Lưu trữ dữ liệu tạm thời (Cache) và trạng thái ghế đang giao dịch (Locking) để giảm tải cho MySQL và tăng tốc độ phản hồi API.

1.2.3 Tích hợp thanh toán trực tuyến MoMo

a. Giới thiệu MoMo là ví điện tử phổ biến hàng đầu tại Việt Nam, cung cấp hệ sinh thái thanh toán đa dạng và API mở cho phép các nhà phát triển tích hợp vào ứng dụng thương mại điện tử.

b. Cơ chế tích hợp trong hệ thống (Sandbox Environment) Đề tài sử dụng môi trường kiểm thử (Sandbox) của MoMo để mô phỏng quy trình thanh toán thực tế. Quy trình tích hợp bao gồm:

- RESTful API: Backend gửi yêu cầu tạo giao dịch đến MoMo API chứa thông tin đơn hàng, số tiền và returnUrl.
- Cơ chế bảo mật (HMAC_SHA256): Mọi dữ liệu truyền đi đều được ký số (Signature) bằng thuật toán HMAC_SHA256 kết hợp với AccessKey và SecretKey do MoMo cung cấp, đảm bảo tính toàn vẹn dữ liệu.
- IPN (Instant Payment Notification): Sử dụng cơ chế Webhook để nhận kết quả thanh toán từ MoMo theo thời gian thực, giúp hệ thống cập nhật trạng thái đơn hàng ngay lập tức mà không cần người dùng phải quay lại website.

Lý do sử dụng: Momo là phương thức thanh toán tiện lợi và phổ biến với người dùng Việt Nam, giúp đơn giản hóa việc thanh toán trong các ứng dụng trực tuyến, đặc biệt là khi người dùng không có tài khoản ngân hàng. Hệ thống thanh toán Momo cho phép người dùng thực hiện thanh toán dễ dàng cho việc mua vé mà không cần phải rời khỏi ứng dụng.

1.3 Công nghệ xử lý dữ liệu và thời gian thực

Trong các hệ thống đặt vé trực tuyến có lưu lượng truy cập lớn (High Concurrency), hai thách thức kỹ thuật lớn nhất là: (1) Đảm bảo tính nhất quán dữ liệu khi nhiều người cùng tranh chấp một tài nguyên (ghế ngồi) và (2) Cập nhật trạng thái hiển thị tức thì cho người dùng. Để giải quyết triệt để các vấn đề này, đề tài sử dụng kết hợp Redis và giao thức WebSocket.

1.3.1 Redis và Cơ chế khóa tài nguyên (Resource Locking)

a. Cơ sở lý thuyết Redis (Remote Dictionary Server) là một hệ thống lưu trữ cấu trúc dữ liệu trong bộ nhớ (In-memory Data Structure Store) mã nguồn mở. Khác với MySQL lưu trữ trên ổ cứng, Redis lưu dữ liệu trên RAM, cho phép tốc độ đọc/ghi cực nhanh (có thể lên tới hàng triệu thao tác/giây).

Một tính năng quan trọng của Redis được ứng dụng trong đề tài là TTL (Time-To-Live). Tính năng này cho phép thiết lập thời gian tồn tại cho một khóa (key). Khi hết thời gian quy định, Redis sẽ tự động xóa khóa đó khỏi bộ nhớ mà không cần can thiệp thủ công.

b. Ứng dụng trong đề tài: Giải quyết bài toán "Giữ chỗ tạm thời" Trong đề tài này, Redis đóng vai trò như một "người gác cổng" (Gatekeeper) để quản lý kho vé trong thời gian thực, ngăn chặn tình trạng bán quá số lượng (Overselling).

- Vấn đề: Khi một trận đấu HOT chỉ còn vài vé cuối cùng, hàng chục người có thể cùng bấm "Thanh toán". Nếu chỉ xử lý bằng MySQL, độ trễ ổ cứng có thể dẫn đến việc hệ thống nhận đơn hàng cho 12 vé trong khi thực tế chỉ còn 10 vé.

- Giải pháp thực hiện: Hệ thống sử dụng cơ chế Locking dựa trên TTL:

1. Khóa slot (Acquire Lock): Khi Khách hàng A bấm đặt vé, hệ thống tạo một key trong Redis giữ một "slot" mua hàng.
2. Thiết lập thời gian giữ (TTL): Key này được cài đặt thời gian tồn tại là 180 giây (3 phút). Đây là khoảng thời gian dành cho khách hàng A thực hiện thanh toán.
3. Kiểm tra hạn mức: Redis sẽ đếm số lượng key đang tồn tại. Nếu tổng số key (người đang giữ vé) + số vé đã bán $\geq$ tổng sức chứa, hệ thống sẽ từ chối các yêu cầu đặt vé mới ngay lập tức.
4. Tự động giải phóng (Auto Release):
 - Trường hợp 1 (Thanh toán xong): Key trong Redis bị xóa, dữ liệu vé chính thức được ghi vào MySQL.
 - Trường hợp 2 (Hết giờ/Không mua): Sau 3 phút, Redis tự động xóa key (hết TTL). Số vé mua này được trả lại kho để người khác có thể đặt.

Hình 1.3: Mô hình giữ chỗ tạm thời sử dụng Redis TTL

1.3.2 WebSocket và Cơ chế phát tín hiệu (Signaling)

a. Cơ sở lý thuyết:

- Giao thức HTTP truyền thống hoạt động theo mô hình Request-Response: Client hỏi, Server trả lời. Để biết trạng thái mới, Client phải liên tục gửi yêu cầu (Polling), gây tốn tài nguyên và có độ trễ.

- Ngược lại, WebSocket là giao thức truyền thông song công (Full-duplex) qua một kết nối TCP duy nhất. Sau khi thiết lập kết nối (Handshake), Server có thể chủ động đẩy (Push) dữ liệu xuống Client ngay khi có sự kiện xảy ra mà không cần Client phải gửi yêu cầu trước.

b. Ứng dụng trong đề tài: Cập nhật số lượng vé tồn kho thời gian thực. Trong kiến trúc của đề tài, WebSocket đóng vai trò là kênh thông báo biến động kho vé tức thì đến người dùng cuối.

- Vấn đề: Khi Khách hàng A đã hoàn tất mua vé (hoặc đang giữ vé), số lượng vé khả dụng trong Database giảm xuống. Nếu màn hình của Khách hàng B vẫn hiện con số cũ (ví dụ: "Còn 50 vé"), họ sẽ có cảm giác sai lệch về độ khan hiếm của vé, dẫn đến trải nghiệm người dùng kém.

- Giải pháp thực hiện:

1. Kết nối: Khi người dùng vào trang chi tiết sự kiện, Frontend thiết lập kết nối WebSocket tới Django Server.
2. Phân nhóm (Grouping): Tất cả người dùng đang xem cùng một trận đấu sẽ được đưa vào một "Room" chung.
3. Phát sóng (Broadcasting): Ngay khi có một giao dịch đặt vé thành công (hoặc hủy vé), Backend truy vấn lại tổng số vé còn lại chính xác từ MySQL/Redis và gửi bản tin vào Room.
4. Đồng bộ hiển thị: Frontend của tất cả người dùng khác nhận được bản tin này sẽ lập tức cập nhật con số hiển thị trên màn hình (Ví dụ: số 50 nhảy xuống 49) mà không cần tải lại trang.

1.4 Trí tuệ nhân tạo và học máy

Để nâng cao tính cạnh tranh và trải nghiệm người dùng, đề tài tích hợp sâu các công nghệ trí tuệ nhân tạo vào hai phân hệ cốt lõi: Hỗ trợ khách hàng (Customer Support) và Tối ưu hóa kinh doanh (Business Optimization).

1.4.1 Kiến trúc RAG và AI Agent (Ứng dụng cho Chatbot)

a. Cơ sở lý thuyết Các mô hình ngôn ngữ lớn (LLM) truyền thống thường gặp phải vấn đề "ảo giác" (Hallucination) - tức là tự bịa ra thông tin khi không có dữ liệu thực tế, và giới hạn về kiến thức (Knowledge Cutoff). Trong bối cảnh bán vé, việc Chatbot cung cấp sai giá vé hoặc lịch thi đấu là không thể chấp nhận được.

Để giải quyết vấn đề này, đề tài sử dụng kiến trúc RAG (Retrieval-Augmented Generation). RAG là kỹ thuật tối ưu hóa đầu ra của LLM bằng cách tham chiếu đến một cơ sở tri thức tin cậy (Knowledge Base) trước khi sinh câu trả lời.

b. Ứng dụng trong đề tài: Chatbot Hybrid thông minh Hệ thống Chatbot không chỉ là một mô hình hỏi-đáp đơn thuần mà được thiết kế theo kiến trúc Hybrid (Lai ghép) gồm 3 lớp bảo vệ để đảm bảo tính chính xác tuyệt đối:

1. Lớp FAQ (Phản hồi nhanh): Hệ thống ưu tiên kiểm tra các câu hỏi thường gặp đã được định nghĩa sẵn (ví dụ: "Làm sao để hoàn vé?"). Nếu khớp, trả lời ngay lập tức để tiết kiệm tài nguyên AI.

2. Lớp RAG (Truy xuất dữ liệu thật):

- Data Layer: Dữ liệu về trận đấu, bảng giá (Match, SectionPrice) được chuyển đổi thành văn bản và nhúng (Embedding) thành các vector số học bằng model paraphrase-multilingual-MiniLM-L12-v2. Các vector này được lưu trữ trong Vector Database (ChromaDB).
- Retrieval: Khi người dùng hỏi "Giá vé trận MU vs MC?", hệ thống tìm kiếm trong ChromaDB các đoạn văn bản có ý nghĩa gần nhất (Semantic Search) thay vì tìm từ khóa chính xác.

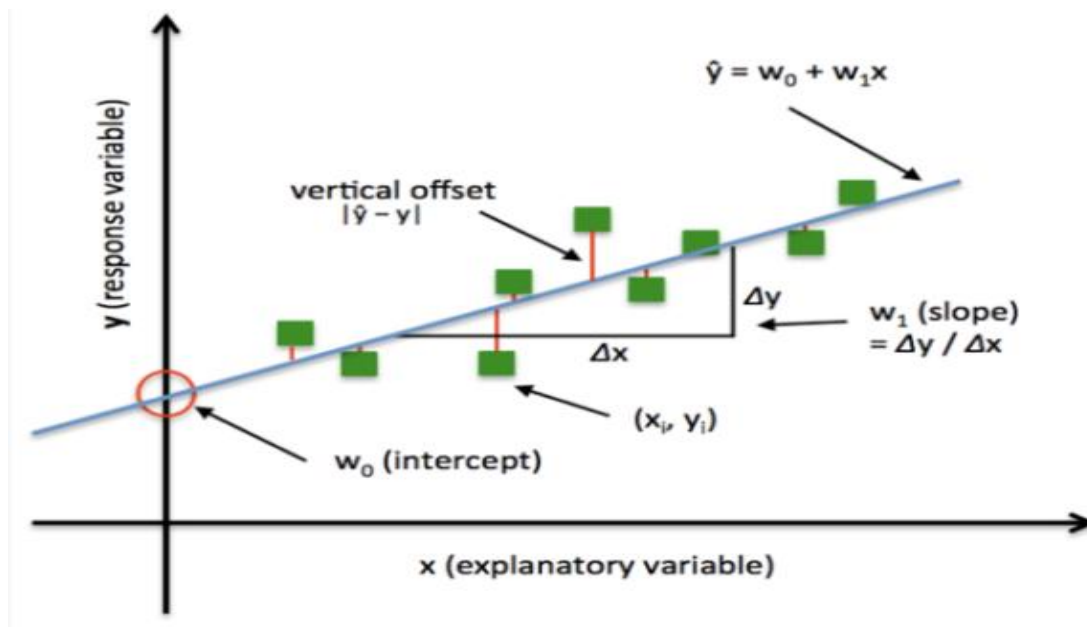
3. Lớp AI Agent (Tổng hợp câu trả lời):

- Sử dụng LLM LLaMA (thông qua nền tảng Groq để đạt tốc độ phản hồi cực nhanh).
- Agent nhận ngữ cảnh từ bước RAG và sinh câu trả lời tự nhiên: "Hiện tại trận MU vs MC có giá thấp nhất là 200k và cao nhất là 500k..." thay vì trả về dữ liệu thô.

1.4.2 Các thuật toán dự báo và lý do chọn Random Forest

Để giải quyết bài toán dự báo nhu cầu và tối ưu hóa giá vé, đề tài đã nghiên cứu và thử nghiệm trên 3 mô hình học máy phổ biến: Hồi quy tuyến tính, Prophet và Random Forest.

- Hồi quy tuyến tính (Linear Regression):



Hình 1: Mô hình Linear Regression

Trong lĩnh vực học có giám sát (supervised learning), hồi quy tuyến tính (Linear Regression) là một trong những thuật toán cơ bản và phổ biến nhất. Mô hình này được sử dụng để dự đoán giá trị số (liên tục) của một biến phụ thuộc dựa trên một hoặc nhiều biến độc lập. Linear Regression giả định tồn tại mối quan hệ tuyến tính giữa các biến đầu vào và đầu ra, được biểu diễn thông qua một phương trình đường thẳng.

Mô hình hồi quy tuyến tính học từ dữ liệu huấn luyện bằng cách tìm các hệ số (trọng số) tối ưu sao cho tổng sai số giữa giá trị dự đoán và giá trị thực tế là nhỏ nhất, thường được đo bằng sai số bình phương trung bình (Mean Squared Error). Sau khi huấn luyện, mô hình có thể được đánh giá trên tập kiểm thử thông qua các chỉ số như R^2 (hệ số xác định), MAE, hoặc RMSE.

Linear Regression được ứng dụng rộng rãi trong các lĩnh vực như kinh doanh (dự báo doanh thu theo tháng), bất động sản (dự đoán giá nhà), tài chính (dự báo lợi nhuận), và

y tế (dự đoán chỉ số sức khỏe). Trong các bài toán thực tế, dữ liệu đầu vào cần được xử lý và chuẩn hóa để đảm bảo tính ổn định của mô hình, và kiểm tra các giả định như phân phối chuẩn của phần dư và phương sai không đổi.

Mặc dù đơn giản, Linear Regression đóng vai trò nền tảng quan trọng trong khai phá dữ liệu và là điểm khởi đầu tốt để hiểu rõ hơn các mô hình phức tạp hơn trong học máy.

- Facebook Prophet:



Hình 2: Mô hình Prophet

Một trong những mô hình nổi bật và được sử dụng phổ biến để giải quyết các bài toán dự báo là Prophet, do Facebook phát triển. Prophet là một công cụ mã nguồn mở được thiết kế nhằm đơn giản hóa việc xây dựng các mô hình dự báo chuỗi thời gian phức tạp mà không yêu cầu kiến thức chuyên sâu về thống kê.

Khác với các mô hình truyền thống đòi hỏi quá trình tiền xử lý và điều chỉnh thủ công nhiều tham số, Prophet hoạt động theo nguyên lý phân tách chuỗi thời gian thành ba thành phần chính: xu hướng dài hạn (trend), yếu tố mùa vụ (seasonality) và các dị thường hoặc sự kiện đặc biệt (holiday effects). Điều này giúp mô hình giải thích được các thay đổi trong dữ liệu một cách trực quan và linh hoạt.

Prophet thuộc nhóm thuật toán học có giám sát, trong đó dữ liệu đầu vào thường là một bảng gồm hai cột: thời gian (ds) và giá trị cần dự báo (y). Mô hình sẽ học từ dữ liệu lịch sử để dự đoán các giá trị tương lai trong một khoảng thời gian xác định. Prophet hỗ trợ người dùng dễ dàng thêm các thông tin như ngày lễ, kỳ nghỉ hoặc các sự kiện ảnh hưởng đến doanh số, giúp tăng độ chính xác của dự báo.

Một ưu điểm nổi bật của Prophet là khả năng tạo ra dải dự báo (forecast interval), cho phép doanh nghiệp đánh giá mức độ tin cậy của kết quả dự đoán và đưa ra các kịch bản ứng phó phù hợp. Prophet còn có thể xử lý tốt các chuỗi thời gian bị thiếu dữ liệu, có biến động mạnh hoặc chứa xu hướng phi tuyến tính.

- Mô hình được chọn: Random Forest Regressor (Rừng ngẫu nhiên)

Một trong những thuật toán học máy mạnh mẽ và được ứng dụng rộng rãi nhất trong các bài toán dự báo kinh tế là Random Forest (Rừng ngẫu nhiên). Đây là một thuật toán thuộc nhóm học tổ hợp (Ensemble Learning), được thiết kế để khắc phục các nhược điểm về độ ổn định và khả năng khái quát hóa của các mô hình cây quyết định truyền thống.

Khác với các mô hình hồi quy tuyến tính cố gắng tìm ra một đường thẳng đơn giản để mô tả dữ liệu, Random Forest hoạt động theo nguyên lý xây dựng một "khu rừng" gồm nhiều Cây quyết định (Decision Trees) độc lập. Trong quá trình huấn luyện, mỗi cây sẽ học từ một tập con ngẫu nhiên của dữ liệu (kỹ thuật Bagging) và đưa ra dự báo riêng. Kết quả cuối cùng của mô hình là trung bình cộng của tất cả các dự báo từ các cây thành phần, giúp loại bỏ các sai số cá biệt và tạo ra một kết quả tổng quát đáng tin cậy.

Random Forest thuộc nhóm thuật toán học có giám sát, với khả năng xử lý linh hoạt cả dữ liệu số (như giá vé, giờ thi đấu) và dữ liệu phân loại (như độ hot của trận đấu, loại giải đấu). Mô hình sẽ tự động học các quy tắc ra quyết định phức tạp (ví dụ: "Nếu là trận Derby VÀ giá dưới 500k thì doanh thu cao") mà không cần người dùng phải thiết lập các phương trình toán học cứng nhắc.

Ưu điểm vượt trội của Random Forest trong bối cảnh đề tài này là khả năng xử lý dữ liệu phi tuyến tính và chống hiện tượng quá khớp (overfitting) cực tốt. Mô hình có thể nắm bắt chính xác các ngưỡng giá phân tầng (tiered pricing) hoặc các hành vi mua sắm bất quy tắc của khách hàng (cầu cứng) - điều mà các thuật toán đơn giản thường bỏ sót. Ngoài ra, Random Forest còn cung cấp thông tin về độ quan trọng của các đặc trưng (Feature Importance), giúp người quản trị hiểu rõ yếu tố nào đang chi phối doanh thu mạnh nhất.

Sau quá trình so sánh, đề tài quyết định lựa chọn Random Forest vì các ưu điểm vượt trội:

Xử lý phi tuyến tính: Random Forest là tập hợp của nhiều cây quyết định (Decision Trees), cho phép nó học được các quy luật phức tạp như "Cầu cứng" (Inelastic Demand) - ví dụ: với trận Chung kết, giá vé tăng cao vẫn có người mua.

Học được logic phân tầng (Tiered Pricing): Thuật toán tự động phát hiện các khoảng giá (Price ranges) tối ưu, phù hợp với chiến lược bán vé theo hạng (Tier 1, Tier 2) mà Linear Regression không làm được.

Chống quá khớp (Overfitting): Nhờ cơ chế lấy trung bình kết quả từ nhiều cây, mô hình hoạt động ổn định và ít bị nhiễu bởi các dữ liệu ngoại lai.

Chương 2: Thiết kế hệ thống

2.1 Phân tích yêu cầu người dùng và Đối tượng sử dụng

2.1.1 Yêu cầu chung

- Bảo mật: Hệ thống sử dụng mã hóa (OTP) để bảo vệ thông tin cá nhân, tài khoản và giao dịch của quản lý, nhân viên và khách hàng.
- Tính ổn định: Đảm bảo hệ thống hoạt động liên tục, hỗ trợ truy cập đồng thời từ nhiều người dùng mà không bị gián đoạn.
- Giao diện thân thiện: Thiết kế giao diện trực quan, dễ sử dụng cho cả quản lý, nhân viên và khách hàng.

2.1.2 Yêu cầu cho Quản lý

Quản lý tài nguyên

- Các đội sẽ có thông tin như tên đội, logo, huấn luyện viên trưởng, loại thể thao mà đội đó thi đấu, chỉ số sức hút và mô tả sơ lược về đội tuyển.
- Mỗi sân vận động sẽ có tên cụ thể, địa điểm, sức chứa theo đầu người, bố cục sân vận động và thông tin giới thiệu sân vận động đó. Trong sân vận động sẽ được chia ra thành nhiều khu vực khác nhau. Các khu vực được đánh tên cụ thể là A, B,... và có sức chứa cụ thể.
- Mỗi khi có giải đấu mới được diễn ra, thông tin về tên giải đấu, ngày bắt đầu và kết thúc sẽ được cập nhật.
- Cho phép thêm, chỉnh sửa hoặc xóa thông tin về các đội tuyển, sân vận động hay các giải đấu.
- Quản lý sẽ thêm, chỉnh sửa hoặc xóa thông tin về một trận đấu (ngày giờ, đội tham gia, sân vận động tổ chức, giải đấu và vòng loại). Thông tin về trận đấu có thể được thay đổi khi có sự cố phải dời lịch thi đấu, thông tin bị sai lệch,... Các thông tin về sự thay đổi phải được lưu lại bao gồm cả lý do. Thông báo thay đổi lịch trình trận đấu qua email đến khách hàng đã mua vé.
- Tra cứu và thêm các vé cho trận đấu. Bao gồm khu vực cụ thể mà vé được bán (phụ thuộc vào số lượng khu vực của sân vận động), số vé là sức chứa của khu vực, giá như thế nào và thời gian chính thức bán vé. Nếu có thay đổi về giá thì Quản lý sẽ chịu trách nhiệm nhập giá mới, lý do cho sự thay đổi và thời gian áp dụng giá đó. Hiện thị chi tiết vé trên hệ thống như thông tin về trận đấu, các khu vực ghế ngồi, giá vé, số lượng vé còn lại.
- Hiện thị thông báo đối với những yêu cầu hoàn vé.

Quản lý tài khoản

- Quản lý sẽ có các thông tin cơ bản như họ tên, giới tính, ngày sinh, căn cước công dân, email, số điện thoại, địa chỉ liên lạc, vai trò cụ thể và hình ảnh nhận diện của người đó. Được phép chỉnh sửa thông tin cá nhân của mình và thay đổi mật khẩu.
- Tạo tài khoản mới cho nhân viên hoặc quản lý ngang cấp với các thông tin cá nhân cơ bản. Tên đăng nhập và mật khẩu ban đầu của tài khoản sẽ là Căn cước công dân.
- Xem danh sách và tra cứu thông tin các nhân viên, quản trị viên trong hệ thống.
- Tắt hoặc bật trạng thái hoạt động cho tài khoản nhân viên khi họ nghỉ việc hoặc ngược lại.
- Chỉnh sửa thông tin cá nhân của nhân viên khi cần thiết.

Quản lý doanh thu và báo cáo

- Tạo báo cáo doanh thu theo thời gian, trận đấu và khu vực (tổng doanh thu, lợi nhuận).
- Hiện thị số liệu chi tiết: doanh thu theo trận đấu, từng khu vực hay sân vận động.
- Xuất báo cáo dưới dạng file (PDF, Excel, Word) để lưu trữ và phân tích.
- Hiện thị dashboard tương tác theo dõi tình hình kinh doanh như tổng doanh thu, các vé đã bán,....

Hỗ trợ khách hàng

- Đặt vé hộ khách hàng khi khách mua vé trực tiếp và nhập thông tin cần thiết.

Quản lý đơn hàng

- Theo dõi chi tiết và trạng thái đơn hàng (đang xử lý, hoàn tất, đã hủy).
- Xem lịch sử đơn hàng của tất cả khách hàng theo trận đấu, trạng thái, thời gian,...
- Theo dõi lịch sử bán vé và giao dịch (trực tiếp/trực tuyến) theo thời gian hoặc trận đấu.

Quản lý khuyến mãi

- Thêm chương trình khuyến mãi cho vé hoặc trận đấu (giảm giá, combo vé).
- Chỉnh sửa hoặc xóa thông tin của đợt khuyến mãi khi cần thiết hoặc có sự cố.

Hỗ trợ hoàn vé

- Lập phiếu hoàn vé cho các trường hợp hợp lệ (sự kiện hủy, vé lỗi) cho khách hàng mua trực tiếp tại quầy bán vé.
- Xử lý yêu cầu bảo hành/xác minh vé bị lỗi (mã QR không hoạt động, thông tin sai).
- Duyệt các phiếu hoàn vé cho khách hàng nếu hợp lệ.

2.1.3 Yêu cầu cho Nhân viên

Hỗ trợ khách hàng

- Đặt vé hộ khách hàng khi khách mua vé trực tiếp và nhập thông tin cần thiết.

Đăng nhập và quản lý tài khoản cá nhân

- Đăng nhập bằng tài khoản được quản lý cấp (tên đăng nhập, mật khẩu).
- Thay đổi mật khẩu cá nhân khi cần. Lúc quên mật khẩu thì có thể dùng mã OTP được gửi qua điện thoại để tạo lại mật khẩu mới.

Truy cập tài nguyên

- Xem và tra cứu các vé đã, đang và sắp được bán.

Xử lý đơn hàng

- Xem và duyệt đơn đặt vé trực tuyến từ khách hàng.
- Xuất hóa đơn điện tử (Word, PDF) và gửi qua email hoặc in trực tiếp cho khách.

Quản lý khuyến mãi

- Thêm chương trình khuyến mãi cho vé hoặc trận đấu (giảm giá, combo vé).
- Chỉnh sửa hoặc xóa thông tin của đợt khuyến mãi theo chỉ đạo từ quản lý.

Xem lịch sử giao dịch

- Theo dõi lịch sử bán vé và giao dịch (trực tiếp/trực tuyến) theo thời gian hoặc trận đấu.

Hỗ trợ hoàn vé

- Lập phiếu hoàn vé cho các trường hợp hợp lệ (sự kiện hủy, vé lỗi) cho khách hàng mua trực tiếp tại quầy bán vé.
- Xử lý yêu cầu bảo hành/xác minh vé bị lỗi (mã QR không hoạt động, thông tin sai).
- Duyệt các phiếu hoàn vé cho khách hàng nếu hợp lệ.

2.1.4 Yêu cầu cho Khách hàng

Quản lý tài khoản

- Đăng ký tài khoản qua website với thông tin cơ bản (họ tên, email, số điện thoại, mật khẩu) hoặc đăng nhập một lần qua các tài khoản mạng xã hội (Google, Facebook).
- Chỉnh sửa thông tin cá nhân và thay đổi mật khẩu trên trang cá nhân.
- Khi quên mật khẩu thì sẽ được hệ thống gửi một mã OTP về số điện thoại hoặc email để khách hàng có thể tiến hành đặt lại mật khẩu mới.
- Xem danh sách các đơn hàng hay vé đã đặt trong lịch sử giao dịch tài khoản.

Xem thông tin vé

- Tra cứu danh sách trận đấu sắp diễn ra với bộ lọc (đội, ngày, giải đấu). Lịch sử tra cứu có thể lưu lại dựa vào những tìm kiếm trước đó thông qua cookie (những tìm kiếm đã qua thời gian hiện tại sẽ được loại bỏ)
- Hiển thị chi tiết vé: giá, vị trí ghế, ngày giờ trận đấu, số lượng vé còn lại.

Đặt mua vé

- Đặt vé trực tuyến với sơ đồ ghế ngồi tương ứng trong sân vận động (chọn khu vực muốn ngồi, số lượng vé).
- Hỗ trợ thanh toán qua nhiều phương thức: chuyển khoản, thẻ ngân hàng, ví điện tử, tiền mặt.
- Nhận vé điện tử qua email với mã QR và thông tin chi tiết. Xuất vé giấy có kèm mã QR cho khách hàng trực tiếp.

Theo dõi đơn hàng

- Xem trạng thái đơn hàng (đã đặt, hoàn tất, bị hủy) trên tài khoản cá nhân.
- Hủy đơn hàng nếu chưa được xử lý (trước thời gian quy định).

Đổi trả và bảo hành vé

- Gửi yêu cầu đổi trả vé qua website với lý do hợp lệ (sự kiện hủy, thay đổi lịch).
- Đăng ký bảo hành vé khi gặp lỗi kỹ thuật (mã QR không quét được, thông tin sai).

2.1.5 Yêu cầu xử lý rủi ro và tích hợp

Quản lý rủi ro

- Lỗi hệ thống: Giám sát liên tục, sao lưu dữ liệu định kỳ, thông báo lỗi cho quản lý/nhân viên.
- Hết vé: Cập nhật số lượng vé theo thời gian thực, khóa giao dịch khi hết vé.
- Gian lận: Tạo mã QR duy nhất cho mỗi vé, kiểm tra tính hợp lệ tại cổng vào.

- Thay đổi lịch trình: Tự động thông báo qua email/SMS và hỗ trợ đổi/hoàn tiền.

Tích hợp

- Kết nối với máy in vé để in trực tiếp tại quầy (cho nhân viên).
- Hỗ trợ máy quét mã QR tại cổng vào sân vận động.
- Tích hợp cổng thanh toán trực tuyến (Visa, MasterCard, Momo, ZaloPay, v.v.) cho khách hàng.

2.1.6 Yêu cầu báo cáo và phân quyền

Báo cáo

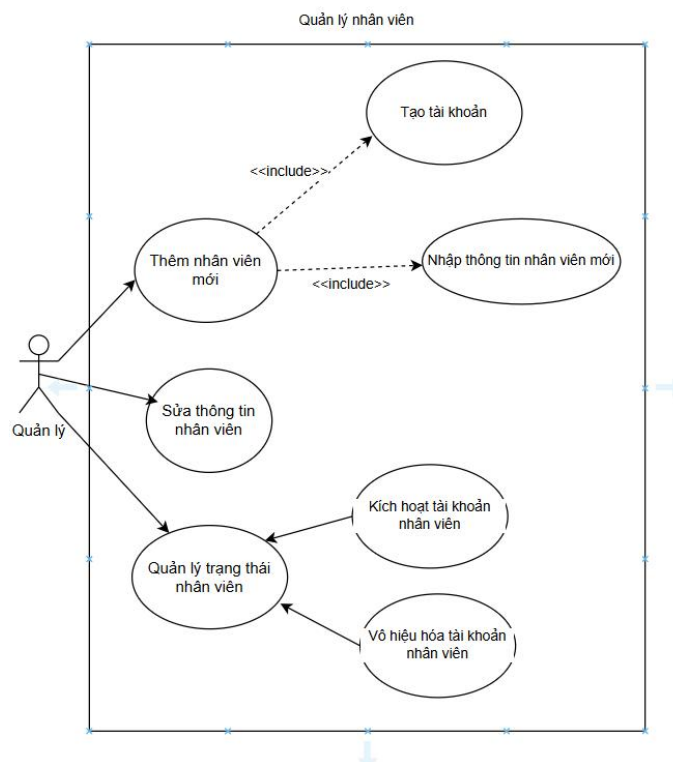
- Hệ thống cung cấp báo cáo chi tiết (doanh thu, số vé bán, đơn hàng) cho quản lý; báo cáo giao dịch cơ bản cho nhân viên.

Phân quyền

- Quản lý: Truy cập toàn bộ hệ thống (quản lý sự kiện, quản lý vé, nhân viên, doanh thu).
- Nhân viên: Truy cập hạn chế (quản lý vé, đơn hàng, hỗ trợ khách hàng).
- Khách hàng: Chỉ truy cập tài khoản cá nhân và chức năng đặt vé.

2.2 Mô hình Use Case

2.2.1 Quản lý nhân viên



Hình 3: Usecase Quản lý nhân viên

Use Case: Thêm nhân viên mới

- Mô tả: Quản lý thêm một nhân viên mới vào hệ thống, bao gồm nhập thông tin và tạo tài khoản cho nhân viên.
- Actor: Quản lý
- Mục tiêu: Thêm thành công nhân viên mới và tạo tài khoản đăng nhập cho nhân viên.
- Tiền đề: Quản lý đã đăng nhập hệ thống.
- Luồng sự kiện:
 1. Quản lý chọn chức năng thêm nhân viên mới.
 2. Nhập đầy đủ thông tin nhân viên (họ tên, ngày sinh, số điện thoại, email...).
 3. Hệ thống kiểm tra, lưu thông tin và tự động tạo tài khoản cho nhân viên.
 4. Thông báo kết quả cho quản lý.

Use Case: Sửa thông tin nhân viên

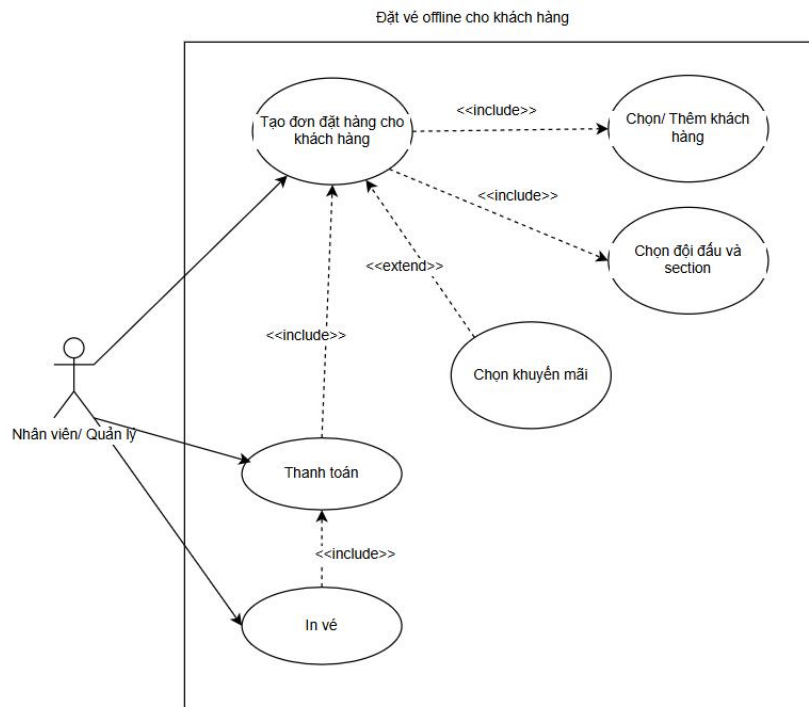
- Mô tả: Quản lý cập nhật thông tin cho nhân viên khi có thay đổi.
- Actor: Quản lý
- Mục tiêu: Đảm bảo thông tin nhân viên luôn chính xác.
- Tiền đề: Nhân viên đã tồn tại trên hệ thống.
- Luồng sự kiện:
 1. Quản lý chọn nhân viên cần sửa thông tin.
 2. Nhập thông tin mới cần cập nhật.
 3. Hệ thống kiểm tra và lưu thông tin mới.

Use Case: Quản lý trạng thái nhân viên

- Mô tả: Quản lý thực hiện kích hoạt hoặc vô hiệu hóa tài khoản nhân viên.
- Actor: Quản lý
- Mục tiêu: Kiểm soát quyền truy cập hệ thống của nhân viên.
- Tiền đề: Nhân viên đã có tài khoản trên hệ thống.
- Luồng sự kiện:
 1. Quản lý chọn nhân viên cần thay đổi trạng thái tài khoản.
 2. Chọn kích hoạt hoặc vô hiệu hóa tài khoản.

3. Hệ thống cập nhật trạng thái tài khoản và thông báo cho quản lý.

2.2.2 Chức năng đặt đơn cho khách hàng



Hình 4: Usecase Đặt vé offline cho khách hàng

Use Case: Tạo đơn đặt hàng cho khách hàng

- Mô tả: Nhân viên/Quản lý tạo đơn đặt vé cho khách hàng tại quầy, bao gồm chọn khách hàng, chọn trận đấu, vị trí ngồi và (nếu có) áp dụng khuyến mãi.
- Actor: Nhân viên/Quản lý
- Mục tiêu: Tạo thành công đơn đặt vé offline cho khách hàng.
- Tiền đề: Nhân viên/Quản lý đã đăng nhập hệ thống.
- Luồng sự kiện:
 1. Chọn hoặc thêm mới thông tin khách hàng.
 2. Chọn trận đấu và section/vị trí ngồi cho khách hàng.
 3. (Tùy chọn) Chọn khuyến mãi áp dụng nếu có.
 4. Hệ thống kiểm tra thông tin và tạo đơn đặt vé.
 5. Thông báo kết quả cho nhân viên/quản lý.

Use Case: Thanh toán

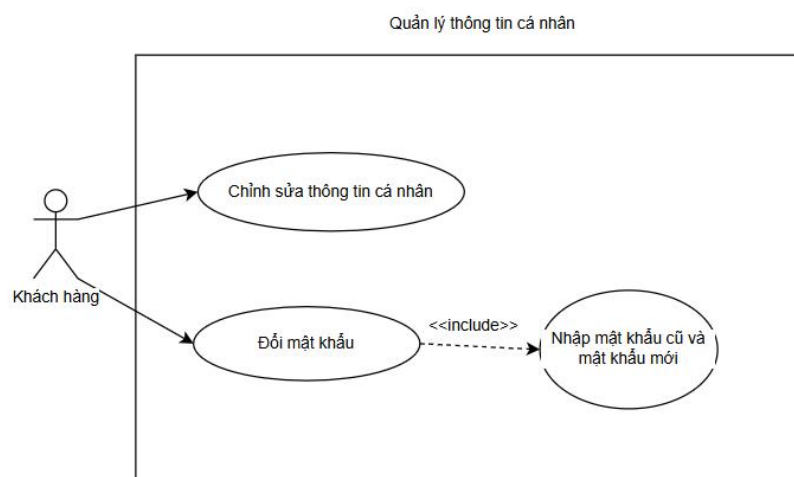
- Mô tả: Nhân viên/Quản lý thực hiện thanh toán cho đơn đặt vé của khách hàng.
- Actor: Nhân viên/Quản lý

- Mục tiêu: Hoàn tất thanh toán cho đơn đặt vé.
- Tiền đề: Đơn đặt vé đã được tạo thành công.
- Luồng sự kiện:
 1. Nhân viên/Quản lý kiểm tra lại thông tin đơn hàng.
 2. Thực hiện các bước thanh toán (tiền mặt, chuyển khoản...).
 3. Hệ thống xác nhận thanh toán thành công.

Use Case: In vé

- Mô tả: Nhân viên/Quản lý in vé cho khách hàng sau khi thanh toán thành công.
- Actor: Nhân viên/Quản lý
- Mục tiêu: Khách hàng nhận được vé sau khi đã thanh toán.
- Tiền đề: Thanh toán cho đơn hàng đã hoàn tất.
- Luồng sự kiện:
 1. Nhân viên/Quản lý chọn chức năng in vé.
 2. Hệ thống in vé và giao cho khách hàng.

2.2.3 Quản lý thông tin cá nhân của khách hàng



Hình 5: Usecase Quản lý thông tin cá nhân của khách hàng

Use Case: Chỉnh sửa thông tin cá nhân

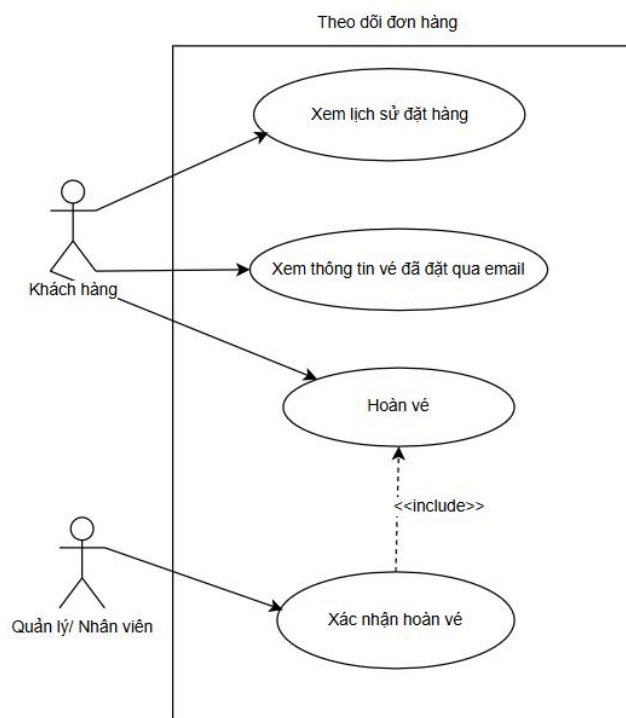
- Mô tả: Khách hàng cập nhật lại các thông tin cá nhân của mình trên hệ thống.
- Actor: Khách hàng
- Mục tiêu: Thông tin cá nhân luôn chính xác, đầy đủ.
- Tiền đề: Khách hàng đã đăng nhập hệ thống.

- **Luồng sự kiện:**
 1. Khách hàng chọn chức năng chỉnh sửa thông tin cá nhân.
 2. Nhập hoặc sửa các thông tin cần thay đổi (họ tên, số điện thoại, email...).
 3. Hệ thống kiểm tra và cập nhật thông tin mới.

Use Case: Đổi mật khẩu

- **Mô tả:** Khách hàng thay đổi mật khẩu đăng nhập tài khoản.
- **Actor:** Khách hàng
- **Mục tiêu:** Đảm bảo tài khoản an toàn hơn cho khách hàng.
- **Tiền đề:** Khách hàng đã đăng nhập hệ thống.
- **Luồng sự kiện:**
 1. Khách hàng chọn chức năng đổi mật khẩu.
 2. Nhập mật khẩu cũ và mật khẩu mới.
 3. Hệ thống kiểm tra và cập nhật mật khẩu mới nếu hợp lệ.

2.2.4 Khách hàng theo dõi đơn hàng



Hình 6: Usecase Khách hàng theo dõi đơn hàng của mình

Use Case: Xem lịch sử đơn hàng

- **Mô tả:** Khách hàng xem danh sách các đơn đặt vé trước đó của mình trên hệ thống.

- Actor: Khách hàng
- Mục tiêu: Giúp khách hàng quản lý, kiểm tra lại các đơn đặt vé đã thực hiện.
- Tiền đề: Khách hàng đã đăng nhập hệ thống.
- Luồng sự kiện:
 1. Khách hàng chọn chức năng xem lịch sử đặt hàng.
 2. Hệ thống hiển thị danh sách các đơn hàng đã đặt.

Use Case: Xem thông tin vé đã đặt qua email

- Mô tả: Khách hàng kiểm tra thông tin chi tiết về vé đã đặt thông qua email xác nhận.
- Actor: Khách hàng
- Mục tiêu: Xem lại chi tiết mã vé, thời gian, vị trí... đã được gửi qua email.
- Tiền đề: Đã có đơn đặt vé thành công và khách hàng có email xác nhận.
- Luồng sự kiện:
 1. Khách hàng kiểm tra email xác nhận từ hệ thống.
 2. Xem thông tin chi tiết về vé trong email.

Use Case: Hoàn vé

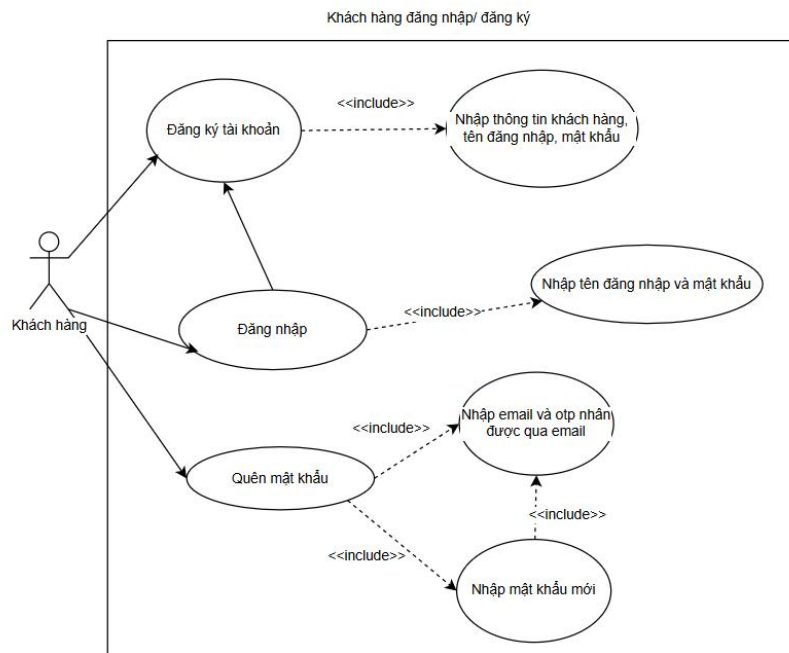
- Mô tả: Khách hàng yêu cầu hoàn trả vé đã đặt (hủy vé).
- Actor: Khách hàng
- Mục tiêu: Được hoàn lại tiền vé khi không sử dụng.
- Tiền đề: Vé đã đặt thành công, nằm trong thời hạn hoàn vé.
- Luồng sự kiện:
 1. Khách hàng chọn đơn cần hoàn vé.
 2. Gửi yêu cầu hoàn vé lên hệ thống.
 3. Chờ nhân viên/ quản lý xác nhận hoàn vé.

Use Case: Xác nhận hoàn vé

- Mô tả: Nhân viên/Quản lý kiểm tra và xác nhận yêu cầu hoàn vé của khách hàng.
- Actor: Quản lý/Nhân viên
- Mục tiêu: Đảm bảo quy trình hoàn vé diễn ra minh bạch, chính xác.
- Tiền đề: Có yêu cầu hoàn vé từ khách hàng.

- Luồng sự kiện:
 1. Nhân viên/Quản lý nhận yêu cầu hoàn vé.
 2. Kiểm tra thông tin và điều kiện hoàn vé.
 3. Xác nhận hoàn vé và hoàn tiền cho khách hàng nếu hợp lệ.

2.2.5 Khách hàng đăng nhập/đăng ký tài khoản



Hình 7: Usecase Khách hàng đăng nhập, đăng ký tài khoản

Use Case: Đăng ký tài khoản

- Mô tả: Khách hàng tạo tài khoản mới trên hệ thống bằng cách nhập thông tin cá nhân, tên đăng nhập và mật khẩu.
- Actor: Khách hàng
- Mục tiêu: Tạo thành công tài khoản để sử dụng hệ thống.
- Tiền đề: Khách hàng chưa có tài khoản.
- Luồng sự kiện:
 1. Khách hàng chọn chức năng đăng ký tài khoản.
 2. Nhập thông tin cá nhân, tên đăng nhập, mật khẩu.
 3. Hệ thống kiểm tra hợp lệ và tạo tài khoản mới.
 4. Thông báo kết quả đăng ký cho khách hàng.

Use Case: Đăng nhập

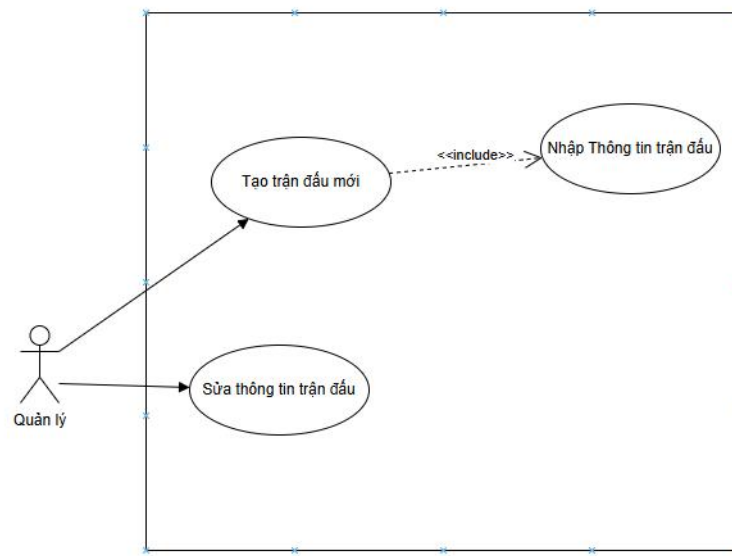
- Mô tả: Khách hàng đăng nhập vào hệ thống bằng tên đăng nhập và mật khẩu.

- Actor: Khách hàng
- Mục tiêu: Truy cập vào hệ thống với tài khoản hợp lệ.
- Tiền đề: Khách hàng đã có tài khoản.
- Luồng sự kiện:
 1. Khách hàng chọn chức năng đăng nhập.
 2. Nhập tên đăng nhập và mật khẩu.
 3. Hệ thống kiểm tra thông tin đăng nhập.
 4. Thông báo kết quả đăng nhập cho khách hàng.

Use Case: Quên mật khẩu

- Mô tả: Khách hàng lấy lại mật khẩu khi không nhớ mật khẩu đăng nhập.
- Actor: Khách hàng
- Mục tiêu: Đặt lại mật khẩu mới để tiếp tục sử dụng hệ thống.
- Tiền đề: Khách hàng đã có tài khoản và quên mật khẩu.
- Luồng sự kiện:
 1. Khách hàng chọn chức năng "Quên mật khẩu".
 2. Nhập email và mã OTP nhận được qua email.
 3. Nhập mật khẩu mới.
 4. Hệ thống xác nhận và cập nhật mật khẩu mới nếu hợp lệ.
 5. Thông báo kết quả cho khách hàng.

2.2.6 Quản lý trận đấu

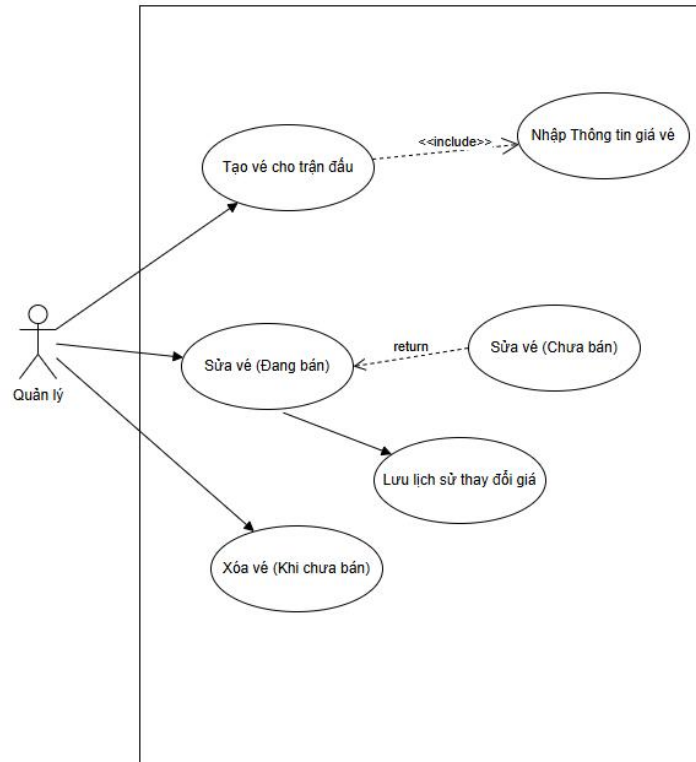


Hình 8: Usecase Quản lý trận đấu

Use Case: Thêm/Sửa trận đấu mới

- Mô tả: Quản lý thêm/sửa một trận đấu mới vào hệ thống.
- Actor: Quản lý
- Mục tiêu: Thêm/sửa trận đấu mới thành công.
- Tiền đề: Quản lý đã đăng nhập hệ thống.
- Luồng sự kiện:
 1. Quản lý chọn chức năng thêm/sửa trận đấu mới.
 2. Nhập đầy đủ thông tin (Thời gian, sân, giải đấu,...).
 3. Hệ thống kiểm tra, lưu thông tin và tự động tạo cho trận đấu mới.
 4. Thông báo kết quả cho quản lý.

2.2.7 Quản lý vé



Hình 9: Usecase Quản lý vé

Use Case: Tạo vé cho trận đấu

- Mô tả: Quản lý tạo mới vé cho một trận đấu thể thao, bao gồm cấu hình mức giá và khu vực chỗ ngồi.
- Actor: Quản lý
- Mục tiêu: Tạo thành công vé cho một trận đấu.
- Tiền đề: Trận đấu đã được tạo và chưa có vé nào được gán.
- Luồng sự kiện:
 1. Quản lý chọn trận đấu cần tạo vé.
 2. Nhập thông tin các mức giá cho từng khu vực chỗ ngồi.
 3. Hệ thống kiểm tra và lưu thông tin vé.
 4. Thông báo kết quả tạo vé thành công.

Use Case: Sửa thông tin vé

- Mô tả: Quản lý cập nhật thông tin vé, bao gồm mức giá hoặc thông tin khu vực. Tùy trạng thái, hệ thống có thể lưu lại lịch sử thay đổi giá.
- Actor: Quản lý
- Mục tiêu: Cập nhật thông tin vé chính xác và ghi nhận lịch sử khi cần thiết.
- Tiền đề: Vé đã được tạo.
- Luồng sự kiện:
 - TH1 - Vé chưa mở bán:
 1. Quản lý chọn vé cần sửa.

2. Nhập thông tin giá mới.
3. Hệ thống lưu thông tin mới (không ghi vào lịch sử thay đổi giá).
4. Thông báo cập nhật thành công.

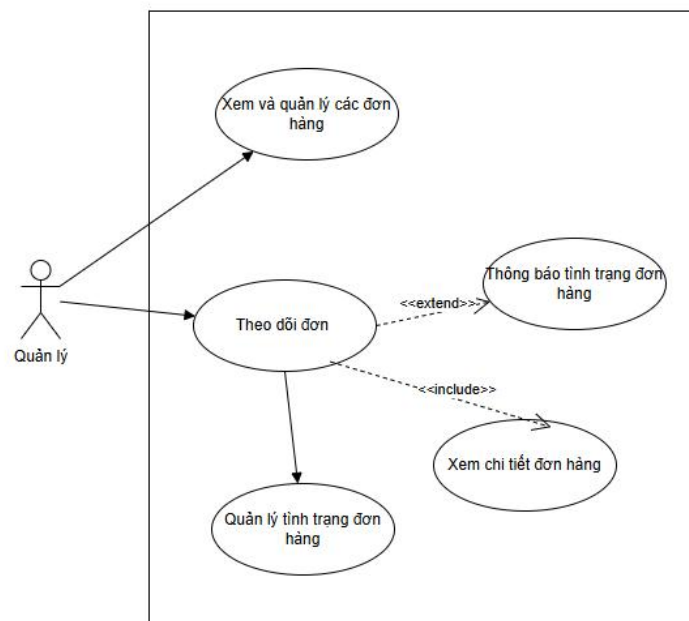
TH2 - Vé đang mở bán:

1. Quản lý chọn vé cần sửa.
2. Nhập thông tin giá mới.
3. Hệ thống lưu thông tin mới và ghi nhận lịch sử thay đổi giá.
4. Thông báo cập nhật thành công.

Use Case: Xóa vé

- Mô tả: Quản lý xóa vé nếu vé chưa được mở bán.
- Actor: Quản lý
- Mục tiêu: Loại bỏ vé không cần thiết trước khi bán.
- Tiền đề: Vé chưa được mở bán.
- Luồng sự kiện:
 1. Quản lý chọn vé cần xóa.
 2. Hệ thống kiểm tra trạng thái vé.
 3. Nếu vé chưa mở bán, hệ thống xóa vé.
 4. Thông báo kết quả cho quản lý.

2.2.8 Theo dõi đơn hàng



Hình 10: Usecase Quản lý đơn hàng

Use Case: Xem danh sách đơn hàng

- Mô tả: Quản lý xem tất cả các đơn hàng đã được đặt trên hệ thống, kèm thông tin cơ bản như mã đơn, khách hàng, trạng thái.
- Actor: Quản lý
- Mục tiêu: Theo dõi tình hình đặt hàng và xử lý đơn.
- Tiền đề: Đã có đơn hàng được tạo bởi khách hàng.
- Luồng sự kiện:

1. Quản lý truy cập chức năng quản lý đơn hàng.
2. Hệ thống hiển thị danh sách các đơn hàng.
3. Quản lý có thể lọc, tìm kiếm theo trạng thái hoặc mã đơn.

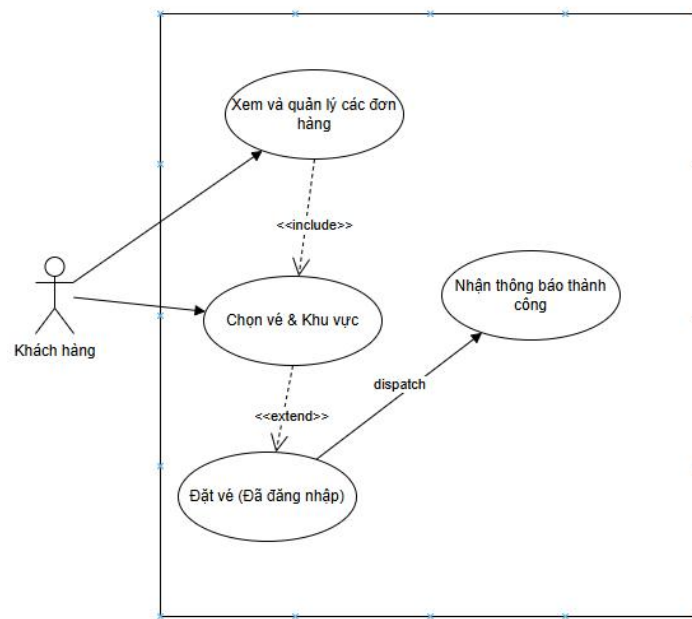
Use Case: Xem chi tiết đơn hàng

- Mô tả: Quản lý xem chi tiết thông tin một đơn hàng cụ thể, bao gồm vé đã đặt, thông tin khách hàng, lịch sử trạng thái.
- Actor: Quản lý
- Mục tiêu: Hiểu rõ nội dung từng đơn hàng để xử lý chính xác.
- Tiền đề: Đơn hàng tồn tại trên hệ thống.
- Luồng sự kiện:
 1. Quản lý chọn một đơn hàng từ danh sách.
 2. Hệ thống hiển thị chi tiết đơn hàng: danh sách vé, giá trị đơn, thông tin khách hàng, lịch sử cập nhật trạng thái.
 3. Quản lý có thể quay lại danh sách hoặc tiến hành cập nhật trạng thái.

Use Case: Cập nhật trạng thái đơn hàng

- Mô tả: Quản lý thay đổi trạng thái đơn hàng để phản ánh tiến trình xử lý (ví dụ: từ “Chờ xử lý” sang “Đang giao”).
- Actor: Quản lý
- Mục tiêu: Đảm bảo đơn hàng được cập nhật đúng theo quá trình xử lý thực tế.
- Tiền đề: Đơn hàng đang ở trạng thái có thể cập nhật (chưa bị hủy hoặc đã giao).
- Luồng sự kiện:
 1. Quản lý chọn đơn hàng cần cập nhật.
 2. Chọn trạng thái mới (ví dụ: Đã thanh toán, Đang giao, Đã hủy...).
 3. Hệ thống kiểm tra tính hợp lệ của việc chuyển trạng thái.
 4. Nếu hợp lệ, cập nhật trạng thái và ghi vào lịch sử đơn hàng.
 5. Thông báo kết quả cho quản lý.

2.2.9 Đặt vé trực tuyến



Hình 11: Usecase Đặt vé trực tuyến

Use Case: Xem danh sách trận đấu

- Mô tả: Khách hàng xem danh sách các trận đấu sắp diễn ra để lựa chọn trận muốn mua vé.
- Actor: Khách hàng
- Mục tiêu: Lựa chọn trận đấu yêu thích để đặt vé.
- Tiền đề: Hệ thống đã có danh sách trận đấu được mở bán vé.
- Luồng sự kiện:
 1. Khách hàng đăng nhập vào hệ thống.
 2. Chọn chức năng xem danh sách trận đấu.
 3. Hệ thống hiển thị thông tin các trận đấu sắp diễn ra, có thể lọc theo thời gian, giải đấu.

Use Case: Chọn vé và khu vực chỗ ngồi

- Mô tả: Khách hàng chọn khu vực ngồi và số lượng vé muốn đặt cho một trận đấu cụ thể.
- Actor: Khách hàng
- Mục tiêu: Xác định chính xác vé cần mua.
- Tiền đề: Trận đấu đang mở bán vé.
- Luồng sự kiện:
 1. Khách hàng chọn một trận đấu từ danh sách.
 2. Hệ thống hiển thị sơ đồ sân và các mức giá theo khu vực.
 3. Khách hàng chọn khu vực và số lượng vé cần mua.
 4. Hệ thống kiểm tra số lượng còn lại.

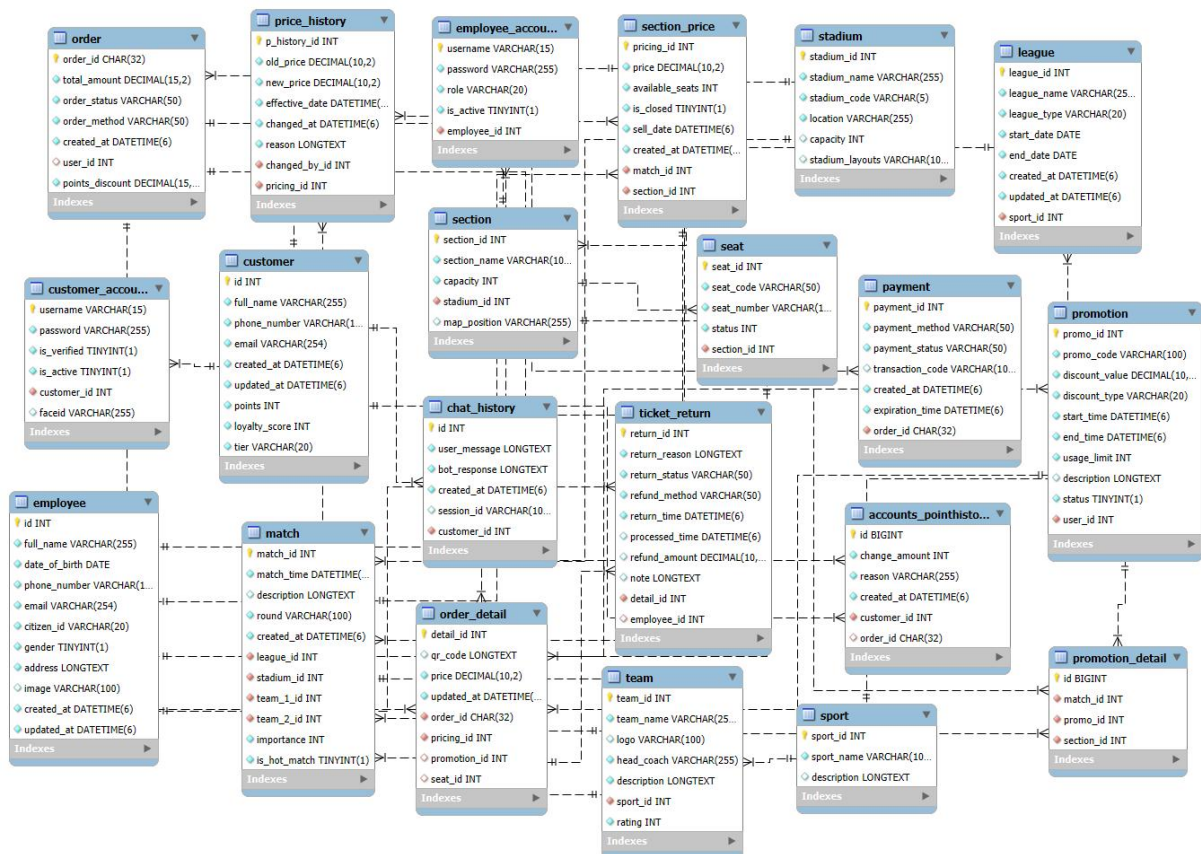
Use Case: Thanh toán đơn vé

- Mô tả: Khách hàng thực hiện thanh toán cho đơn vé đã chọn bằng các hình thức được hỗ trợ (ví dụ: ví điện tử, thẻ ngân hàng).
- Actor: Khách hàng
- Mục tiêu: Hoàn tất quá trình đặt vé và nhận xác nhận từ hệ thống.

- Tiền đề: Đã chọn đủ thông tin vé cần mua.
- Luồng sự kiện:
 1. Khách hàng nhấn “Thanh toán”.
 2. Hệ thống hiển thị tổng tiền và lựa chọn phương thức thanh toán.
 3. Khách hàng nhập thông tin thanh toán và xác nhận.
 4. Hệ thống xử lý giao dịch.
 5. Nếu thành công: tạo đơn vé và gửi xác nhận qua email/app.
 6. Nếu thất bại: hiển thị lỗi và yêu cầu thử lại.
 7. Khách hàng nhận thông báo đặt vé thành công (hoặc lỗi nếu có).

2.3 Thiết kế cơ sở dữ liệu

2.3.1 Diagram



Hình 12: Sơ đồ quan hệ cơ sở dữ liệu

2.3.2 Các bảng thực thể

1. Employee

Tên trường	Kiểu dữ liệu	Mô tả
id	INT (PK, AUTO_INCREMENT)	Mã định danh người của hệ thống.
email	VARCHAR	Email đăng ký (not null).
phone_number	VARCHAR	Số điện thoại.
citizen_id	VARCHAR	Số Căn cước công dân.
gender	BIT	Giới tính. (0: Nam, 1: Nữ)

full_name	VARCHAR	Họ và tên.
date_of_birth	DATE	Ngày sinh.
address	VARCHAR	Địa chỉ.
image	VARCHAR	Đường dẫn của hình ảnh.
created_at	DATETIME	Thời điểm tạo tài khoản.
updated_at	DATETIME	Thời điểm cập nhật cuối cùng.

Unique Key: phone_number, email và citizen_id.

2. Employee Account

Tên trường	Kiểu dữ liệu	Mô tả
username	VARCHAR (PK)	Số điện thoại đăng nhập.
password	VARCHAR	Mã băm mật khẩu.
role	ENUM('staff', 'manager')	Vai trò của tài khoản.
is_active	BOOLEAN	Trạng thái hoạt động của tài khoản.
employee_id	INT (FK Employee.employee_id) →	Mã nhân viên sở hữu tài khoản.

3. Customer

Tên trường	Kiểu dữ liệu	Mô tả
id	INT (PK, AUTO_INCREMENT)	Mã định danh khách hàng.
email	VARCHAR	Email đăng ký (not null).
phone_number	VARCHAR	Số điện thoại.
full_name	VARCHAR	Họ và tên.
created_at	DATETIME	Thời điểm tạo tài khoản.
updated_at	DATETIME	Thời điểm cập nhật cuối cùng.
points	INT	Điểm tiêu dùng hiện có
loyalty_score	INT	Tổng điểm lịch sử
tier	VARCHAR	Phân hạng khách hàng thân thiết

Unique Key: phone_number và email.

4. Customer Account

Tên trường	Kiểu dữ liệu	Mô tả
username	VARCHAR (PK)	Số điện thoại đăng nhập.
password	VARCHAR	Mã băm mật khẩu.
is_verified	BOOLEAN	Tình trạng xác thực của tài khoản.
is_active	BOOLEAN	Tình trạng hoạt động của tài khoản.
customer_id	INT (FK → Customer.customer_id)	Mã khách hàng sở hữu tài khoản.

5. Sport

Tên trường	Kiểu dữ liệu	Mô tả
sport_id	INT (PK, AUTO_INCREMENT)	Mã định danh môn thể thao.
sport_name	VARCHAR(100)	Tên môn thể thao (Ví dụ: Bóng đá, Bóng rổ). Ràng buộc: UNIQUE (Không trùng tên).
description	TEXT	Mô tả chi tiết về môn thể thao.

6. Team

Tên trường	Kiểu dữ liệu	Mô tả
team_id	INT (PK, AUTO_INCREMENT)	Mã đội bóng.
team_name	VARCHAR	Tên đội bóng.
logo	VARCHAR	Đường dẫn logo (nếu có).
head_coach	VARCHAR	Tên huấn luyện viên của đội bóng
description	TEXT	Mô tả hoặc thông tin thêm.
rating	INT	Chỉ số sức hút (Thang điểm 1-10).

Unique Key (sport_id, team_name)

7. Stadium

Tên trường	Kiểu dữ liệu	Mô tả
------------	--------------	-------

stadium_id	INT (PK, AUTO_INCREMENT)	Mã sân vận động.
stadium_code	VARCHAR(5)	Mã định danh của sân vận động
stadium_name	VARCHAR	Tên sân vận động.
location	VARCHAR	Địa chỉ.
capacity	INT	Sức chứa (số ghế).
stadium_layout	TEXT	Sơ đồ sân vận động

Unique Key: stadium_name, stadium_code.

8. League

Tên trường	Kiểu dữ liệu	Mô tả
league_id	INT (PK, AUTO_INCREMENT)	Mã giải đấu.
league_name	VARCHAR	Tên giải đấu.
league_type	VARCHAR(20)	Thể thức thi đấu. Các giá trị cho phép: - round_robin: Đấu vòng tròn. - knockout: Loại trực tiếp. - hybrid: Hỗn hợp. - friendly: Giao hữu.
start_date	DATE	Thời gian diễn ra giải đấu.
end_date	DATE	Thời gian kết thúc giải đấu.
created_at	DATETIME	Thời gian tạo giải đấu.
updated_at	DATETIME	Thời gian cập nhật giải đấu.
sport_id	INT (FK)	Khóa ngoại tham chiếu đến bảng Sport.

Unique Key: league_name

9. Match

Tên trường	Kiểu dữ liệu	Mô tả
match_id	INT (PK, AUTO_INCREMENT)	Mã trận đấu.
team_1	INT (FK → Team.team_id)	Mã đội chủ nhà (hoặc đội thứ nhất).
team_2	INT (FK → Team.team_id)	Mã đội khách (hoặc đội thứ hai).

match_time	DATETIME	Thời gian diễn ra trận đấu.
stadium_id	INT (FK → Stadium.stadium_id)	Sân vận động tổ chức trận đấu.
league_id	INT (FK → League.league_id)	Mã định danh giải đấu.
round	VARCHAR	Vòng thi đấu (vòng loại, tứ kết, chung kết ...)
description	TEXT	Mô tả tổng quan về trận đấu.
created_at	DATETIME	Thời gian tạo trận đấu
is_hot_match	BOOLEAN	Cờ đánh dấu "Trận tâm điểm".
importance	INT	Mức độ quan trọng (Thang 1-5). Được tính toán tự động dựa trên Rating của 2 đội.

Unique Key: match_time + stadium_id, league_id + round_id + team_1 + team_2.

10. Section

Tên trường	Kiểu dữ liệu	Mô tả
section_id	INT (PK, AUTO_INCREMENT)	Mã khu vực ghế.
section_name	VARCHAR	Tên khu vực ghế.
stadium_id	INT (FK → Stadium.stadium_id)	Mã sân vận động.
capacity	VARCHAR	Số lượng ghế của section.

Unique Key: stadium_id + section_name.

Validate Logic (Clean Data):

Hai đội thi đấu (team_1, team_2) và giải đấu (league) phải thuộc cùng một môn thể thao (sport).

Đội 1 và Đội 2 không được trùng nhau.

11. Seat

Tên trường	Kiểu dữ liệu	Mô tả
seat_id	INT (PK, AUTO_INCREMENT)	Mã ghế.
section_id	INT (FK → Section.section_id)	Mã khu vực, hạng ghế.
seat_number	VARCHAR	Vị trí ghế (A1, A2, B1...)
seat_code	VARCHAR	Mã ghế (kết hợp stadium_code + section_name + seat_number. Ví dụ:

		MDVIP-A1)
status	ENUM('available', 'holding', 'reserved')	Trạng thái của ghế.

Unique Key: seat_code

12. Section Price

Tên trường	Kiểu dữ liệu	Mô tả
pricing_id	INT (PK, AUTO_INCREMENT)	Mã vé theo khu vực.
section_id	INT (FK →Section.section_id)	Mã khu vực trong sân vận động.
match_id	INT (FK →Match.match_id)	Mã trận đấu của vé.
price	DECIMAL	Giá vé của ghế đó cho trận đấu
available_seats	INT	Tổng số vé còn khả dụng.
is_closed	BOOLEAN	TRUE: đóng, FALSE: mở
sell_date	DATETIME	Thời gian mở bán vé
created_at	DATETIME	Thời gian khu vực bán vé được tạo.

Unique Key: match_id + section_id.

13. Price History

Tên trường	Kiểu dữ liệu	Mô tả
p_history_id	INT (PK, AUTO_INCREMENT)	Mã thay đổi giá.
pricing_id	INT (FK →SectionPrice.pricing_id)	Mã vé theo khu vực trong sân vận động.
changed_by	INT (FK →Employee.employee_id)	Mã nhân viên thay đổi giá.
old_price	DECIMAL	Giá cũ.
new_price	DECIMAL	Giá mới.
effective_date	DATETIME	Thời gian áp dụng giá mới.
changed_at	DATETIME	Thời gian tạo thay đổi.
reason	VARCHAR	Lý do thay đổi.

14. Order

Tên trường	Kiểu dữ liệu	Mô tả
------------	--------------	-------

order_id	INT (PK, AUTO_INCREMENT)	Mã đơn hàng.
<i>user_id</i>	INT (FK → Customer.customer_id)	Mã khách hàng mua vé.
total_amount	DECIMAL	Tổng số tiền của đơn hàng.
order_status	ENUM('holding', 'received', 'canceled')	Tình trạng đơn hàng.
order_method	BIT	Phương thức thanh toán (0: online, 1: offline)
created_at	DATETIME	Thời điểm đơn hàng được tạo.

15. Payment

Tên trường	Kiểu dữ liệu	Mô tả
payment_id	INT (PK, AUTO_INCREMENT)	Mã thanh toán.
<i>order_id</i>	INT (FK → Order.order_id)	Liên kết với đơn hàng.
total_amount	DECIMAL	Số tiền thanh toán cuối cùng.
payment_method	ENUM('credit card', 'bank transfer', 'cash')	Phương thức thanh toán.
payment_status	ENUM('successful', 'failed', 'pending')	Tình trạng giao dịch.
transaction_code	VARCHAR	Mã giao dịch.
expiration_time	DATETIME	Thời gian hết hạn giao dịch
created_at	DATETIME	Thời gian giao dịch.

Unique Key: transaction_code.

16. OrderDetail

Tên trường	Kiểu dữ liệu	Mô tả
detail_id	INT (PK, AUTO_INCREMENT)	Mã chi tiết đơn hàng
<i>order_id</i>	INT (FK → Order.order_id)	Mã đơn hàng.
<i>pricing_id</i>	INT (FK → SectionPricing.pricing_id)	Mã vé theo khu vực.
qr_code	VARCHAR	Mã qr code để quét.
<i>seat_id</i>	INT (FK → Seat.seat_id)	Mã ghế.
price	DECIMAL	Giá vé sau khi đã trừ đi khuyến mãi.

<i>promo_id</i>	INT (FK →Promotion.promo_id)	Mã khuyến mãi áp dụng cho vé.
updated_at	DATETIME	Thời điểm cập nhật vé.

Unique Key: order_id + seat_id.

17. Promotion

Tên trường	Kiểu dữ liệu	Mô tả
promo_id	INT (PK, AUTO_INCREMENT)	Mã khuyến mãi.
promo_code	VARCHAR	Mã giảm giá (unique).
description	TEXT	Mô tả chương trình khuyến mãi.
discount_type	ENUM('percentage', 'amount')	Loại giảm giá: theo % hoặc theo số tiền.
discount_value	DECIMAL	Giá trị giảm giá (ví dụ: 10% hoặc 50,000 VND).
start_time	DATETIME	Ngày bắt đầu khuyến mãi.
end_time	DATETIME	Ngày kết thúc.
usage_limit	INT	Số lần sử dụng tối đa (nếu có).
status	ENUM('active', 'inactive')	Trạng thái khuyến mãi.
<i>user_id</i>	INT (FK →Employee.employee_id)	Nhân viên tạo đợt khuyến mãi.

Unique Key: promotion_code

18. PromotionDetail

Tên trường	Kiểu dữ liệu	Mô tả
<i>promo_id</i>	INT (FK →Promotion.promo_id)	Mã khuyến mãi.
match_id	INT (FK →Match.match_id)	Mã trận đấu mà khuyến mãi áp dụng
section_id	INT (FK →Section.section_id)	Mã khu vực ghế ngồi mà khuyến mãi áp dụng

Unique Key: match_id + section_id.

19. TicketReturn

Tên trường	Kiểu dữ liệu	Mô tả
return_id	INT (PK, AUTO_INCREMENT)	Mã định danh của yêu cầu hoàn vé.

<i>detail_id</i>	INT (FK → OrderDetail.detail_id)	Mã chi tiết đơn hàng của vé cần được hoàn.
<i>return_date</i>	DATETIME	Thời gian gửi yêu cầu hoàn vé.
<i>refund_amount</i>	DECIMAL(10,2)	Số tiền hoàn trả cho vé.
<i>return_reason</i>	TEXT	Lý do yêu cầu hoàn vé.
<i>return_status</i>	ENUM('pending', 'rejected', 'completed') DEFAULT 'pending'	Trạng thái của yêu cầu hoàn vé
<i>return_method</i>	ENUM('credit card', 'bank transfer', 'cash')	
<i>processed_time</i>	DATETIME	Thời gian yêu cầu được xử lý hoàn tất.
<i>employee_id</i>	INT (FK → Employee.employee_id)	Mã nhân viên xử lý.
<i>note</i>	TEXT	Ghi chú nếu có sự kiện đặc biệt.

2.4 Thiết kế thuật toán AI - AGENT và học máy

Phần này trình bày thiết kế chi tiết cho hai phân hệ trí tuệ nhân tạo của hệ thống: Chatbot hỗ trợ khách hàng và Mô hình dự báo giá vé tối ưu.

2.4.1 Thiết kế Hệ thống Chatbot (Hybrid RAG & AI Agent)

Hệ thống Chatbot được thiết kế theo mô hình Hybrid (Lai ghép), kết hợp giữa tốc độ của FAQ và trí thông minh của Generative AI, đảm bảo nguyên tắc: "Không bịa đặt - Luôn ưu tiên dữ liệu thật".

a. Kiến trúc phân tầng (Layered Architecture)

Hệ thống được chia thành 3 tầng xử lý chính:

(A) Data Layer (Tầng dữ liệu):

- Nguồn dữ liệu: Các bảng MySQL (Match, Section, SectionPrice, Promotion).
- Cơ chế Indexing: Chuyển đổi dữ liệu có cấu trúc (SQL) thành văn bản mô tả tự nhiên (Document), sau đó lưu trữ vào Vector Database (ChromaDB) phục vụ tìm kiếm ngữ nghĩa.

(B) Intelligence Layer (Tầng trí tuệ):

- Embedding Model: Sử dụng model paraphrase-multilingual-MiniLM-L12-v2 để chuyển đổi văn bản thành vector số học đa chiều.
- LLM (Large Language Model): Sử dụng LLaMA-3.1-8b-instant (thông qua Groq API) để xử lý ngôn ngữ tự nhiên và sinh câu trả lời.

- **Session Memory:** Lưu trữ lịch sử hội thoại vào Database để AI hiểu ngữ cảnh (Context-aware).

(C) API Layer (Tầng điều phối):

- Tiếp nhận request từ Frontend, điều hướng luồng xử lý giữa FAQ và RAG Agent.

b. Thiết kế luồng xử lý dữ liệu (Data Flow Design)

Quy trình xử lý một câu hỏi của người dùng đi qua 8 bước chặt chẽ:

Input: Nhận câu hỏi, `customer_id`, và `session_id`.

FAQ Check: Kiểm tra từ khóa nhanh (Ví dụ: "xin chào", "mua vé"). Nếu khớp → Trả kết quả ngay (Bỏ qua AI).

Query Rewriting (Tái cấu trúc câu hỏi):

Sử dụng LLM để viết lại câu hỏi dựa trên lịch sử 3 câu hội thoại gần nhất.

Mục đích: Giải quyết các câu hỏi thiếu ngữ cảnh

Retrieval (Truy xuất thông tin - RAG):

Embed câu hỏi đã rewrite thành vector.

Thực hiện **Cosine Similarity Search** trong ChromaDB để lấy ra $k=3$ documents liên quan nhất (Lịch thi đấu, giá vé, khuyến mãi).

Prompt Engineering:

Xây dựng System Prompt với chỉ thị nghiêm ngặt: *"Chỉ sử dụng thông tin trong context, không được tự bịa. Nếu có ý định mua, hãy gợi ý link: {FRONTEND_URL}/match/<id>"*.

Generation: Gửi Prompt + Context + Lịch sử chat tới LLM để sinh câu trả lời tự nhiên.

History Logging: Lưu cặp câu hỏi - trả lời vào Database để phục vụ huấn luyện lại sau này.

Output: Trả về câu trả lời kèm theo Link đặt vé (nếu có) cho Frontend hiển thị.

2.4.2 Thiết kế Mô hình Dự báo Giá (Price Prediction Model)

Mô hình được thiết kế để giải quyết bài toán tối ưu hóa doanh thu dựa trên định giá động.

a. Quy trình lựa chọn mô hình (Model Selection)

Đề chọn ra thuật toán tối ưu, hệ thống thiết kế kịch bản thực nghiệm so sánh trên cùng một tập dữ liệu chuẩn hóa:

Hồi quy tuyến tính (Linear Regression): Được sử dụng làm Baseline. Kết quả cho thấy sai số cao do không nắm bắt được các điểm gãy khúc của thị trường (Ví dụ: Giá tăng nhẹ không giảm cầu, nhưng vượt ngưỡng tâm lý thì cầu giảm mạnh).

Facebook Prophet: Tốt cho dự báo chuỗi thời gian tuần tủy (Seasonality) nhưng khó tích hợp các biến ngoại lai (Độ hot trận đấu, Rating đội bóng).

Random Forest Regressor (Được chọn):

Hiệu quả vượt trội trong việc xử lý dữ liệu phi tuyến tính.

Học được logic "**Phân tầng giá**" (Tiered Pricing) và "**Cầu cứng**" (Inelastic Demand) đặc thù của thể thao.

b. Thiết kế Pipeline Huấn luyện (Training Pipeline)

Data Collection & Injection:

Thu thập dữ liệu lịch sử từ bảng `OrderDetail` và `Match`.

Tiêm (Inject) dữ liệu giả định dựa trên 4 phân tầng (Tiers): Tier 1 (Siêu Hot - Giá cao vẫn mua), Tier 2 (Sao số), Tier 3 (Trung bình), Tier 4 (Ế).

Preprocessing:

Mã hóa (Encoding) các biến phân loại: `is_hot_match`, `importance`.

Xử lý biến thời gian: Tách `match_time` thành `day_of_week`, `hour`.

Modeling:

Sử dụng `RandomForestRegressor` với tham số `n_estimators=300`.

Lưu trữ model (`joblib.dump`) để phục vụ dự báo thời gian thực.

Inference (Dự báo): API nhận thông tin trận đấu → Load Model → Trả về biểu đồ doanh thu dự kiến theo các mức giá khác nhau.

Chương 3: Triển khai hệ thống

3.1 Môi trường cài đặt và công cụ phát triển

Để đảm bảo hệ thống hoạt động ổn định, đồng bộ và dễ dàng mở rộng, việc thiết lập môi trường phát triển (Development Environment) được thực hiện dựa trên các công cụ và dịch vụ tiêu chuẩn công nghiệp sau đây:

3.1.1 Phần cứng và Hệ điều hành

- Máy tính phát triển: CPU Intel Core i5/i7 hoặc tương đương, RAM tối thiểu 8GB (Khuyến nghị 16GB để chạy Docker và các tác vụ huấn luyện AI nhẹ).
- Hệ điều hành: Windows 10/11 (Sử dụng WSL2) hoặc Ubuntu 20.04 LTS.

3.1.2 Môi trường phát triển ứng dụng (Application Stack)

Hệ thống được xây dựng trên mô hình Client-Server với các công nghệ lõi:

Ngôn ngữ lập trình:

Python 3.10+: Ngôn ngữ chính cho Backend và AI.

JavaScript (ES6+): Xử lý logic Frontend và kết nối WebSocket.

Framework & Thư viện Web:

Django 4.2: Xây dựng RESTful API và quản lý cấu trúc dự án.

Django Channels: Xử lý giao thức WebSocket cho tính năng thời gian thực.

HTML5 / CSS3 (Bootstrap): Xây dựng giao diện người dùng.

Cơ sở dữ liệu & Bộ nhớ đệm:

MySQL 8.0: Lưu trữ dữ liệu quan hệ bền vững.

Redis 6.0+: Server lưu trữ cấu trúc dữ liệu trong bộ nhớ, phục vụ Caching, Locking ghế và Message Broker cho WebSocket.

3.1.3 Môi trường tích hợp thanh toán (Payment Gateway)

Hệ thống tích hợp cổng thanh toán MoMo trên môi trường kiểm thử (Sandbox) để mô phỏng quy trình giao dịch thực tế mà không phát sinh chi phí thật.

Môi trường: MoMo Developer Sandbox (Testnet).

Cấu hình tích hợp:

- Partner Code: Mã định danh đối tác được cấp bởi MoMo.

- Access Key & Secret Key: Cặp khóa bảo mật dùng để tạo chữ ký điện tử (Signature) cho mỗi Request/Response.

Công cụ kiểm thử: Ứng dụng MoMo Test App cài đặt trên điện thoại di động để quét mã QR và xác thực thanh toán giả lập.

3.1.4 Môi trường phát triển Trí tuệ nhân tạo (AI Ecosystem)

Đây là phần môi trường phức tạp nhất, được chia thành hai nhóm phục vụ cho Chatbot và Dự báo giá.

a. Nhóm Generative AI (Cho Chatbot RAG & Agent):

Thay vì tự vận hành các mô hình ngôn ngữ lớn (LLM) tốn kém tài nguyên phần cứng, hệ thống sử dụng các dịch vụ API hiệu năng cao kết hợp với xử lý cục bộ:

Inference Engine (LLM Provider):

- Nền tảng: Groq Cloud API.
- Model: LLaMA-3.1-8b-instant.
- Lý do chọn: Tốc độ phản hồi cực nhanh (Low latency), phù hợp cho Chatbot thời gian thực.

Vector Database:

- Công cụ: ChromaDB.
- Chế độ: Chạy dưới dạng thư viện nhúng (Embedded) hoặc Server cục bộ để lưu trữ và truy xuất vector văn bản.

Embedding Model:

- Thư viện: sentence-transformers.
- Model: paraphrase-multilingual-MiniLM-L12-v2 (Hỗ trợ tốt đa ngôn ngữ, bao gồm tiếng Việt).

b. Nhóm Predictive AI (Cho Dự báo giá) Môi trường huấn luyện và chạy mô hình học máy truyền thống:

Thư viện lõi:

- Scikit-learn: Dùng để khởi tạo và huấn luyện thuật toán RandomForestRegressor.
- Pandas & NumPy: Dùng để làm sạch dữ liệu, xử lý ma trận và thao tác trên DataFrame.

Serialization:

- Joblib: Dùng để lưu (dump) mô hình đã huấn luyện thành file .pkl và nạp (load) lại vào Django khi khởi động server.

3.1.5 Công cụ quản lý và triển khai (DevOps Tools)

- IDE: Visual Studio Code (với các Extension hỗ trợ Python, Django, MySQL).
- API Testing: Postman (Dùng để kiểm thử các API Endpoint độc lập với Frontend).
- Quản lý mã nguồn: Git & GitHub/GitLab.
- Containerization (Tùy chọn): Docker & Docker Compose để đóng gói toàn bộ môi trường (Web, DB, Redis) thành các container độc lập, giúp việc triển khai đồng nhất trên mọi máy.

3.2 Các chức năng chính

3.2.1 Chức năng dành cho Quản lý

- **Quản lý tài nguyên:**
 - Thêm, sửa, xóa thông tin đội bóng, sân vận động.
 - Quản lý trận đấu (tạo, sửa, xóa thông tin: ngày giờ, đội tham gia, sân đấu, giải đấu).
 - Theo dõi chi tiết vé: tình trạng vé, các khu vực ghế, giá vé, số lượng vé còn lại.
 - Nhận và xử lý yêu cầu hoàn/đổi vé theo thời gian thực.
- **Quản lý tài khoản nhân viên:**
 - Tạo mới, chỉnh sửa, vô hiệu hóa/tái kích hoạt tài khoản nhân viên.
 - Quản lý vai trò, thông tin cá nhân nhân viên.
- **Quản lý doanh thu và báo cáo:**
 - Thống kê, báo cáo doanh thu (ngày, tháng, quý, năm), lợi nhuận.
 - Báo cáo doanh thu theo trận, khu vực, sân vận động.
 - Xuất file báo cáo (PDF, Excel).
 - Dashboard tình hình kinh doanh thời gian thực.
- **Quản lý đơn hàng:**
 - Theo dõi trạng thái đơn hàng (đang xử lý, hoàn tất, đã hủy).
 - Xem lịch sử đơn hàng, thống kê doanh số bán vé.

3.2.2 Chức năng dành cho Nhân viên

- **Hỗ trợ khách hàng:**
 - Tạo tài khoản, đặt vé hộ khách hàng mua trực tiếp.

- **Quản lý tài khoản cá nhân:**
 - Đăng nhập, đổi mật khẩu cá nhân.
- **Quản lý vé và thông tin trận đấu:**
 - Thêm, sửa, xóa vé (khu vực, giá vé).
 - Cập nhật giá vé, gửi thông báo giá mới.
 - Sửa thông tin trận đấu (khi có quyền).
- **Xử lý đơn hàng:**
 - Xem, duyệt đơn đặt vé trực tuyến.
 - Xuất hóa đơn điện tử, gửi mail/in cho khách.
- **Quản lý khuyến mãi:**
 - Thêm, sửa, xóa chương trình khuyến mãi.
- **Lịch sử giao dịch, đổi/trả vé:**
 - Xem lịch sử bán vé, hỗ trợ lập phiếu đổi trả, xác minh/bảo hành vé lỗi.

3.2.3 Chức năng dành cho Khách hàng

- **Quản lý tài khoản cá nhân:**
 - Đăng ký, đăng nhập (có thể đăng nhập bằng Google, Facebook).
 - Chỉnh sửa thông tin, đổi mật khẩu, xem lịch sử vé đã đặt.
- **Xem và đặt vé:**
 - Xem thông tin, lịch trận đấu, bộ lọc theo đội, ngày, giải đấu.
 - Đặt vé trực tuyến với sơ đồ ghế tương tác (chọn ghế, số lượng).
 - Đa dạng hình thức thanh toán: chuyển khoản, thẻ, ví điện tử, tiền mặt tại quầy.
 - Nhận vé điện tử qua email, có mã QR để vào sân.
- **Theo dõi đơn hàng, đổi trả/bảo hành vé:**
 - Theo dõi trạng thái đơn hàng.
 - Hủy đơn nếu chưa xử lý.
 - Gửi yêu cầu đổi trả, bảo hành vé qua website.

3.2.4 Chức năng kỹ thuật, vận hành và tích hợp

- **Quản lý rủi ro:**
 - Theo dõi, cảnh báo lỗi hệ thống; backup dữ liệu.

- Cập nhật số lượng vé thời gian thực, chống gian lận (mỗi vé có mã QR duy nhất).
- **Tích hợp:**
 - In vé trực tiếp tại quầy, quét mã QR tại cổng vào sân.
 - Tích hợp thanh toán online (Visa, MasterCard, Momo, ZaloPay...).
- **Phân quyền:**
 - Quản lý: toàn quyền hệ thống.
 - Nhân viên: truy cập hạn chế, đúng chức năng.
 - Khách hàng: chỉ dùng tài khoản cá nhân và đặt vé.

3.2.5 Báo cáo và thống kê

- Cung cấp báo cáo chi tiết về doanh thu, đơn hàng, giao dịch cho quản lý; báo cáo giao dịch cơ bản cho nhân viên.

3.3 Trí tuệ nhân tạo

3.3.1 Học máy - dự báo doanh thu và khuyến nghị giá

Một trong những tính năng quan trọng nhất của hệ thống là khả năng hỗ trợ người quản trị ra quyết định định giá vé dựa trên dữ liệu. Để hiện thực hóa điều này, nhóm nghiên cứu đã triển khai module học máy với nhiệm vụ dự báo tổng lượng vé bán ra (`total_sold`) và doanh thu dự kiến dựa trên các tham số đầu vào như giá vé, thời gian thi đấu và tính chất trận đấu.

Chiến lược chuẩn bị và Xử lý dữ liệu (Data Pipeline)

Do hệ thống mới được triển khai, dữ liệu lịch sử thực tế (Real Data) chưa đủ lớn để các mô hình học máy hội tụ tốt. Để giải quyết bài toán "khởi động lạnh" (Cold Start) này, nhóm đã xây dựng một chiến lược dữ liệu lai (Hybrid Data Strategy) kết hợp giữa dữ liệu thực tế trích xuất từ Database và dữ liệu giả lập dựa trên tri thức chuyên gia.

a. Trích xuất dữ liệu thực tế (Real Data Extraction)

Hệ thống sử dụng Django ORM để truy vấn dữ liệu từ các bảng nghiệp vụ. Quy trình trích xuất được thực hiện như sau:

Từ bảng `Match`, hệ thống lấy các thông tin ngữ cảnh: Ngày trong tuần (`day_of_week`), Giờ thi đấu (`hour`), Cờ đánh dấu trận HOT (`is_hot_match`) và Mức độ quan trọng (`importance`).

Từ bảng `SectionPrice`, tính toán giá vé trung bình (`avg_price`) của trận đấu.

Từ bảng `OrderDetail`, đếm tổng số vé đã bán thành công (`total_sold`).

b. Kỹ thuật Tiêm dữ liệu giả định (Synthetic Data Injection)

Để mô hình có thể học được các quy luật kinh tế phức tạp ngay cả khi thiếu dữ liệu thật, nhóm đã viết thuật toán tiêm dữ liệu (Data Injection) dựa trên 4 phân tầng thị trường (Market Tiers). Đoạn mã dưới đây minh họa logic sinh dữ liệu cho hai tầng điển hình:

Tier 1 - Sự kiện "Siêu Hot" (Inelastic Demand): Đây là các trận Quan trọng hoặc Chung kết (Importance = 5). Giả định hành vi khách hàng là "Cầu cứng" - tức là nhu cầu ít thay đổi theo giá.

Logic Code: Với mức giá dưới 800.000 VNĐ, tỷ lệ lấp đầy (Fill Rate) đạt 95-100%. Khi giá tăng lên 1.500.000 VNĐ, tỷ lệ lấp đầy vẫn duy trì ở mức cao (70-90%).

Mục đích: Dạy cho mô hình hiểu rằng với các trận đấu đặc biệt, việc tăng giá không làm giảm doanh thu tương ứng.

Tier 4 - Sự kiện "Kém hấp dẫn" (Elastic Demand): Đây là các trận đấu giữa các đội yếu hoặc giờ thi đấu xấu (Importance = 1-2).

Logic Code: Khách hàng cực kỳ nhạy cảm với giá. Nếu giá vé > 200.000 VNĐ, tỷ lệ lấp đầy sẽ giảm mạnh về mức 0-20%.

Mục đích: Giúp mô hình nhận diện điểm gãy (Breaking point) của thị trường để tránh việc Admin định giá quá cao cho các trận đấu này.

Cấu hình và Thử nghiệm so sánh các mô hình (Model Benchmarking)

Để chọn ra thuật toán tối ưu nhất, nhóm đã xây dựng một kịch bản kiểm thử (Benchmark Script) chạy song song 3 mô hình: Hồi quy tuyến tính (Linear Regression), Facebook Prophet, và Random Forest.

Dữ liệu được chia theo tỷ lệ 80% cho tập huấn luyện (Train Set) và 20% cho tập kiểm thử (Test Set). Tổng số mẫu dữ liệu sau khi gộp là 552 mẫu.

a. Thiết lập Hồi quy tuyến tính (Linear Regression) Mô hình này được sử dụng làm mốc cơ sở (Baseline).

Thư viện: `sklearn.linear_model.LinearRegression`.

Đặc điểm: Cố gắng tìm một phương trình đường thẳng $y = ax + b$ để mô tả mối quan hệ giữa Giá và Lượng vé bán.

b. Thiết lập Facebook Prophet Đây là mô hình chuyên dụng cho chuỗi thời gian, được kỳ vọng sẽ xử lý tốt các yếu tố ngày giờ.

Cấu hình: Tắt các thành phần mùa vụ tự động (`daily_seasonality=False`) do dữ liệu vé không liên tục hàng ngày.

Regressors: Thêm các biến nguyên nhân (price, hot, imp) vào mô hình để hỗ trợ dự báo.

Xử lý thời gian: Do Prophet yêu cầu cột ds (ngày tháng), nhóm đã phải thực hiện biến đổi dữ liệu giả lập từ day_of_week và hour sang dạng datetime ảo bắt đầu từ năm 2024.

c. Thiết lập Random Forest Regressor Mô hình rừng ngẫu nhiên được cấu hình để khai thác khả năng học phi tuyến tính.

Thư viện: sklearn.ensemble.RandomForestRegressor.

Tham số: n_estimators=300 (Số lượng cây quyết định), random_state=42 (Đảm bảo kết quả tái lập được).

Kết quả đánh giá và Phân tích hiệu năng

Sau khi chạy thực nghiệm trên tập Test (111 mẫu), kết quả thu được từ hệ thống log như sau:

Mô hình	Sai số tuyệt đối trung bình (MAE)	Độ chính xác (R ² Score)	Đánh giá
Linear Regression	157.9 vé	0.592	Hiệu năng thấp, không nắm bắt được xu hướng.
Prophet (Facebook)	285.6 vé	0.168	Rất kém, sai số lớn nhất.
Random Forest	39.3 vé	0.955	Xuất sắc, dự báo sát thực tế.

Bảng 1: Kết quả so sánh giữa 3 mô hình

Phân tích nguyên nhân:

- + Tại sao Prophet thất bại ($R^2 = 0.17$) Prophet được thiết kế để dự báo các chuỗi thời gian có tính lịch sử dài hạn (ví dụ: nhiệt độ hàng ngày, giá cổ phiếu). Trong bài toán bán vé, yếu tố quyết định không phải là "ngày nào" (Time-series) mà là "trận gì" và "giá bao nhiêu" (Cross-sectional attributes). Việc cố gắng ép dữ liệu sự kiện rời rạc vào mô hình chuỗi thời gian đã tạo ra nhiễu nhiều (noise), khiến Prophet dự báo sai lệch lớn (sai số trung bình lên tới 285 vé).
- + Tại sao Linear Regression chỉ đạt trung bình ($R^2 = 0.59$) Mô hình tuyến tính giả định rằng khi giá tăng, lượng bán sẽ giảm một cách đều đặn (tuyến tính). Tuy nhiên, dữ liệu thực tế (và dữ liệu tiêm Tier 1/Tier 4) cho thấy hành vi khách hàng có tính "bậc thang". Ví dụ: Ở Tier 1, giá tăng từ 500k lên 800k lượng bán gần như không đổi

(đường nằm ngang), nhưng qua mức 1.5 triệu thì giảm đột ngột. Linear Regression không thể uốn cong để khớp với các đoạn gãy khúc này.

+ Sự vượt trội của Random Forest ($R^2 = 0.96$): Random Forest, với bản chất là tập hợp các cây quyết định (if-else), cực kỳ phù hợp để học các quy tắc phân tầng. Nó dễ dàng học được các logic như: "Nếu là trận HOT (Tier 1) VÀ Giá < 1.5tr THÌ Bán = 100%". Sai số chỉ 39.3 vé trên tổng sức chứa 1200 chỗ (tương đương 3.2%) là con số lý tưởng để áp dụng vào thực tế.

Kết luận và Tích hợp hệ thống

Dựa trên kết quả thực nghiệm cho thấy Random Forest đạt độ chính xác vượt trội ($R^2 = 0.96$) so với các mô hình khác, nhóm phát triển đã quyết định lựa chọn thuật toán này làm lõi xử lý chính thức. Quy trình tích hợp mô hình vào hệ thống Django được thực hiện thông qua một quy trình khép kín gồm: Chuẩn bị dữ liệu lại → Huấn luyện → Đóng gói → Mô phỏng & Tối ưu hóa.

Dưới đây là chi tiết kỹ thuật của quá trình tích hợp dựa trên mã nguồn thực tế:

a. Kiến trúc Mô-đun Huấn luyện (Training Module Architecture)

Mô-đun huấn luyện được tách biệt thành một Django Management Command (BaseCommand), cho phép kích hoạt quá trình học lại (Retrain) định kỳ.

Quy trình xử lý dữ liệu huấn luyện được chia làm 2 giai đoạn:

+ Giai đoạn 1 (Real Data): Trích xuất dữ liệu lịch sử từ bảng OrderDetail và Match.

+ Giai đoạn 2 (Data Injection): Tiêm tri thức chuyên gia để khắc phục vấn đề "khởi động lạnh". Hệ thống dạy cho mô hình hiểu các quy luật như "Cầu cứng" (Inelastic Demand) ở các trận HOT và độ nhạy cảm về giá ở các trận đấu thường.

b. Cơ chế Đóng gói (Model Serialization)

Sau khi huấn luyện, mô hình RandomForestRegressor (300 cây quyết định) được đóng gói thành file nhị phân price_optimization_model.pkl sử dụng thư viện joblib. Việc này giúp tách biệt quá trình "Học" (tốn tài nguyên) và quá trình "Dự báo" (cần tốc độ tức thì).

c. Triển khai API Gọi ý giá và Thuật toán Tối ưu hóa (Inference & Optimization Engine)

Đây là phân hệ phức tạp nhất, được triển khai tại API SuggestOptimalPriceView. Thay vì chỉ dự báo doanh thu cho một mức giá đơn lẻ, hệ thống thực hiện kỹ thuật Mô phỏng Monte Carlo giản lược để tìm ra mức giá tối ưu nhất (Sweet Spot).

Quy trình xử lý trong API diễn ra qua 4 bước:

Bước 1: Thiết lập không gian tìm kiếm (Price Range Scanning) Hệ thống không tìm kiếm mù quáng mà áp dụng Luật kinh doanh (Business Rules) để giới hạn dải giá quét dựa trên tính chất trận đấu (importance):

- + Trận Siêu Hot (Tier 1): Quét dải giá rộng từ 50.000đ đến 1.500.000đ.
- + Trận Trung bình (Tier 3): Giới hạn trần ở mức 500.000đ.
- + Trận Ế (Tier 4): Chỉ cho phép tối đa 300.000đ.
- + Bước nhảy (Step): 20.000đ để đảm bảo độ mịn của biểu đồ.

Bước 2: Vòng lặp Mô phỏng (Simulation Loop) Với mỗi mức giá P trong dải quét, hệ thống gọi mô hình AI để dự báo lượng vé bán ra Q:

$$+ Q_{\text{pred}} = \text{Model}(\text{Day}, \text{Hour}, \text{Hot}, \text{Importance}, P)$$

Sau đó tính toán hai chỉ số quan trọng:

- + Doanh thu dự kiến: $R = Q_{\text{pred}} \times P$
- + Tỷ lệ lấp đầy: $\text{FillRate} = (Q_{\text{pred}} / \text{Capacity}) \times 100$

Bước 3: Hàm mục tiêu đa biến (Multi-objective Scoring Function) Để đưa ra gợi ý tốt nhất, hệ thống không chỉ tối đa hóa doanh thu (vì có thể dẫn đến giá quá cao, sân vắng tanh) mà sử dụng hàm chấm điểm cân bằng:

- + Score = $(\text{NormalizedRevenue} \times 0.7) + (\text{FillRate} \times 0.3)$
- + Trọng số: Ưu tiên 70% cho Doanh thu và 30% cho Tỷ lệ lấp đầy sân.

Cơ chế phạt (Penalty): Nếu tỷ lệ lấp đầy < 40%, điểm số bị trừ nặng (-30 điểm) để tránh các phương án "giá cắt cổ" khiến sân vận động trống vắng, ảnh hưởng đến hình ảnh giải đấu.

```
rev_score = (sim['revenue'] / max_revenue_found) * 100 if max_revenue_found > 0 else 0

fill_score = sim['fill_rate'] # 0-100

final_score = (rev_score * 0.7) + (fill_score * 0.3)

# Phạt nặng nếu lấp đầy quá thấp (< 40%) -> Trừ 30 điểm

# Để tránh việc AI chọn giá cắt cổ bán cho 10 người

if sim['fill_rate'] < 40:

    final_score -= 30
```

Bước 4: Trả về kết quả và Biểu đồ API trả về mức giá có Score cao nhất, kèm theo lý do gợi ý (ví dụ: "PROFIT_MAX" cho trận Hot, "SAFE_OPTION" cho trận ế) và mảng dữ liệu chart_data để Frontend vẽ biểu đồ đường, giúp Admin có cái nhìn trực quan về sự đánh đổi giữa Giá và Lượng bán.

d. Kết luận chung về phân hệ AI

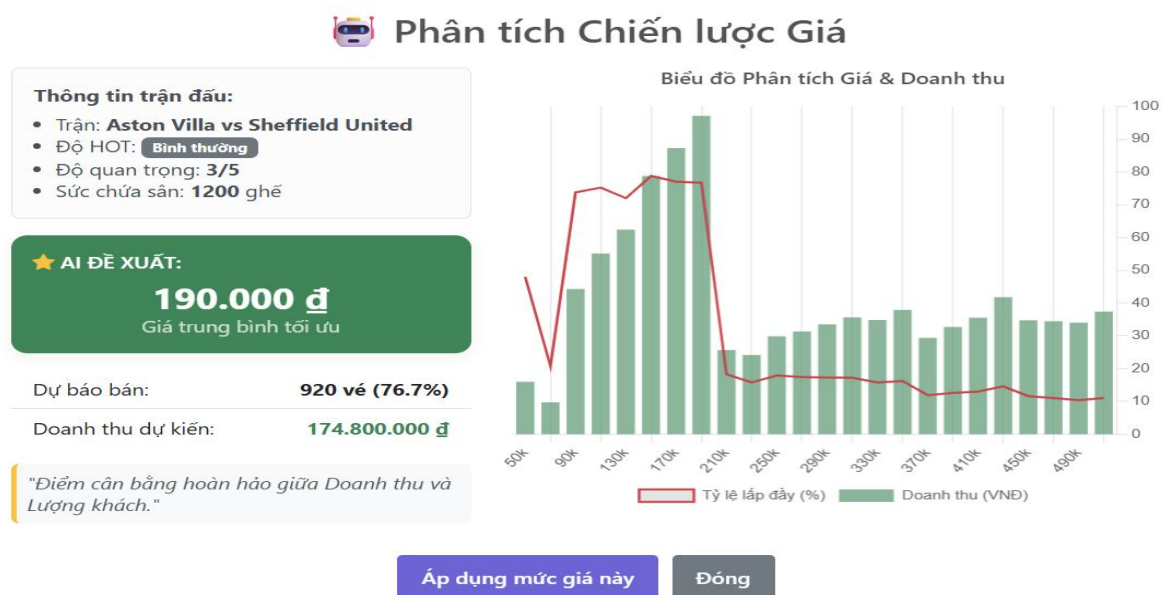
Việc tích hợp thành công mô hình Random Forest cùng thuật toán tối ưu hóa đa biến đã mang lại cho hệ thống khả năng:

Tự động hóa định giá: Giảm thiểu sự cảm tính của con người.

Cân bằng lợi ích: Đảm bảo doanh thu cao nhất nhưng vẫn giữ được không khí sôi động trên sân (Fill Rate).

Trực quan hóa: Cung cấp công cụ hỗ trợ ra quyết định (DSS) mạnh mẽ thông qua biểu đồ mô phỏng.

Hệ thống đã sẵn sàng để triển khai thực tế, đóng vai trò là trợ lý đắc lực cho nhà quản lý sự kiện.



Hình 13: Giao diện AI khuyến nghị giá

3.3.2 CHATBOT RAG + AI AGENT

- **Cơ sở lý thuyết**

Hệ thống chatbot trong đề tài được xây dựng dựa trên hai thành phần cốt lõi: RAG (Retrieval-Augmented Generation) và LLM (Large Language Model). Hai thành phần này kết hợp chặt chẽ để chatbot vừa có khả năng hiểu câu hỏi tự nhiên, vừa trả lời đúng theo dữ liệu thực tế từ hệ thống bán vé thể thao.

RAG (Retrieval-Augmented Generation)

Khái niệm

RAG là phương pháp kết hợp giữa kỹ thuật truy xuất thông tin (Retrieval) và mô hình sinh ngôn ngữ (Generation).

Thay vì để LLM tự “nghĩ ra” câu trả lời, RAG buộc mô hình phải dựa trên ngữ liệu thật được truy xuất từ cơ sở dữ liệu.

Nhờ đó, câu trả lời:

- Chính xác hơn, không bị “bịa” (hallucination)
- Cập nhật theo dữ liệu mới nhất
- Có dẫn chứng từ vector database

Cơ chế hoạt động

RAG trong đề tài hoạt động qua ba bước:

(1) Chuẩn hóa dữ liệu → Embedding → Lưu vào Vector Database

Các thông tin liên quan đến trận đấu, thời gian, giá vé, khu vực, khuyến mãi... được chuyển thành dạng văn bản rồi đưa vào mô hình embedding để tạo vector. Các vector này được lưu vào ChromaDB.

Mỗi vector gắn kèm metadata:

- match_id
- tên trận
- giá vé
- khu vực
- trạng thái còn/cháy vé

→ Đây chính là “tri thức” để chatbot tra cứu.

(2) Người dùng đặt câu hỏi → Hệ thống chuyển câu hỏi thành vector

Câu hỏi sau khi được rewrite (làm rõ nghĩa) sẽ được embedding thành vector.

(3) Tìm kiếm ngữ nghĩa (Semantic Search) trong Chroma

Vector của câu hỏi được so sánh với vector dữ liệu bằng **cosine similarity**, trả về top-K kết quả phù hợp nhất.

Kết quả tìm kiếm gồm:

- Các đoạn context mô tả đúng trận đấu liên quan
- match_id của trận phù hợp nhất
- Dữ liệu để LLM sử dụng khi sinh câu trả lời

Kết luận về vai trò RAG:

RAG giúp đảm bảo chatbot chỉ trả lời bằng dữ liệu đúng, phản ánh chính xác số lượng vé còn lại, giá vé, thời gian diễn ra, khu vực sân cỏ.

LLM (Large Language Model) – AI Agent

Khái niệm

LLM là mô hình học sâu được huấn luyện trên hàng tỷ câu để có khả năng:

- Hiểu ngôn ngữ tự nhiên
- Suy luận theo bối cảnh
- Viết lại câu hỏi
- Sinh câu trả lời như con người

Đề tài sử dụng mô hình LLaMA 3.1 thông qua nền tảng Groq với tốc độ phản hồi nhanh.

Hai nhiệm vụ chính của LLM trong hệ thống**(1) Rewrite Query – Diễn giải lại câu hỏi**

Người dùng thường hỏi thiếu ngữ cảnh:

- “Khu A còn không?”
- “Giá bao nhiêu vậy?”
- “Có khuyến mãi không?”

LLM sẽ dựa vào 3 tin nhắn trước đó + câu hỏi hiện tại để viết lại câu hỏi đầy đủ:

Ví dụ:

→ “Tình trạng vé khu A của trận Hà Nội vs HAGL ngày 21/01.”

Rewrite giúp RAG tìm đúng dữ liệu hơn.

(2) Sinh câu trả lời cuối cùng

Sau khi RAG trả về context, LLM thực hiện:

- Tổng hợp dữ liệu
- Trả lời rõ ràng, tự nhiên
- Không được phép bịa vì có system prompt ràng buộc
- Tạo link trận đấu theo match_id để người dùng bấm mua vé

Ví dụ câu trả lời:

“Trận Hà Nội vs HAGL diễn ra lúc 19:00, khu A còn 12 vé giá 350.000đ. Bạn có thể mua tại đây: /match/15.”

Toàn bộ LƯỒNG XỬ LÝ của chatbot

Toàn bộ kiến trúc chatbot bao gồm ba thành phần chính: FAQ trả lời nhanh, RAG để truy xuất dữ liệu thật, và AI Agent (LLM) để hiểu ngữ cảnh và tạo phản hồi. Luồng hoạt động chi tiết được mô tả như sau:

1. Tiếp nhận câu hỏi từ người dùng

Khi người dùng gửi tin nhắn qua widget chatbot, frontend (chatbot.js) gọi API:

```

fetch(API_URL, {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    message: text,
    customer_id: localStorage.getItem('customer_id'),
    session_id: currentSessionId
  })
})

```

Backend Django tiếp nhận trong views.py:

```

def chatbot_api(request):
    data = json.loads(request.body)
    user_message = data.get("message")
    customer_id = data.get("customer_id")
    session_id = data.get("session_id")

```

Ý nghĩa triển khai:

Hệ thống thu được đầy đủ 3 thông tin quan trọng (message + user + session) để tạo ngữ cảnh hội thoại.

2. Kiểm tra câu hỏi có thuộc FAQ hay không (faq_service.py)

Bước đầu tiên giúp chatbot trả lời nhanh những câu phổ biến, giảm chi phí AI.

Triển khai:

```

def normalize(text):
    text = text.lower().strip()
    text = ".join(c for c in unicodedata.normalize('NFD', text)
                  if unicodedata.category(c) != 'Mn')
    return text

faq_data = {
    ("xin chao", "hello"): "Xin chào! Tôi có thể hỗ trợ bạn.",
    ("khuyen mai", "uu dai"): "Bạn có thể xem khuyến mãi trên trang chủ.",
    ("thanh toan", "tra tien"): "Hệ thống hỗ trợ thẻ ngân hàng, ví điện tử, QR.",
    ("huy ve", "doi ve"): "Bạn có thể hủy/đổi vé nếu đủ điều kiện.",
}

```

```
def get_faq_answer(user_input):
    normalized = normalize(user_input)
    for keywords, answer in faq_data.items():
        if normalized in keywords:
            return answer
    return None
```

Kết quả:

Những câu hỏi dạng "xin chào", "khuyến mãi", "thanh toán " sẽ được trả luôn, không cần AI.

3. Rewrite Query bằng AI (nlp_service.py)

Đây là phần quan trọng để AI hiểu đúng ngữ cảnh hội thoại.

```
last_chats = ChatHistory.objects.filter(session_id=session_id).order_by("-created_at")[:3]
history_text = ""
for chat in reversed(last_chats):
    history_text += f"Người dùng: {chat.user_message}\nBot: {chat.bot_response}\n"
```

Sau đó gửi vào LLM Groq:

```
response = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {"role": "system", "content": "Bạn là trợ lý giúp diễn giải câu hỏi"},
        {"role": "user",
         "content": f"Lịch sử hội thoại:\n{history_text}\n\nNgười dùng vừa hỏi: {user_message}"
        }
    ]
)
```

Ví dụ rewrite thực tế:

“Còn khu A không?”
→ rewrite thành

“Người dùng hỏi: tình trạng vé khu A của trận Hà Nội vs HAGL ngày 21/01.”

4. RAG – Truy xuất dữ liệu thật bằng Vector Database Chroma (db_service.py)

4.1 Build index từ SQL sang Vector

Triển khai conversion Match → Document:

```
text = (
    f'match_id {match_id}, Giải {league_name} ({sport_type}), "
    f'Trận {match_name}, Thời gian {match_time}, "
    f'Khu vực {section}, giá {price:,}đ, {status} {promo_text}, còn {seats}
    chỗ."
)
docs.append(Document(page_content=text,metadata={"match_id": match_id}))
```

Sau đó lưu vào Chroma:

```
db = Chroma.from_documents(
    docs, embedding=embeddings, persist_directory=CHROMA_PATH
)
```

4.2 Search ngữ nghĩa

```
db=Chroma(persist_directory=CHROMA_PATH,
embedding_function=embeddings)
results = db.similarity_search(user_message, k=3)
RAG đảm bảo AI không bịa, chỉ dùng dữ liệu thật từ hệ thống Match – Section – Price – Promotion.
```

5. AI Agent – Sinh câu trả lời cuối cùng (nlp_service.py) và trả kết quả về giao diện người dùng

Sau khi có *context* từ RAG, hệ thống gọi AI Agent để tổng hợp câu trả lời:

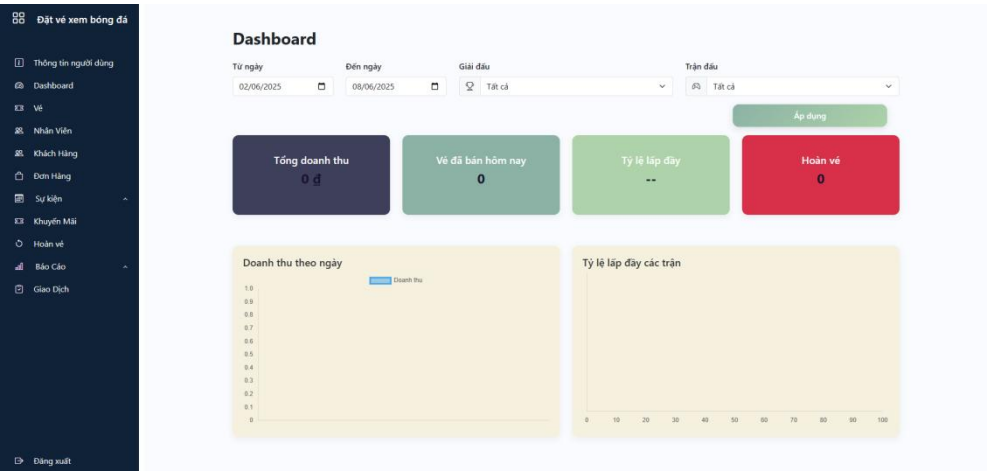
```
response = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {"role": "system",
         "content": "Luôn trả lời dựa trên context, không được tự bịa."},
        {"role": "user",
         "content": f'Context:\n{context}\n\nCâu hỏi: {user_message}'}
    ]
)

answer = response.choices[0].message.content
```


CHƯƠNG 4: KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ

4.1 Kiểm thử chức năng hệ thống và giao diện

4.1.1 Màn hình Dashboard



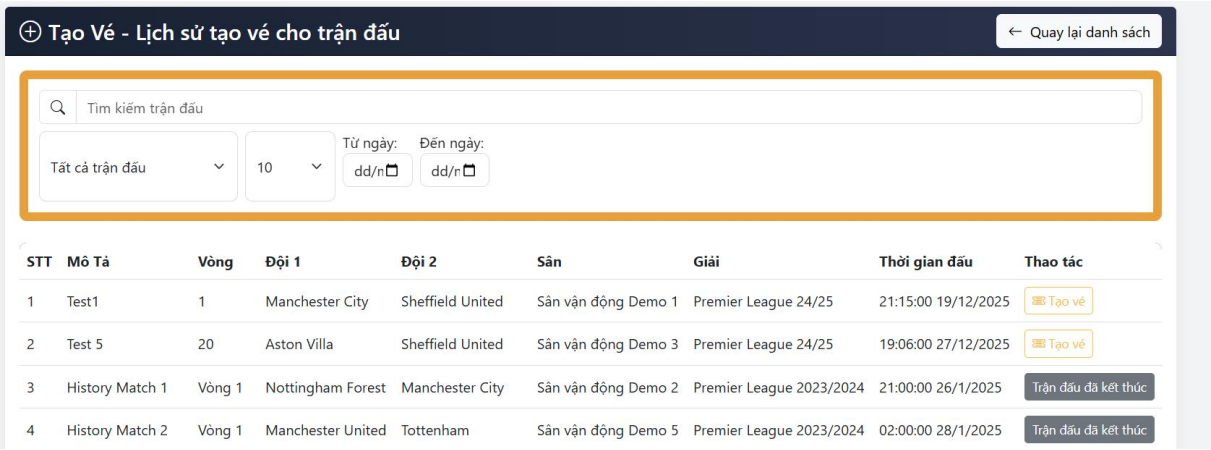
Hình 14: Màn hình Các chỉ số thống kê

4.1.2 Màn hình quản lý vé

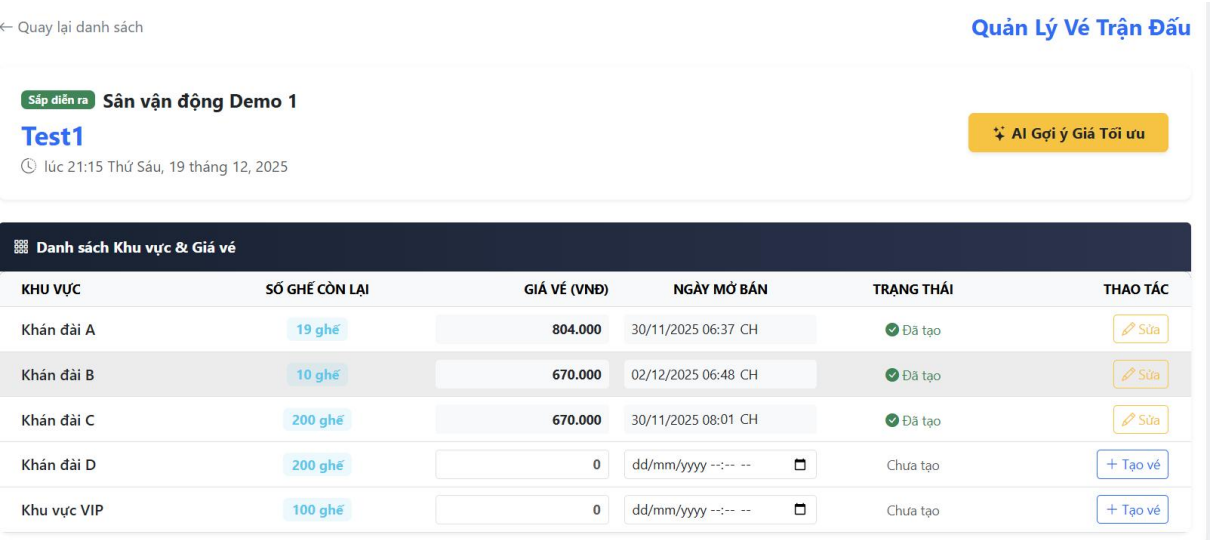
The screenshot shows the 'Quản lý Vé' interface with a search bar and filters. Below is a table with the following data:

STT	Tên trận	Tên sân vận động	Tên khu vực	Giá vé	Số ghế còn	Trạng thái	Ngày bán	Thời gian thi đấu	Hành động
17	Trận giao lưu	Olympic Stadium	A	1.200.000 đ	5	Đang đóng	2025-06-10T05:40:00	2025-06-20T16:35:00	Đã Vé chưa bán Xóa
1	Trận 1	Bernabeo	A	1.000.000 đ	3	Đang đóng	2025-05-03T21:12:00	2025-05-03T21:15:00	Trận đã đấu
2	Trận 1	Bernabeo	B	4.000.000 đ	4	Đang đóng	2025-05-03T21:12:00	2025-05-03T21:15:00	Trận đã đấu
3	Trận 1	Bernabeo	C	300.000 đ	5	Đang đóng	2025-05-03T21:14:00	2025-05-03T21:15:00	Trận đã đấu
4	Trận 2	Bernabeo	B	72.000 đ	0	Đang đóng	2025-05-03T21:12:00	2025-05-25T21:12:00	Trận đã đấu
5	Trận 2	Bernabeo	A	60.000 đ	0	Đang đóng	2025-05-03T21:17:00	2025-05-25T21:12:00	Trận đã đấu
6	Trận 2	Bernabeo	C	80.000 đ	0	Đang đóng	2025-05-03T21:18:00	2025-05-25T21:12:00	Trận đã đấu
7	Trận 3	Bernabeo	A	1.000.000 đ	0	Đang đóng	2025-05-03T21:23:00	2025-05-17T21:20:00	Trận đã đấu
8	Trận 3	Bernabeo	B	91.000 đ	0	Đang đóng	2025-05-05T08:53:00	2025-05-17T21:20:00	Trận đã đấu

Hình 15: Màn hình quản lý vé

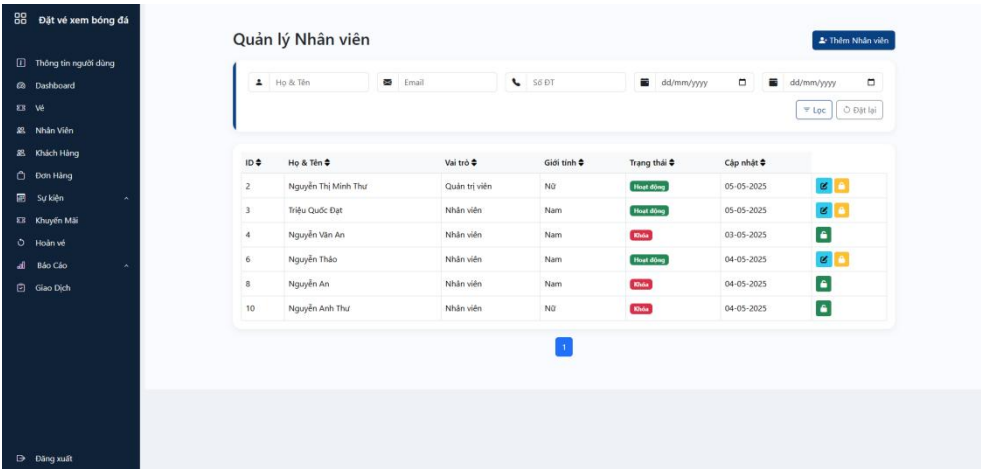


Hình 16: Màn hình tạo vé



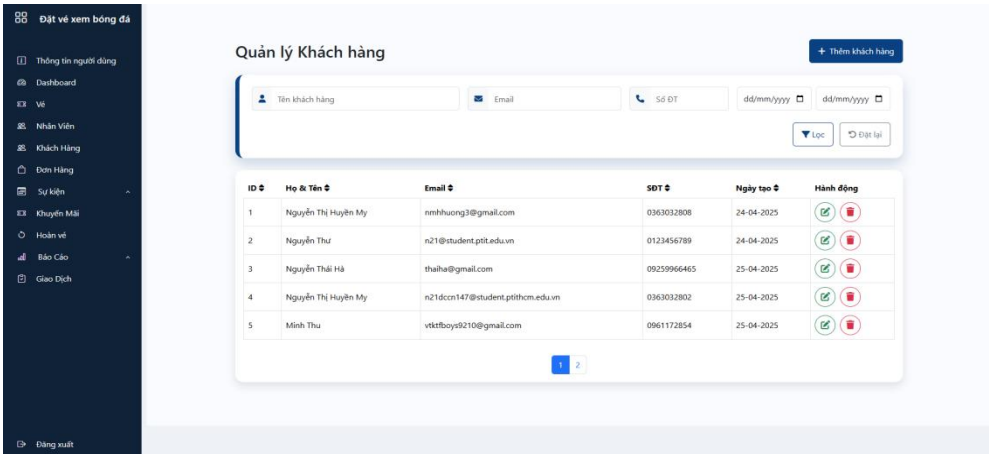
Hình 17: Thao tác tạo vé cho trận đấu

4.1.3 Màn hình quản lý nhân viên



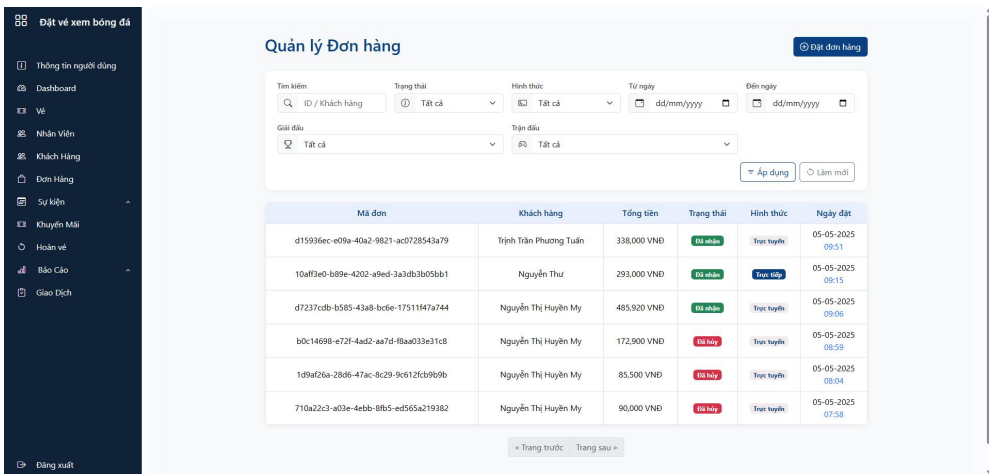
Hình 18: Màn hình Quản lý nhân viên

4.1.4 Màn hình quản lý thông tin khách hàng



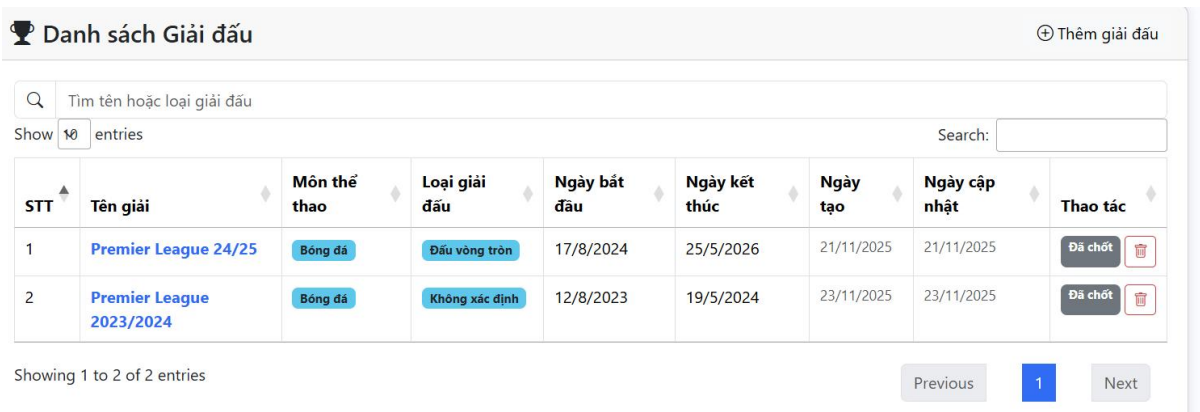
Hình 19: Màn hình Quản lý thông tin khách hàng

4.1.5 Màn hình quản lý đơn hàng



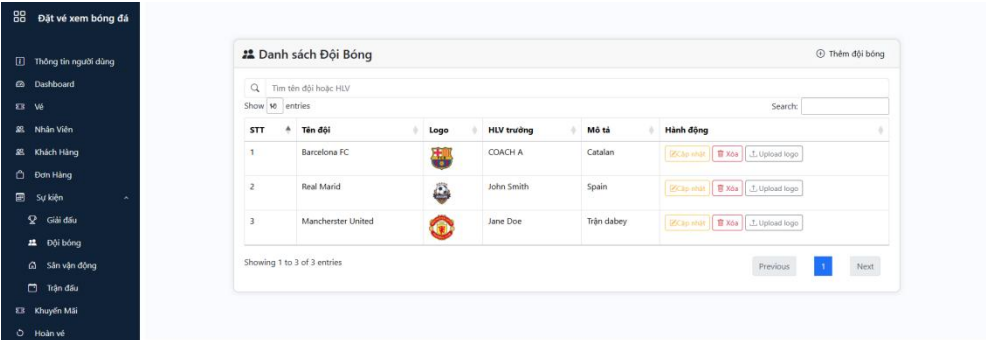
Hình 20: Màn hình Quản lý đơn hàng

4.1.6 Màn hình quản lý giải đấu



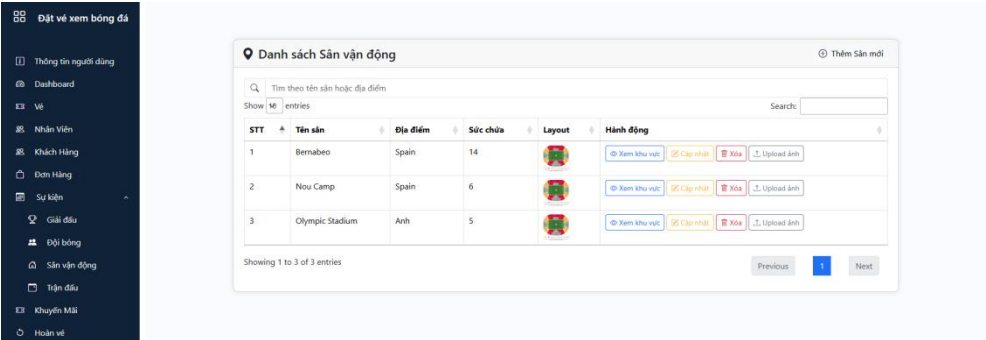
Hình 21: Màn hình Quản lý danh sách các giải đấu

4.1.7 Màn hình quản lý các đội

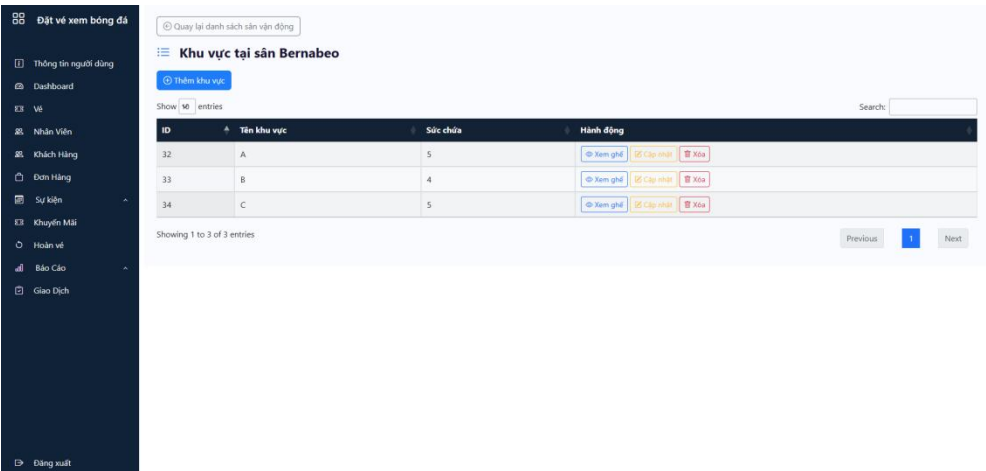


Hình 22: Màn hình Danh sách các đội

4.1.8 Màn hình quản lý sân vận động

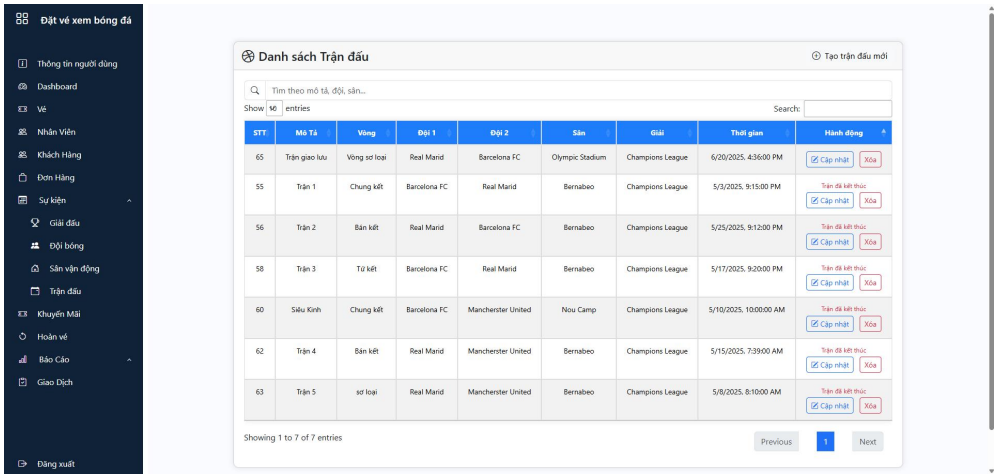


Hình 23: Màn hình Danh sách các sân vận động

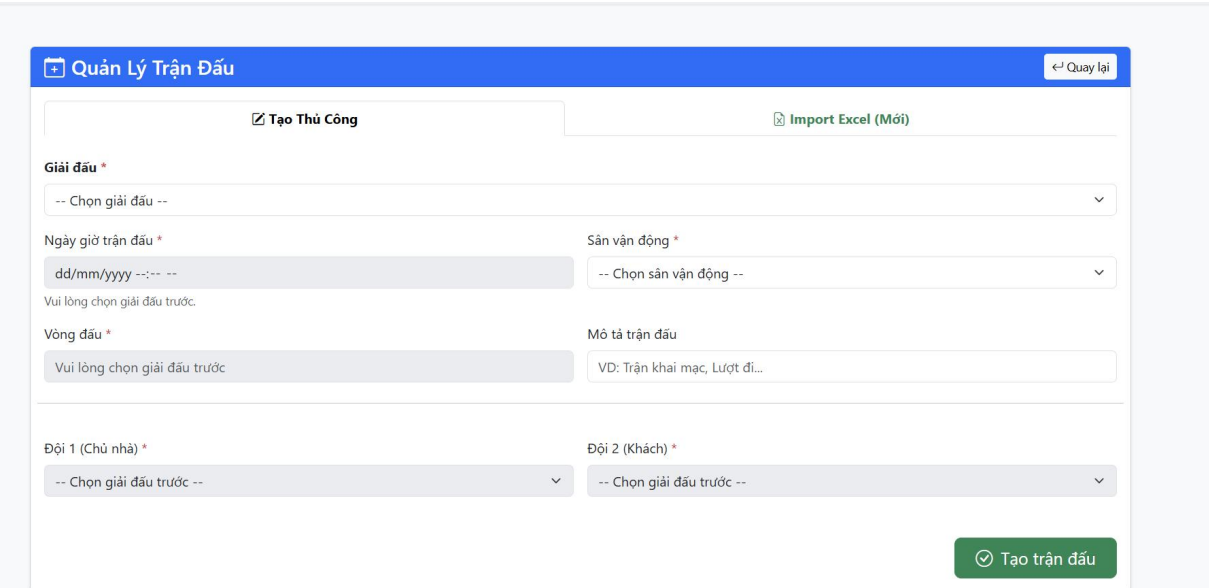


Hình 24: Danh sách khu vực của từng sân

4.1.9 Màn hình quản lý trận đấu

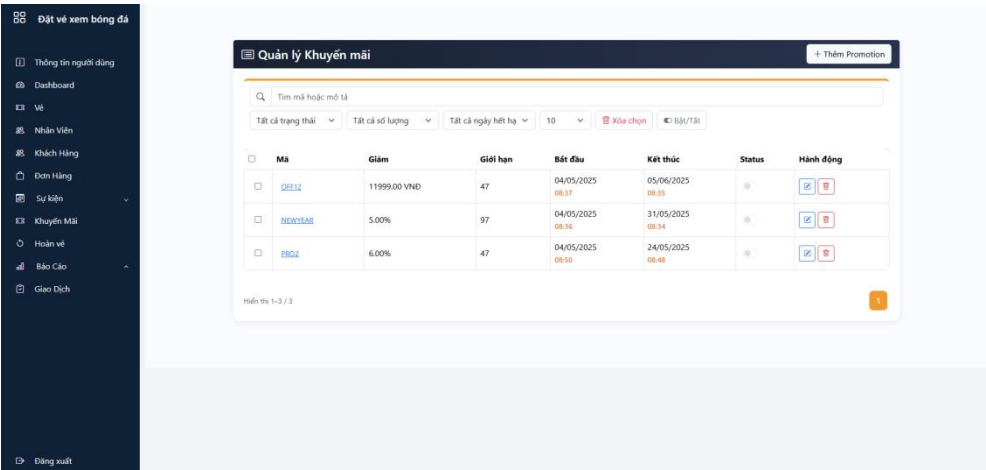


Hình 25: Màn hình Danh sách các trận đấu



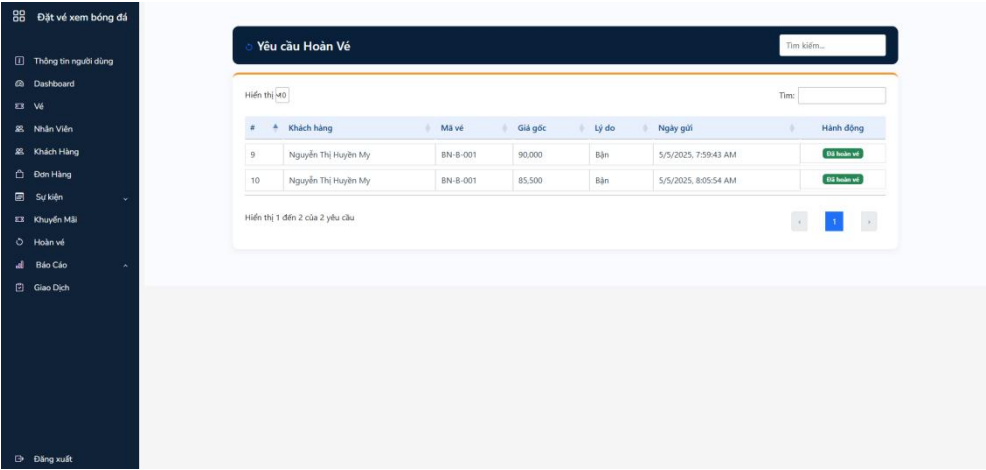
Hình 26: Màn hình thêm lịch thi đấu

4.1.10 Màn hình quản lý khuyến mãi



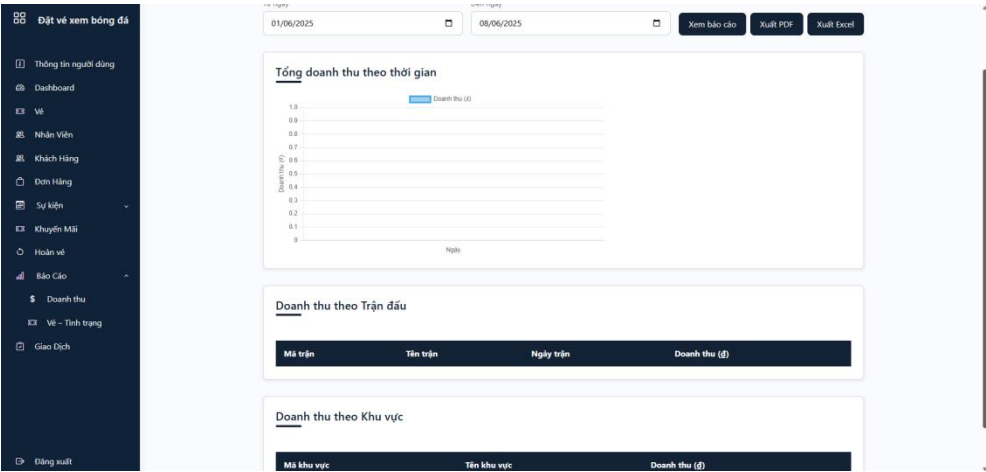
Hình 27: Màn hình Danh sách các đợt khuyến mãi

4.1.11 Màn hình quản lý hoàn vé



Hình 28: Màn hình quản lý hoàn vé

4.1.12 Màn hình báo cáo



Hình 29: Màn hình Báo cáo theo doanh thu

4.1.13 Màn hình quản lý các giao dịch

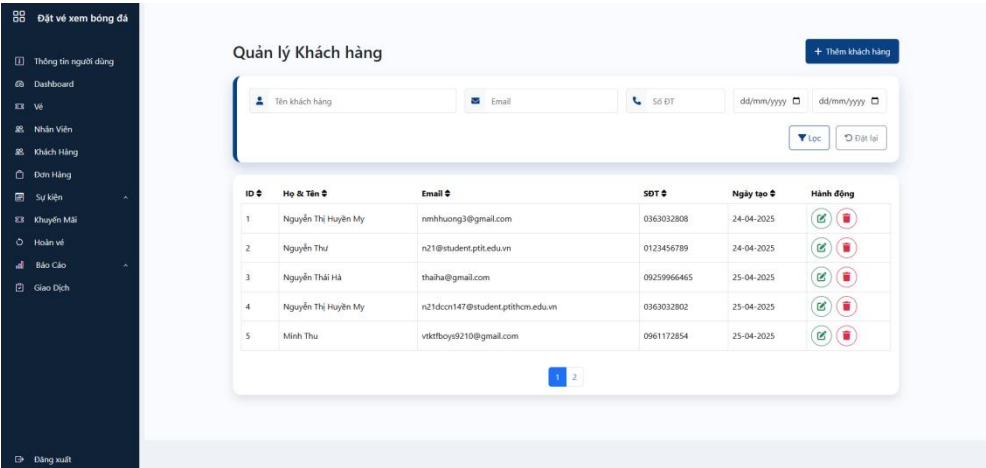
The screenshot shows a 'Quản lý Thanh toán' (Payment Management) interface. It includes a search bar with 'Mã đơn / Mã giao dịch' and a filter dropdown set to 'Tất cả'. There are also date range inputs for 'Từ ngày' and 'Đến ngày'. Below the search bar is a table of transactions. The table has columns: ID, Mã đơn, Phương thức, Trạng thái, Mã giao dịch, Ngày tạo, and Thao tác. The data rows are as follows:

ID	Mã đơn	Phương thức	Trạng thái	Mã giao dịch	Ngày tạo	Thao tác
107	d15936ec-e09a-40a2-9821-ac0728543a79	Chuyển khoản	Thành công	3301359421	09:51:01 5/5/2025	[icon]
106	10aff3e0-b89e-4202-a9ed-3a3db3b05bb1	Tiền mặt	Thành công	CASH-10aff3e0-b89e-4202-a9ed-3a3db3b05bb1-1746411315	09:15:15 5/5/2025	[icon]
105	d7237c0b-b585-43a0-bc6e-1751147a744	Chuyển khoản	Thành công	3301359144	09:06:15 5/5/2025	[icon]
104	b0c14699-e72f-4ae2-aa7d-f8aa033e31c8	Chuyển khoản	Thất bại	null	08:59:28 5/5/2025	[icon]
103	1d9af05e-20d6-47ac-8c29-8cd12fc98f8b	Chuyển khoản	Thành công	3301358822	08:04:36 5/5/2025	[icon]
102	710a22c3-a03e-4ebb-8f55-ed595a219382	Chuyển khoản	Thành công	3301358756	07:58:45 5/5/2025	[icon]

At the bottom of the table, there are navigation links: '« Trang trước' and 'Trang sau »'.

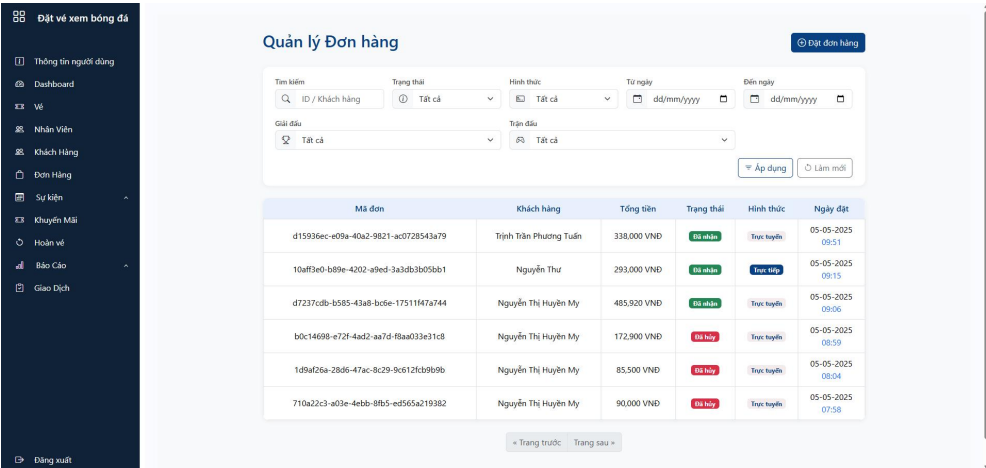
Hình 30: Màn hình Quản lý các giao dịch

4.1.14 Màn hình quản lý khách hàng



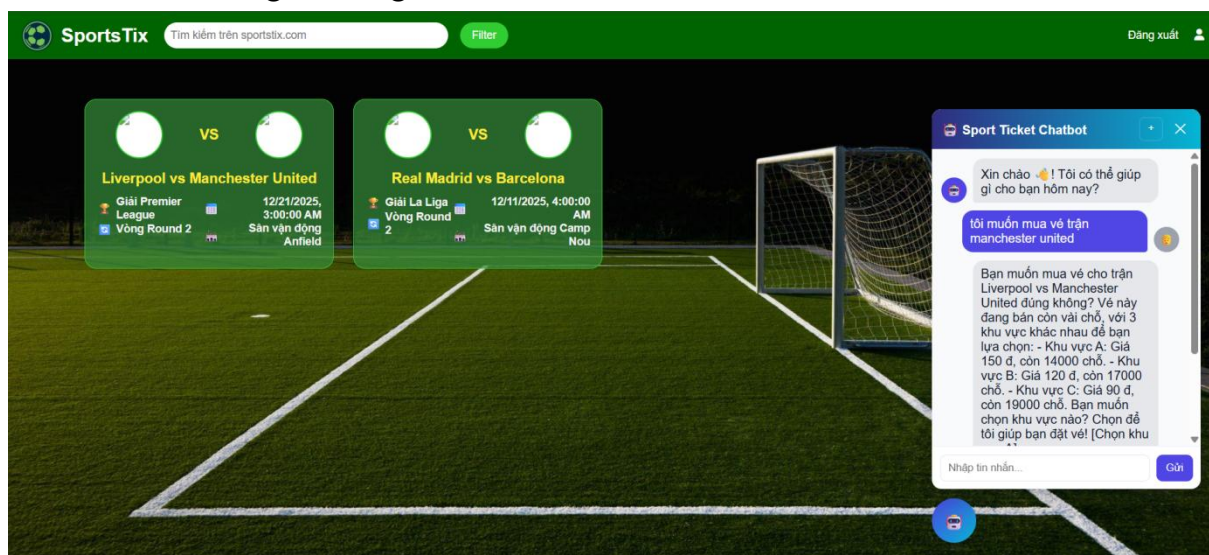
Hình 31: Màn hình Quản lý thông tin khách hàng

4.1.15 Màn hình quản lý đơn hàng



Hình 32: Màn hình Quản lý đơn hàng

4.1.16 Giao diện người dùng và Chatbot AI



Hình 33: Giao diện người dùng và Chatbot hỗ trợ khách hàng

4.2 Kiểm thử chức năng hoàn vé - phản hồi qua email

Hệ thống tạo nội dung email động dựa trên dữ liệu vé và thao tác của nhân viên. Dưới đây là nội dung thực tế khách hàng nhận được:

a. Trường hợp Chấp nhận hoàn vé

Tiêu đề: [Hoàn vé] Yêu cầu #50 đã được CHẤP NHẬN

Nội dung: Chào Nguyen Van A,

Yêu cầu hoàn vé của bạn đã được chấp nhận.

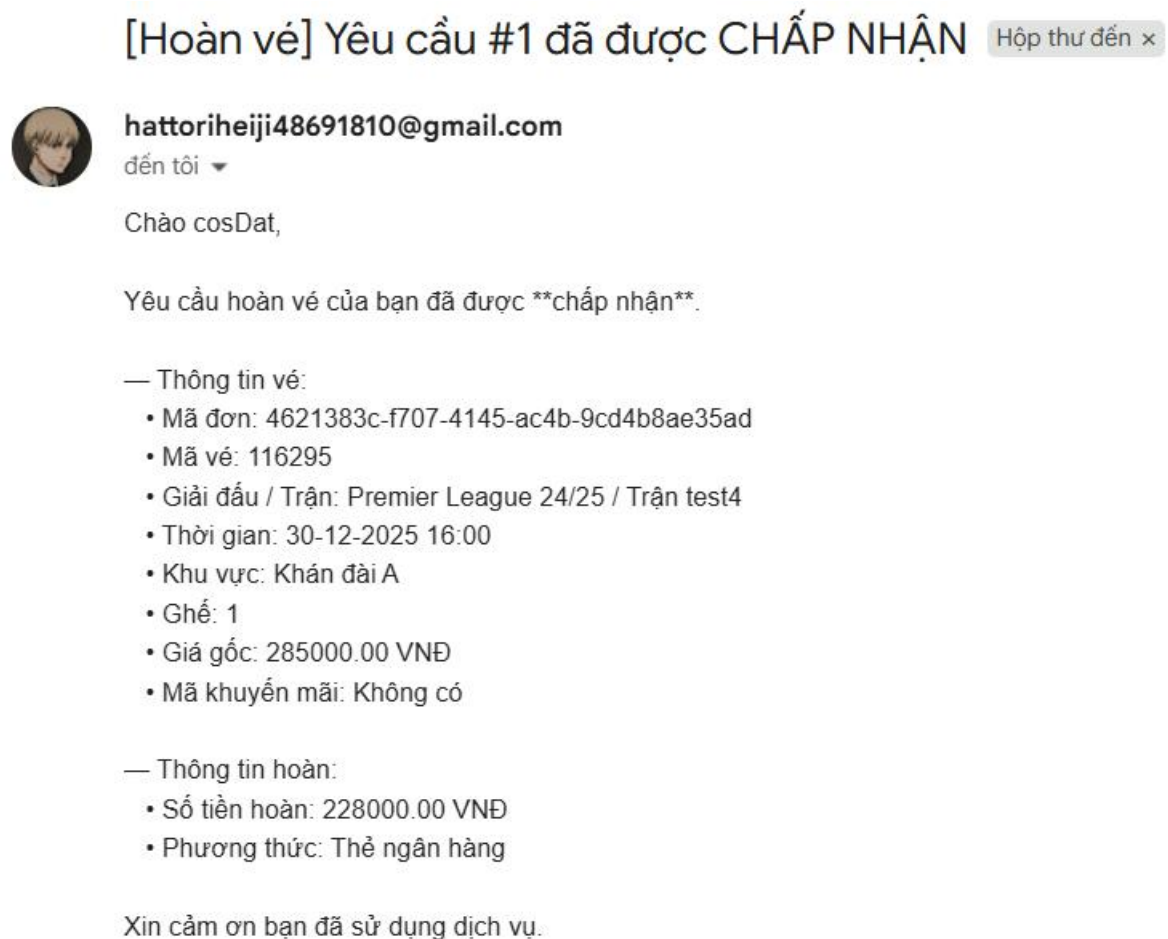
— Thông tin vé:

- Mã đơn: 10234 • Mã vé: 88
- Giải đấu / Trận: V-League 2024 / Hà Nội FC vs HAGL
- Thời gian: 20-05-2024 19:00
- Khu vực: Khán đài A
- Ghế: A-15
- Giá gốc: 300,000 VNĐ

— Thông tin hoàn:

- Số tiền hoàn: 240,000 VNĐ (Đã trừ phí)
- Phương thức: Chuyển khoản

Xin cảm ơn bạn đã sử dụng dịch vụ.



Hình 34: Màn hình email khách hàng nhận được sau khi được hoàn vé thành công

b. Trường hợp Từ chối hoàn vé

Tiêu đề: [Hoàn vé] Yêu cầu #51 đã BỊ TỪ CHỐI

Nội dung: Chào Nguyen Van B,

Yêu cầu hoàn vé của bạn đã bị từ chối. Lý do: Quá thời hạn hoàn vé quy định (trước 24h).

— Thông tin vé:

- Mã đơn: 10235
- Mã vé: 92 ... (Thông tin chi tiết vé) ...

Xin liên hệ bộ phận hỗ trợ nếu bạn cần thêm thông tin.

c. Đánh giá kỹ thuật

Tính nhất quán dữ liệu: Việc sử dụng `transaction.atomic()` bao bọc toàn bộ quá trình cập nhật kho vé và trạng thái đơn hàng đảm bảo không bao giờ xảy ra tình trạng "Tiền đã hoàn nhưng Vé chưa nhả ra kho".

Trải nghiệm người dùng: Thông báo qua Email giúp quy trình minh bạch, khách hàng không cần phải đăng nhập liên tục vào website để kiểm tra trạng thái yêu cầu.

4.3 Kiểm thử tính năng giữ ghế tạm thời và websocket

Hệ thống sử dụng cơ chế phối hợp chặt chẽ giữa Redis và WebSocket:

a. Cơ chế Khóa tài nguyên (Redis Resource Locking)

Sử dụng lệnh `SET key value NX EX time` của Redis.

NX (Not Exists): Chỉ đặt key nếu key đó chưa tồn tại. Đây là chốt chặn quan trọng nhất để ngăn 2 người cùng mua 1 ghế.

EX (Expire): Thiết lập thời gian sống (TTL).

Key format: `seat_lock:{match_id}:{seat_id}`.



Hình 35: Giữ ghế tạm thời cho khách hàng

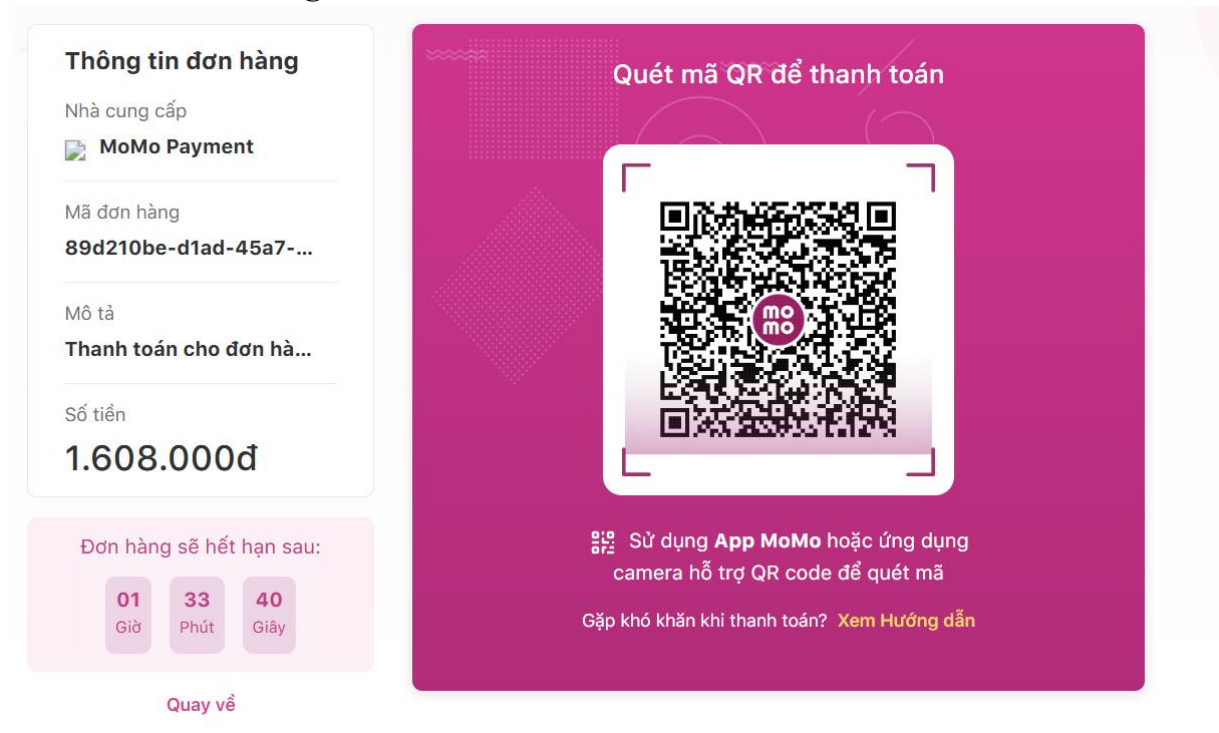
b. Cơ chế Phát tín hiệu (WebSocket Signaling - "Loa phường")

Sử dụng thư viện **Django Channels** để quản lý kết nối song công.

Khi Redis khóa ghế thành công, Backend kích hoạt một sự kiện `group_send` đến nhóm (Room) của trận đấu đó.

Nội dung bản tin (Payload) chứa ID ghế và trạng thái mới, giúp Frontend của hàng nghìn người dùng khác cập nhật giao diện ngay lập tức mà không cần tải lại trang.

4.4 Thanh toán bằng MoMo



Hình 36: Màn hình thanh toán bằng MoMo

Chức năng thanh toán đóng vai trò then chốt trong quy trình thương mại điện tử. Hệ thống tích hợp cổng thanh toán MoMo (môi trường Sandbox) cho phép người dùng thanh toán vé thông qua ứng dụng ví điện tử (quét mã QR). Quy trình bao gồm 2 bước chính:

Khởi tạo giao dịch: Hệ thống gửi thông tin đơn hàng sang MoMo để lấy đường dẫn thanh toán (`payUrl`).

Xử lý kết quả (IPN): MoMo gọi ngược lại (Callback) API của hệ thống để thông báo trạng thái giao dịch (Thành công/Thất bại).



Hình 37: Màn hình sau khi thanh toán MoMo thành công

4.5 Tự động tích điểm khi khách hàng thanh toán

4.5.1 Mục tiêu kiểm thử

Xác nhận hệ thống tự động xử lý chính xác việc cộng điểm thưởng khi thanh toán thành công và hoàn lại điểm khi đơn hàng bị hủy, đảm bảo tính công bằng và chính xác cho hệ thống khách hàng thân thiết (Loyalty Program).

4.5.2 Quy trình nghiệp vụ (Logic hệ thống)

Hệ thống sử dụng cơ chế Django Signals để lắng nghe các thay đổi từ model Payment và Order:

1. Tích điểm: Khi Payment chuyển trạng thái success:

Điểm cơ bản: 1 điểm cho mỗi 10.000 VNĐ.

Hệ số nhân theo hạng: Silver (x1.1), Gold (x1.2), Diamond (x1.5).

Thưởng vượt ngưỡng (Diamond): Tặng thêm 500 điểm cho mỗi 2.000 điểm uy tín tăng thêm (trên mốc 5.000).

2. Hoàn điểm: Khi Order chuyển trạng thái cancelled, hệ thống tự động kiểm tra lịch sử và hoàn lại số điểm khách hàng đã sử dụng để thanh toán đơn hàng đó.

Sử dụng điểm tích lũy

Bạn đang có: **2.144** điểm

Nhập số điểm muốn dùng (VD: 50)

Tỷ lệ: 1 điểm = 1.000 VND

Số tiền được giảm: **-0 VND**

Tổng tiền: 4.470.000 VND (Lưu ý! Kiểm tra thông tin trước khi thanh toán) **+536 điểm**

Thanh toán

Hình 38: Khách hàng được sử dụng điểm tích được khi đặt hàng

User A
Thành viên Vàng

Điểm khả dụng: **2.144**
Điểm xét hạng: **2.311**

Tiến trình thăng hạng

Hạng tiếp theo: **Thành viên Kim Cương**

Tích thêm **2.689** điểm xét hạng nữa để thăng hạng.

Lịch sử điểm thưởng Gần đây nhất

Thời gian	Nội dung	Đơn hàng
26/12/2025 00:10	Tích điểm đơn hàng (Hạng Thành viên Vàng)	#ae8554b1-9abf-48d2-a8f4-a1d94382870e

Hình 39: Màn hình thông tin điểm thưởng của khách hàng

4.5.3 Đánh giá kết quả

Đánh giá kết quả:

- Độ chính xác: Thuật toán tính toán điểm theo hạng thành viên hoạt động chính xác theo tỷ lệ đã thiết lập. Việc sử dụng `int()` giúp làm tròn điểm về số nguyên đúng quy định.
- Hiệu năng: Việc sử dụng `post_save signals` giúp tách biệt logic xử lý điểm ra khỏi logic đặt hàng chính, giúp phản hồi phía người dùng (Frontend) nhanh hơn.
- Tính an toàn: Cơ chế kiểm tra `is_already_refunded` đảm bảo dữ liệu điểm không bị sai lệch khi có lỗi thao tác lặp lại từ phía quản trị viên.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Qua quá trình nghiên cứu và thực hiện đề tài "Xây dựng website đặt vé xem các sự kiện thể thao", nhóm đã đạt được những kết quả quan trọng sau:

Về mặt hệ thống: Đã xây dựng thành công một nền tảng bán vé sự kiện thể thao chuyên nghiệp với quy trình nghiệp vụ hoàn chỉnh: từ khâu tìm kiếm sự kiện, quản lý các chương trình khuyến mãi (Promotion), đặt vé đến thanh toán điện tử (tích hợp MoMo Sandbox) và xử lý quy trình hoàn vé (Refund) với thông báo Email xác nhận.

Về công nghệ xử lý thời gian thực: Đã giải quyết triệt để bài toán tranh chấp tài nguyên (ghế ngồi) trong môi trường chịu tải cao nhờ sự kết hợp giữa cơ chế Locking của Redis và phát tin hiệu Broadcasting của WebSocket. Hệ thống đảm bảo tính nhất quán của dữ liệu và trải nghiệm mượt mà cho người dùng.

Về ứng dụng Trí tuệ nhân tạo:

Đã chứng minh được hiệu quả vượt trội của thuật toán **Random Forest** so với các mô hình truyền thống (Linear Regression, Prophet) trong bài toán định giá động, đặc biệt với dữ liệu có tính phân tầng và cầu cứng.

Hệ thống AI Agent (kiến trúc RAG Hybrid) hỗ trợ hiệu quả nghiệp vụ chăm sóc khách hàng đặc thù, giảm thiểu tình trạng ảo giác và cung cấp thông tin chính xác từ cơ sở dữ liệu.

5.2 Hướng phát triển

Để hệ thống hoàn thiện hơn và có thể triển khai thực tế ở quy mô lớn, nhóm đề xuất các hướng phát triển trong tương lai:

Về kiến trúc: Nâng cấp từ mô hình Client-Server hiện tại sang kiến trúc **Microservices**. Tách biệt các service: Đặt vé, Thanh toán, AI Prediction và Chatbot để tăng khả năng mở rộng độc lập và độ ổn định của hệ thống.

Về nghiệp vụ:

Mở rộng mô hình dữ liệu để hỗ trợ đa dạng các môn thể thao khác nhau (Tennis, Esports...) thay vì tập trung chủ yếu vào một vài môn.

Mở rộng tích hợp thanh toán đa kênh thực tế như ZaloPay, Visa/Mastercard (thông qua Stripe hoặc OnePay) thay vì chỉ sử dụng môi trường Sandbox.

Về công nghệ chuyên sâu:

AI: Sử dụng các mô hình Deep Learning tiên tiến (như LSTM hoặc Transformers) để cải thiện độ chính xác dự báo khi dữ liệu lịch sử tích lũy đủ lớn theo thời gian (Time-series data).

WebSocket: Nâng cấp tính năng thời gian thực từ mức độ "Loa phờng" (Server phát - Client nhận) sang tương tác hai chiều (Bi-directional), cho phép các tính năng như đấu giá vé VIP trực tiếp hoặc chat giữa các cổ động viên.

TÀI LIỆU THAM KHẢO

Django Software Foundation. (2024). *Django (Version 4.2) [Computer software]*.
<https://www.djangoproject.com/>

Redis Ltd. (2024). *Redis Documentation*. <https://redis.io/documentation>

Momo. (2024). *Momo Payment Gateway Integration Guide*.
<https://developers.momo.vn/>